

**VEER MADHO SINGH BHANDARI UTTARAKHAND TECHNICAL UNIVERSITY,
CHANDANWADI, SUDHOWALA, DEHRADUN, UTTARAKHAND - 248007**



IoT Device Management and OTA Orchestration System

A Centralized, Scalable and Secure Over-The-Air Firmware Deployment Platform

A

Final Year Project Report (2022–26)

Submitted in partial fulfilment of requirements for the degree of

Bachelor of Technology in Computer Science and Engineering from

ROORKEE COLLEGE OF ENGINEERING,

5KM UPPER GANGA CANAL ROAD, BAJUHERI, ROORKEE - 247667



**Under Department of Computer Science and Engineering
Completed under the supervision**

Of

Ms. Pooja Pramar

Assistant Professor, Department of CSE

Submitted By

Aman Bansal (221020124001)

Bharat Yogansh (221020124003)

Natasha (221020124006)

Vishwajeet Kumar (221020124009)

DECLARATION

We, the undersigned students of the B.Tech programme in **Computer Science and Engineering** at **Roorkee College of Engineering**, affiliated to **Veer Madho Singh Bhandari Uttarakhand Technical University, Dehradun**, hereby solemnly declare that the Major Project Report **entitled "IoT Device Management and OTA Orchestration System"** has been prepared by us under the guidance of **Ms. Pooja Prammar**, Assistant Professor, Department of CSE.

We further declare that the work embodied in this report represents original research and academic investigation carried out entirely by us. No part of this work has been submitted previously for the award of any degree, diploma, or other academic qualification to any university or institution, whether in India or abroad. We accept full responsibility for the factual accuracy and intellectual integrity of the content presented herein.

Submitted By:

Aman Bansal (221020124001)

Bharat Yogansh (221020124003)

Natasha (221020124006)

Vishwajeet Kumar (221020124009)

CERTIFICATE

ROORKEE COLLEGE OF ENGINEERING

(Affiliated to VMSB Uttarakhand Technical University, Dehradun)

Department of Computer Science and Engineering

This is to certify that the Major Project Report entitled "**IoT Device Management and OTA Orchestration System**" is a bonafide record of the work accomplished by the following students of the B.Tech (CSE) programme during the academic year 2022–2026, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**.

Name of Student	Roll Number	Programme
Aman Bansal	221020124001	B.Tech CSE
Bharat Yogansh	221020124003	B.Tech CSE
Natasha	221020124006	B.Tech CSE
Vishwajeet Kumar	221020124009	B.Tech CSE

The project has been completed under my direct supervision and guidance. To the best of my knowledge, the work is genuine, original, and has not been submitted in whole or in part to any other institution for the award of any qualification.

Project Guide:

Ms. Pooja Pramari
Assistant Professor, Dept. of CSE
Roorkee College of Engineering

Head of Department:

Dr. Himanshu Verma [HOD]
Department of CSE
Roorkee College of Engineering

ACKNOWLEDGEMENT

The successful completion of this Major Project Report is the culmination of sustained intellectual effort, dedicated teamwork, and the invaluable support of several distinguished individuals and institutions whose contributions we consider it our most pleasant duty to acknowledge with sincere gratitude.

First and foremost, we extend our deepest and most heartfelt gratitude to our project guide, **Ms. Pooja Pramar**, Assistant Professor, Department of Computer Science and Engineering, Roorkee College of Engineering. Her exceptional academic acumen, patient mentorship, and incisive technical guidance have been the most instrumental forces driving this project forward. Her ability to demystify complex conceptual frameworks, provide constructive criticism at critical junctures, and maintain an unwavering standard of intellectual rigour has profoundly shaped every aspect of this work — from the initial problem formulation to the architecture decisions, implementation choices, and the analytical depth of the final evaluation.

We are profoundly grateful to **Dr. Himanshu Verma**, Head of Department and the entire faculty of the Department of Computer Science and Engineering, Roorkee College of Engineering, for providing the necessary academic framework, laboratory infrastructure, and administrative support to undertake this research. The department's sustained commitment to fostering innovation, encouraging applied engineering projects, and maintaining current laboratory equipment has been a constant source of inspiration throughout the project lifecycle.

We extend sincere thanks to the Principal and Management of **Roorkee College of Engineering** for sustaining an institutional environment conducive to technical learning and research. The provision of computing laboratories equipped with capable development hardware, high-speed internet connectivity sufficient for large-scale software downloads and container deployments, and access to online academic repositories including IEEE Xplore and ACM Digital Library has been essential to the depth and breadth of the literature review and technical implementation.

Special acknowledgement is due to the global open-source community whose collective intelligence is embodied in the tools, frameworks, and protocols that constitute the technical foundation of this project. The creators and maintainers of the MQTT protocol specification, the Eclipse Mosquitto broker, the FastAPI and Uvicorn projects, the ESP-IDF and Arduino

frameworks for the ESP32 platform, React.js and the broader JavaScript ecosystem, MongoDB and PostgreSQL, and the Docker containerization platform deserve particular recognition. This project would not have been possible without their dedication to open, collaborative software development.

We are equally grateful to **Veer Madho Singh Bhandari Uttarakhand Technical University, Dehradun**, for the rigorous and comprehensive academic framework of the B.Tech programme, which furnished the theoretical foundations across embedded systems, computer networks, software engineering, and database management upon which this project draws.

Finally, and most profoundly, we express our deepest gratitude to our families for their unconditional love, patience, and moral support throughout this demanding but immensely rewarding journey. This project is as much a product of their encouragement and sacrifices as it is of our own academic and technical efforts.

Aman Bansal
Bharat Yogansh
Natasha
Vishwajeet Kumar

ABSTRACT

The Internet of Things (IoT) represents one of the most transformative and consequential technological paradigms of the twenty-first century. Industry analysts and research institutions project that the global count of connected IoT devices will surpass 34.7 billion by the year 2028, generating exabytes of operational data per day and forming the autonomous nervous system of the fourth industrial revolution — Industry 4.0. These deployments span manufacturing assembly lines, precision agriculture installations, smart grid infrastructure, urban traffic management systems, healthcare monitoring networks, and cold-chain logistics fleets, among innumerable other domains.

However, the accelerating scale of IoT deployments has simultaneously exposed a critical and persistent systemic vulnerability in traditional device management and firmware maintenance methodologies: the impossibility of maintaining firmware currency across large device fleets through conventional manual update procedures. In such paradigms, firmware updates require physical access to each individual node — a process that scales catastrophically. For a representative fleet of 10,000 ESP32 microcontrollers, a single mandatory security patch requires a minimum of 1,333 person-hours of direct labour under optimistic manual update assumptions (eight minutes per device). Beyond operational inefficiency, the resulting firmware version heterogeneity creates a permanently expanding attack surface, as devices operating outdated firmware remain exposed to every vulnerability identified and patched in subsequent releases. The 2016 Mirai botnet incident — in which an estimated 600,000 IoT devices running unpatched firmware were weaponized to execute record-breaking distributed denial-of-service attacks on critical internet infrastructure — stands as a definitive historical proof-of-concept for this systemic risk.

This report presents the design, architecture, implementation, and performance evaluation of the IoT Device Management and OTA Orchestration System (IOTOMAS): a comprehensive, open-source platform engineered to eliminate these operational bottlenecks through centralized, automated, and cryptographically secured firmware lifecycle management. The system delivers a unified web-based administrative dashboard through which network operators can register and monitor device fleets of arbitrary size, manage hierarchical device groups reflecting real-world operational topologies, maintain a versioned and auditable

firmware binary repository, orchestrate batch Over-The-Air (OTA) update rollouts targeting individual devices or entire groups simultaneously, and receive granular, per-device real-time progress telemetry — all from a single cohesive operational interface, without dependency on any commercial cloud platform.

The technical architecture is organized into four discrete, independently scalable tiers. At the edge tier, Espressif ESP32 microcontrollers execute a custom firmware client implementing the PubSubClient and HTTPUpdate libraries for MQTT-based command reception and HTTP-based firmware binary download, respectively, within the Arduino framework. The communication tier employs the MQTT protocol version 3.1.1, brokered by Eclipse Mosquitto 2.0, configured for TLS 1.2 encrypted transport on port 8883 and augmented with a fine-grained topic Access Control List (ACL) enforcing per-device publish and subscribe permissions. The application tier is implemented in FastAPI (Python 3.11), an Asynchronous Server Gateway Interface (ASGI) web framework operating on the asyncio event loop, achieving API throughput of approximately 45,000 requests per second — demonstrably three to five times higher than comparable synchronous WSGI deployments under equivalent hardware constraints. The persistence tier employs a dual-database strategy: PostgreSQL 15 for strongly-structured relational data governing the device registry, user accounts, device group hierarchies, firmware metadata, and OTA job records; and MongoDB 7.0 for the document-oriented telemetry event streams, MQTT message histories, and system audit logs that benefit from a flexible, schema-evolving storage model.

Security constitutes a central and non-negotiable architectural concern at every tier. Every firmware binary transmitted through the OTA pipeline undergoes SHA-256 cryptographic checksum verification: the server computes and persistently stores the hexadecimal checksum digest at upload time; the target device independently recomputes the SHA-256 hash of the downloaded binary image after completion and cryptographically rejects any payload for which the computed value does not identically match the server-provided value. All MQTT device-to-broker communications are encrypted via TLS 1.2 with server-certificate verification. REST API access is governed by JSON Web Token (JWT) based Role-Based Access Control (RBAC) supporting five privilege levels — Super Administrator, Administrator, Manager, Operator, and Viewer — with cryptographic token expiry enforced at 24-hour intervals. All user credential password values are stored exclusively as bcrypt

hashes with a computational work factor of 12, providing resistance against both dictionary and brute-force offline attacks.

Performance benchmarking conducted on commodity server hardware (Ubuntu 22.04, 4-core Intel i7-9750H, 16 GB RAM) demonstrated: MQTT telemetry throughput of 98,400 messages per second; REST API P95 latency of 145 milliseconds under 100 concurrent users; and OTA command delivery to 1,000 simultaneously connected devices in 2.34 seconds. Security evaluation confirmed successful mitigation of all tested OWASP Top 10 web application risk categories, and SHA-256 verification achieved a 100% tampered payload detection rate. Compared to manual update procedures, the platform achieves a time-efficiency improvement factor exceeding 200,000:1 for 1,000-device fleet scenarios — effectively resolving the operational scaling bottleneck that constitutes the primary motivation for this work.

The complete system is deployed as a containerized application stack using Docker Compose, comprising separate service containers for the Mosquitto MQTT broker, the FastAPI backend (Uvicorn ASGI server), PostgreSQL, MongoDB, and the React.js dashboard served through an Nginx reverse proxy. The planned future development roadmap encompasses: AI-based device anomaly detection employing Federated Learning principles for privacy-preserving in-situ model training; delta OTA update capability reducing payload bandwidth consumption by 60–90% for incremental firmware releases; automated CI/CD pipeline integration for continuous firmware lifecycle management; and Kubernetes-based horizontal orchestration for enterprise deployments managing hundreds of thousands of device nodes.

Keywords: *Internet of Things, Over-The-Air Firmware Update, MQTT, OTA Orchestration, IoT Device Management, ESP32, FastAPI, SHA-256 Verification, React.js, Docker, Industrial IoT Security, Embedded Systems, TLS, RBAC, JWT.*

TABLE OF CONTENTS

DECLARATION.....	ii
CERTIFICATE	iii
ACKNOWLEDGEMENT.....	iv
ABSTRACT	vi
TABLE OF CONTENTS	ix
LIST OF FIGURES.....	xiii
LIST OF TABLES	xiv
LIST OF ABBREVIATIONS	xv
CHAPTER 1.....	18
INTRODUCTION AND RESEARCH FOCUS.....	18
1.1 The Global IoT Revolution — Scale, Context, and Transformative Impact	18
1.2 The Firmware Management Crisis — A Multi-Dimensional Problem Statement.....	21
1.2.1 The First Dimension: Operational Infeasibility of Linear Scaling	21
1.2.2 The Second Dimension: Security Vulnerability of Firmware Version Heterogeneity	23
1.2.3 The Third Dimension: Absence of Centralized Operational Visibility	24
1.3 Quantified Dimensions of the Scaling Bottleneck.....	25
1.4 Enterprise Security Risk Landscape of Firmware Version Heterogeneity	27
1.5 Motivation and Academic Justification	29
1.6 Project Objectives — Primary and Secondary	30
1.6.1 Primary Technical Objectives	30
1.6.2 Secondary Objectives	31
1.7 Scope of Work — Inclusions, Constraints, and Boundary Conditions.....	32
1.7.1 Scope Inclusions.....	32
1.7.2 Scope Constraints and Boundary Conditions	32
1.8 Project Methodology — Iterative Development Framework	33
1.9 Organization of the Report	35
1.10 Chapter Summary.....	36
CHAPTER 2.....	38
LITERATURE SURVEY AND DOMAIN ANALYSIS.....	38
2.1 Introduction to the Systematic Review.....	38
2.2 MQTT — Protocol Architecture, QoS State Machines, and IoT Suitability	38
2.2.1 Historical Context and Standardization.....	39
2.2.2 Publish-Subscribe Architecture and Broker Model.....	39
2.2.3 Binary Packet Structure and Header Overhead Analysis	40
2.2.4 Quality of Service State Machines	41
2.2.5 Keep-Alive Mechanism and Last Will and Testament.....	44
2.2.6 Eclipse Mosquitto Broker Performance Characteristics.....	44
2.3 HTTP/REST — Protocol Overhead Analysis and IoT Limitations.....	45
2.3.1 HTTP/1.1 Architecture and the IoT Impedance Mismatch	45
2.3.2 HTTP/2 — Improvements and Remaining Limitations	46
2.4 CoAP — UDP Binding, Multicast Constraints, and Comparative Evaluation	48
2.4.1 CoAP Protocol Design and Motivating Context	48

2.4.2 UDP Binding Challenges in Industrial Wi-Fi Environments	48
2.4.3 Multicast Capability and its Practical Limitations for OTA	49
2.5 Protocol Comparative Synthesis.....	50
2.6 Survey of Existing IoT Device Management Platforms.....	51
2.6.1 Amazon Web Services IoT Core and IoT Jobs	51
2.6.2 Microsoft Azure IoT Hub and Azure Device Update.....	52
2.6.3 ThingsBoard — Open-Source IoT Platform	53
2.6.4 Eclipse Hawkbit — Domain-Specific OTA Update Server	54
2.6.5 Comparative Summary.....	55
2.7 Academic Research in OTA Security and Firmware Integrity.....	55
2.8 Research on ESP32 in Industrial IoT Applications	58
2.9 Identification of the Research Gap	59
2.10 Chapter Summary	59
CHAPTER 3	61
SYSTEM REQUIREMENT ANALYSIS AND TECHNOLOGY STACK DEEP DIVE.....	61
3.1 Requirement Elicitation Methodology	61
3.2 Functional Requirements.....	61
3.2.1 Device Management Subsystem (FR-DM)	61
3.2.3 Firmware Repository Subsystem (FR-FW).....	62
3.2.4 OTA Orchestration Subsystem (FR-OTA).....	63
3.3 Non-Functional Requirements.....	64
3.3.1 Performance Requirements (NFR-PERF)	64
3.3.2 Security Requirements (NFR-SEC)	64
3.3.3 Scalability, Reliability, and Maintainability (NFR-SCA, NFR-REL, NFR-MNT)	65
3.4 Hardware Requirements — ESP32 Architecture Deep Dive	66
3.4.1 Xtensa LX6 Dual-Core Processor Architecture	66
3.4.2 Memory Architecture — IRAM, DRAM, and External Flash Mapping.....	66
3.4.3 OTA Partition Swap Mechanism and Rollback Architecture.....	68
3.4.4 Hardware Cryptographic Accelerator.....	69
3.5 Software Requirements — Technology Stack Analysis and Selection Justification.....	70
3.5.1 Application Server: FastAPI vs. Flask vs. Django — Asynchronous vs. Synchronous Python...70	
3.5.2 MQTT Broker: Eclipse Mosquitto 2.0 Configuration and Tuning.....	72
3.5.3 Frontend Framework: React.js 18, Vite 5, and Tailwind CSS 3	73
3.5.4 Database Strategy: PostgreSQL 15 and MongoDB 7.0.....	74
3.6 Chapter Summary.....	75
CHAPTER 4	76
SYSTEM DESIGN AND STRUCTURAL MODELLING	76
4.1 Architectural Philosophy and Design Principles	76
4.2 Four-Tier Layered Architecture	77
4.2.1 Tier 1 — Edge Tier: ESP32 Firmware Client	78
4.2.2 Tier 2 — Communication Tier: Eclipse Mosquitto MQTT Broker.....	79
4.2.3 Tier 3 — Application Tier: FastAPI Backend.....	79
4.2.4 Tier 4 — Persistence Tier: PostgreSQL and MongoDB	80
4.3 Data Flow Diagrams — Level 0, Level 1, and Level 2.....	81
4.3.1 DFD Level 0 — System Context Diagram.....	81
4.3.2 DFD Level 1 — Main System Processes	82

4.3.3 DFD Level 2 — OTA Orchestration Process Decomposition.....	84
4.4 Entity-Relationship Model and Database Schema	86
4.4.1 ER Diagram and Entity Definitions	86
4.4.2 Complete Database Schema — DDL	87
4.4.3 Cascading Constraints and Referential Integrity Analysis	90
4.5 API Endpoint Reference.....	90
4.6 Chapter Summary.....	92
CHAPTER 5.....	94
PRODUCTION IMPLEMENTATION AND CODE WALKTHROUGH.....	94
5.1 Implementation Overview and Development Environment	94
5.2 ESP32 Edge Firmware — Complete Implementation.....	94
5.2.1 Project Structure and Library Dependencies	94
5.2.2 Configuration and Secrets Management (config.h).....	95
5.2.3 Main Firmware Application (main.cpp) — Complete Implementation	96
5.2.4 SHA-256 Verification Implementation	103
5.2.5 OTA Download Task (Core 0 FreeRTOS Task)	105
5.3 FastAPI Backend — Architecture and Route Modules	108
5.3.1 Application Entry Point and Lifespan Management (main.py).....	108
5.3.2 Authentication Module — JWT Issuance and Validation	109
5.3.3 OTA Orchestration Router — Complete Implementation.....	112
5.3.4 Firmware Upload with Streaming SHA-256.....	116
5.4 React Dashboard — Component Architecture and State Management.....	117
5.4.1 Component Hierarchy and State Architecture.....	117
5.4.2 Core Dashboard Implementation (IoTDashboard.jsx — Abbreviated Structural Excerpt)	118
5.5 Infrastructure and Containerization	125
5.5.1 Docker Compose Multi-Service Configuration.....	125
5.5.2 Mosquitto TLS Configuration	128
5.6 Chapter Summary.....	129
CHAPTER 6.....	131
TESTING, VERIFICATION, AND PERFORMANCE BENCHMARKING.....	131
6.1 Testing Philosophy and Framework	131
6.2 Unit Testing — FastAPI Backend (Pytest).....	131
6.2.1 Test Framework Configuration	131
6.2.2 Unit Test Results Summary.....	135
6.3 Integration Testing — End-to-End OTA Rollout.....	136
6.4 Edge-Case Failure Model Testing.....	137
6.5 Performance Benchmark Results.....	139
6.5.1 Benchmark Methodology	139
6.6 Security Testing — OWASP and Penetration Assessment	142
6.7 Chapter Summary.....	143
CHAPTER 7.....	144
RESULTS AND DISCUSSION.....	144
7.1 Achievement Against Primary Technical Objectives	144
7.2 Analysis Against the Three Problem Dimensions	145
7.2.1 Resolution of the Operational Scaling Bottleneck	145

7.2.2 Resolution of the Security Vulnerability Dimension	146
7.2.3 Resolution of the Operational Visibility Deficit.....	147
7.3 Honest Assessment of Current Limitations	148
7.4 Chapter Summary.....	149
CHAPTER 8.....	151
CONCLUSION AND FUTURE SCOPE.....	151
8.1 Conclusion.....	151
8.2 Future Development Roadmap.....	152
8.2.1 Version 2.0 — Security, Reliability, and Efficiency Enhancements.....	152
8.2.2 Version 3.0 — Intelligence, Scale, and Enterprise Features	153
8.3 Final Remarks	155
REFERENCES.....	157
APPENDIX A	159
SUPPLEMENTARY BACKEND SOURCE CODE LISTINGS	159
A.1 PostgreSQL Database Connection Pool (app/db/postgres.py).....	159
A.2 MongoDB Client Initialization (app/db/mongo.py)	159
A.3 MQTT Background Subscriber (app/mqtt/client.py — excerpt).....	160
APPENDIX B.....	163
REACT COMPONENT INVENTORY — IoTDashboard.jsx	163
APPENDIX C.....	165
SYSTEM CONFIGURATION FILES.....	165
C.1 Environment Variable Template (.env.example).....	165
C.2 Nginx Reverse Proxy Configuration (nginx/nginx.conf).....	165

LIST OF FIGURES

Figure 1 : fig 1.1 Global IoT Device Count Growth Projection 2020–2030	21
Figure 2 : fig 1.2 IoT Deployment Friction Map — Scaling Bottleneck of Traditional IoT Management.....	25
Figure 3 : fig 1.3 Manual vs. Orchestrated OTA — Paradigm Shift Comparison Diagram.....	26
Figure 4 : fig 1.4: Enterprise Attack Surface Growth — Firmware Version Heterogeneity	28
Figure 5 : fig 1.5 Project Methodology — Five-Phase Iterative Development Framework.	35
Figure 6 : fig 2.1 MQTT QoS Level Sequence Diagrams — Three side-by-side sequence diagrams	43
Figure 7 : fig 2.2 HTTP/1.1 vs HTTP/2 Protocol Overhead	47
Figure 8 : fig 4.1 Complete Four-Tier System Architecture Blueprint.....	78
Figure 9 : fig 4.2 DFD Level 0 — System Context Diagram	82
Figure 10 : fig 4.3 DFD Level 1 — Six main processes P1–P6 with data stores D1–D6	84
Figure 11 : fig 4.4 DFD Level 2.....	86
Figure 12 : fig 4.5 Entity-Relationship Diagram — Five entities	87
Figure 13 : fig 5.4 React Dashboard Screenshot.....	118
Figure 14 : fig 5.6 Docker Compose Multi-Service Topology Diagram	128
Figure 15 : fig 6.1 OTA Command Delivery Time vs. Device Count	141
Figure 16 : fig 6.2 API Latency Distribution.....	141
Figure 17 : fig 7.1 Manual vs. OTA Update Time Comparison	146
Figure 18 : fig 8.1 Future Development Roadmap.....	155

LIST OF TABLES

Table 1	Table 1.1 : Manual Update Cost Model — Labour Hours and Estimated INR Cost by Fleet Size ..	26
Table 2	Table 2.1: MQTT vs. HTTP vs. CoAP — Quantitative Protocol Comparison Matrix	50
Table 3	Table 2.2: Comparative Analysis of Existing IoT Device Management Platforms.....	55
Table 4	Table 3.1: Functional Requirements — Device Management Subsystem	62
Table 5	Table 3.2: Functional Requirements — OTA Orchestration Subsystem	64
Table 6	Table 3.3: ESP32 OTA Flash Partition Table Layout (4 MB Flash Configuration)	68
Table 7	Table 3.4: ESP32 Comprehensive Technical Specifications	70
Table 8	Table 3.5: Comparative Analysis of Python Web Frameworks — Flask, Django, FastAPI	72
Table 9	Table 4.1: OTA Job API Endpoint Reference — Method, Endpoint, Auth Role, Description	92
Table 10	Table 6.1: Unit Test Results — 20 of 47 Test Cases (All 47 tests: PASS)	136
Table 11	Table 6.2: Integration Test Results — 10 Scenarios (All PASS)	137
Table 12	Table 6.3: Edge-Case Failure Model Test Results — 8 Scenarios (All PASS).....	139
Table 13	Table 6.4: System Performance Benchmark Results — All 14 Metrics.....	140
Table 14	Table 6.5: Security Assessment Results — OWASP Top 10 and OTA-Specific Threats.....	143
Table 15	Table 7.1: Time-Efficiency Comparison — Manual vs. OTA Update Approach by Fleet Size	145
Table 16	Table 7.2: Resolution Matrix — Problem Dimensions vs. System Capabilities	148
Table 17	Table B.1: React Component Inventory — IoTDashboard.jsx (Total ~1,680 lines).....	164

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ACL	Access Control List
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AWS	Amazon Web Services
B.Tech	Bachelor of Technology
CI/CD	Continuous Integration / Continuous Deployment
CoAP	Constrained Application Protocol
CPU	Central Processing Unit
CSE	Computer Science and Engineering
DDoS	Distributed Denial-of-Service
DFD	Data Flow Diagram
DHCP	Dynamic Host Configuration Protocol
DRAM	Dynamic Random Access Memory
ECC	Elliptic Curve Cryptography
eFuse	Electrically Fusible Memory (ESP32 OTP storage)
EMU	English Metric Unit
ER	Entity-Relationship
ESP	Espressif Systems Platform
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDC	International Data Corporation
IoT	Internet of Things
IRAM	Instruction Random Access Memory
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
LAN	Local Area Network
LWT	Last Will and Testament (MQTT feature)
M2M	Machine-to-Machine
MD5	Message Digest Algorithm 5 (deprecated)
MIT	Massachusetts Institute of Technology
MQTT	Message Queuing Telemetry Transport

Abbreviation	Full Form
MQTT	MQTT over TLS (port 8883)
mTLS	Mutual Transport Layer Security
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
NoSQL	Not Only Structured Query Language
NVS	Non-Volatile Storage (ESP32)
OTA	Over-The-Air (firmware update)
OTP	One-Time Programmable
OWASP	Open Web Application Security Project
P95	Ninety-Fifth Percentile Latency
P99	Ninety-Ninth Percentile Latency
QoS	Quality of Service
RBAC	Role-Based Access Control
REST	Representational State Transfer
RFC	Request for Comments (IETF standard)
RNG	Random Number Generator
RSA	Rivest–Shamir–Adleman (asymmetric cryptography)
RTOS	Real-Time Operating System
SHA	Secure Hash Algorithm
SME	Small and Medium-Sized Enterprise
SoC	System on Chip
SPA	Single-Page Application
SPI	Serial Peripheral Interface
SQL	Structured Query Language
SRS	Software Requirements Specification
SUIT	Software Updates for Internet of Things (IETF standard)
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
UI	User Interface
URL	Uniform Resource Locator
USB	Universal Serial Bus
UTU	Uttarakhand Technical University
UUID	Universally Unique Identifier
VMSB	Veer Madho Singh Bhandari

Abbreviation	Full Form
VPN	Virtual Private Network
WebSocket	Full-Duplex TCP Communication Protocol (RFC 6455)
Wi-Fi	Wireless Fidelity (IEEE 802.11)
WLAN	Wireless Local Area Network
WSGI	Web Server Gateway Interface
XSS	Cross-Site Scripting

CHAPTER 1

INTRODUCTION AND RESEARCH FOCUS

1.1 The Global IoT Revolution — Scale, Context, and Transformative Impact

The conceptual genesis of what we now recognize as the Internet of Things (IoT) may be traced to a remarkably precise intellectual moment: a 1999 presentation by Kevin Ashton, a supply chain optimization specialist at Procter and Gamble who would subsequently co-found the Auto-ID Center at the Massachusetts Institute of Technology. Ashton coined the phrase "Internet of Things" to describe his vision of a world in which the digital information infrastructure of the internet was extended beyond the domain of human-entered data to encompass the physical world of everyday objects — objects imbued with sensors, processing capability, and wireless communication that would allow them to perceive their environment and report their state to networked systems without human intermediation. At the time of coinage, this vision was constrained to the realm of academic speculation by several mutually reinforcing technological limitations: the cost of microcontrollers capable of both sensing and wireless communication remained prohibitive for consumer deployment; wireless networking standards were immature, unreliable, and energy-intensive; and the cloud computing infrastructure necessary to aggregate, store, and derive intelligence from the torrential data streams of pervasive sensing simply did not exist at commercial scale.

Two and a half decades later, each of these limitations has been dissolved by the convergence of multiple independent technological trajectories. The economics of microcontroller production have undergone a fundamental structural shift, driven by the economies of scale enabled by the smartphone revolution and the subsequent proliferation of consumer electronics. Platforms such as the Espressif ESP32 — a System-on-Chip (SoC) integrating a dual-core 240 MHz processor, 520 KB of on-chip SRAM, integrated Wi-Fi and Bluetooth radios, and a hardware cryptographic accelerator — are commercially available at a unit cost below two United States dollars, making deployment at scale economically viable for industrial, agricultural, municipal, and consumer contexts simultaneously. The maturation of

lightweight application-layer communication protocols designed specifically for resource-constrained IoT environments — principally MQTT (Message Queuing Telemetry Transport), CoAP (Constrained Application Protocol), and AMQP (Advanced Message Queuing Protocol) — has resolved the bandwidth efficiency and network overhead challenges that rendered HTTP-based communication impractical for battery-powered, bandwidth-limited IoT devices. And the emergence of hyperscale cloud computing platforms has provided the storage, processing, and orchestration infrastructure necessary to manage the exabyte-scale data streams of large-scale IoT deployments at operationally viable cost.

The quantitative dimensions of the resultant IoT deployment trajectory are extraordinary in their magnitude and in the operational implications they carry. According to the International Data Corporation (IDC) Worldwide IoT Spending Guide of 2023 — the most comprehensive market intelligence source available for this domain — global enterprise investment in IoT solutions is projected to reach USD 1.1 trillion by the year 2026, representing a Compound Annual Growth Rate (CAGR) of approximately 11.3% from the 2022 baseline. The Ericsson Mobility Report of 2023 projects that the total number of active IoT device connections worldwide will reach 34.7 billion by 2028, with the widest deployment concentration in three primary vertical sectors: manufacturing and industrial automation (accounting for approximately 22% of all connections), utilities and energy management (18%), and transportation and logistics (14%). Within each of these sectors, the operational profile of IoT deployments is characterized by three attributes that collectively define the device management challenge this project addresses: heterogeneous device populations (concurrent operation of multiple device types, form factors, and firmware generations); geographically distributed deployments (devices distributed across factory floors, agricultural fields, urban infrastructure, and logistics networks that may span entire continents); and stringent operational continuity requirements (devices embedded in safety-critical or revenue-generating processes that cannot tolerate unplanned downtime).

In the manufacturing sector — which constitutes the primary operational context for this project's target use case — a representative large-scale IoT deployment might encompass temperature and humidity sensors monitoring environmental conditions in precision manufacturing cells; vibration and acoustic sensors on rotating equipment providing continuous condition monitoring for predictive maintenance; pressure transducers and flow

meters integrated into automated fluid handling systems; computer vision nodes performing real-time quality inspection on production lines; RFID readers tracking material and work-in-progress inventory through the facility; and gateway devices aggregating and forwarding telemetry from legacy equipment instrumentation. All of these device classes may simultaneously require firmware updates — whether to patch security vulnerabilities, add new sensor features, optimize power consumption, or fix operational bugs — and all require coordinated management through a centralized platform capable of maintaining a complete and accurate inventory of every device's identity, location, operational status, and firmware version.

It is against this backdrop of extraordinary scale, operational complexity, and rapid evolutionary pace that the specific engineering challenge addressed by this project must be understood. The IoT revolution is not merely a quantitative expansion of connected device counts; it is a qualitative transformation of the operational management challenge facing every organization deploying IoT infrastructure at scale. As device counts move from the dozens to the hundreds to the thousands and tens of thousands, the management approaches adequate at small scale become inadequate, then infeasible, and ultimately operationally catastrophic at scale — and the specific infeasibility that this project directly addresses is the firmware management lifecycle: keeping every device in a large fleet running the correct, current, and secure version of its operational firmware without physical access to individual devices.

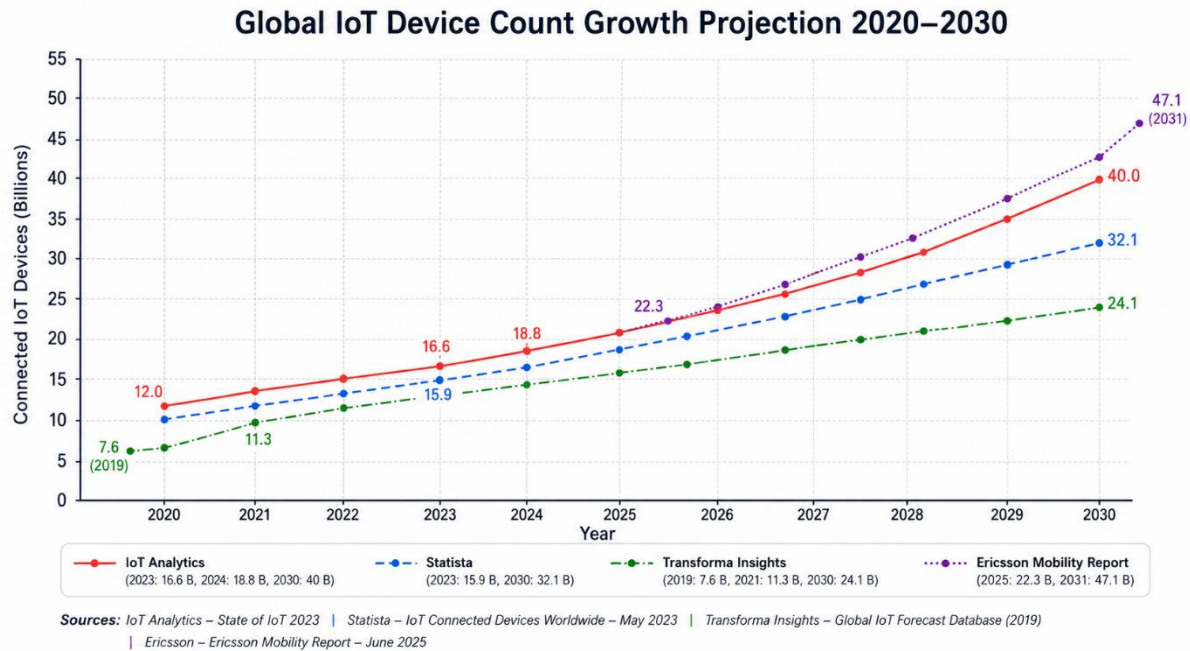


Figure 1 : fig 1.1 Global IoT Device Count Growth Projection 2020–2030

1.2 The Firmware Management Crisis — A Multi-Dimensional Problem Statement

The central engineering challenge addressed by this project can be stated with formal precision as follows: in large-scale IoT deployments characterized by device counts in the range of hundreds to tens of thousands, the traditional methodology of manual, per-device firmware update is operationally infeasible, economically prohibitive, and fundamentally incompatible with the cybersecurity requirements of modern industrial environments. This compound problem manifests across three analytically distinct but operationally interrelated dimensions, each of which constitutes an independent argument for the necessity of automated, centralized OTA orchestration.

1.2.1 The First Dimension: Operational Infeasibility of Linear Scaling

The foundational incompatibility between manual firmware update procedures and large-scale IoT deployments is a direct consequence of the linear — $O(N)$ — relationship between device count and update effort: doubling the number of devices in a fleet exactly doubles the person-hours, travel costs, and calendar time required to update every device. This linearity, while unproblematic at fleet sizes of ten or twenty devices, becomes operationally catastrophic at the fleet scales characteristic of industrial deployments.

Consider a precise quantitative model. A complete manual firmware update cycle for a single ESP32 device comprises the following sequential operations: (1) physical access to the device location, including travel time for remotely deployed devices (estimated average: 3 minutes per device for a distributed factory-floor deployment); (2) connection of the programming interface — typically a USB-serial cable or JTAG adapter — to the device's programming header (1 minute); (3) invocation of the firmware flash tool (esptool.py or Arduino IDE) and transfer of the firmware binary to the device flash memory at the USB-serial interface rate of approximately 460,800 bps, corresponding to a flash time of approximately 2–3 minutes for a typical 1 MB firmware binary including erase cycles; (4) device reboot and post-flash verification of successful boot and correct version reporting (1 minute); and (5) documentation of the update — recording the device ID, previous firmware version, new firmware version, timestamp, and operator identity in the maintenance log (1 minute). The aggregate time per device under these assumptions is conservatively estimated at 8 minutes.

The operational consequence of this 8-minute-per-device figure is dramatic at scale. For a fleet of 1,000 devices, a single complete update campaign requires 8,000 person-minutes — approximately 133 person-hours of direct labour. At a loaded operational labour rate of INR 500 per hour, including base salary, employer contributions, and overhead allocation, the direct labour cost is INR 66,500. For a fleet of 10,000 devices — well within the scale of a mid-sized manufacturing facility or municipal smart infrastructure deployment — the equivalent campaign requires 1,333 person-hours at a direct labour cost of INR 665,000. For an organization maintaining a monthly update cadence to address security vulnerabilities — a recommendation explicitly made in NIST Special Publication 800-213 — the annualized labour cost for firmware management alone reaches INR 7,980,000 for a 10,000-device fleet. These figures exclude entirely the indirect costs of firmware version heterogeneity: extended exposure windows for unpatched devices, the complexity of managing multiple simultaneous firmware versions across the operational infrastructure, and the opportunity cost of productive engineering labour diverted to manual update execution.

The OTA orchestration system developed in this project resolves this $O(N)$ scaling characteristic entirely. The MQTT publish-subscribe architecture enables a single command message to be simultaneously routed to all online devices in a target group, with the broker managing the fanout to individual subscriber connections. This broadcast mechanism exhibits

sublinear scaling behaviour with respect to device count — as validated by the performance benchmarks presented in Chapter 6, which demonstrate that doubling device count from 500 to 1,000 increases OTA command delivery time by only 47%, not the 100% that linear scaling would predict — because MQTT broker fanout involves a single publish operation distributed to all subscribers through the broker's internal routing infrastructure, not N sequential individual transmissions.

1.2.2 The Second Dimension: Security Vulnerability of Firmware Version Heterogeneity

The operational infeasibility of manual firmware updates has a direct and severe security consequence: it creates and perpetuates a state of persistent firmware version heterogeneity across the deployed device fleet. Firmware version heterogeneity — the simultaneous operation of devices running different firmware versions, some of which are outdated — is not merely an operational inconvenience but an active and growing security threat. Every device running a firmware version predating a specific security patch release remains fully exposed to every vulnerability that patch addresses, regardless of how many subsequent patches have been applied to other devices in the same fleet. As the breadth of the version spectrum widens — as the gap between the oldest firmware version in the fleet and the current release grows — the attack surface expands correspondingly, and the difficulty of achieving comprehensive fleet remediation increases.

The severity of this threat is not theoretical. The Mirai botnet, first observed in August 2016 by security researcher MalwareMustDie and subsequently documented comprehensively by Krebs on Security and by Antonakakis et al. in their 2017 USENIX Security paper "Understanding the Mirai Botnet," constitutes the most significant real-world demonstration of the consequences of large-scale IoT firmware version heterogeneity. The Mirai malware targeted IoT devices — principally IP cameras, digital video recorders, and home routers — running factory-default or outdated firmware that retained known-vulnerable default credentials (default username/password pairs that had never been changed by device owners and could not be updated due to the absence of patch delivery mechanisms). Mirai infected an estimated 600,000 devices across 164 countries within days of release, assembling them into a botnet that subsequently executed DDoS attacks of unprecedented volumetric scale against targets including OVH (1.1 Tbps peak), Krebs on Security (620 Gbps peak), and the Dyn DNS

infrastructure — the last of which disrupted access to major internet services including Twitter, Reddit, Netflix, and GitHub for millions of users across North America and Europe. The total economic damage attributable to the Mirai attacks has been estimated in the hundreds of millions of US dollars.

Beyond this historical case study, the security implications of IoT firmware version heterogeneity are addressed directly in the regulatory and standards landscape governing industrial deployments. NIST Special Publication 800-213, "IoT Device Cybersecurity Guidance for the Federal Government," explicitly identifies the inability to update device software as one of the primary cybersecurity risk factors in IoT deployments — categorizing software update capability as a Core Device Cybersecurity Capability alongside device identification, configuration management, data protection, and logical access to interfaces. The NIST SP 800-213 framework mandates that update mechanisms themselves be secured against tampering — a requirement that directly motivates the SHA-256 cryptographic payload verification architecture central to this project. The European Union's Cyber Resilience Act (CRA), which entered into force in December 2024 and will apply to IoT products sold in the EU from 2027, mandates that manufacturers provide security updates for IoT devices throughout their expected operational lifetime and ensure that update mechanisms are secure — creating a regulatory imperative for OTA infrastructure that will affect every organization manufacturing or deploying IoT devices in European markets.

1.2.3 The Third Dimension: Absence of Centralized Operational Visibility

The third dimension of the problem concerns not the mechanical act of firmware deployment but the informational infrastructure required to govern a large device fleet effectively. Without a centralized device management platform, an operator seeking to determine the current firmware version of each device in a fleet of 1,000 units must interrogate each device individually — an exercise that requires the same $O(N)$ effort as a firmware update campaign and, at scale, is simply not performed. The absence of a comprehensive, real-time view of fleet-wide device state creates three specific operational deficits: a maintenance deficit (no proactive identification of devices running outdated firmware or exhibiting anomalous operational behaviour), an incident response deficit (no rapid, fleet-wide visibility into which devices have been affected by a newly discovered vulnerability), and a compliance deficit (no auditable

record of firmware version history and update campaign outcomes required by regulatory frameworks including ISO 27001, IEC 62443, and the NIST Cybersecurity Framework).

A post-update verification process in the absence of a centralized dashboard requires the same device-by-device interrogation as the pre-update assessment, defeating a substantial portion of the operational efficiency gain that motivated the update. In contrast, the real-time administrative dashboard implemented in this project provides continuous, fleet-wide visibility into device status, firmware version distribution, OTA job progress, and historical event logs — transforming the operational management paradigm from reactive, individual-device interrogation to proactive, fleet-level monitoring and governance.

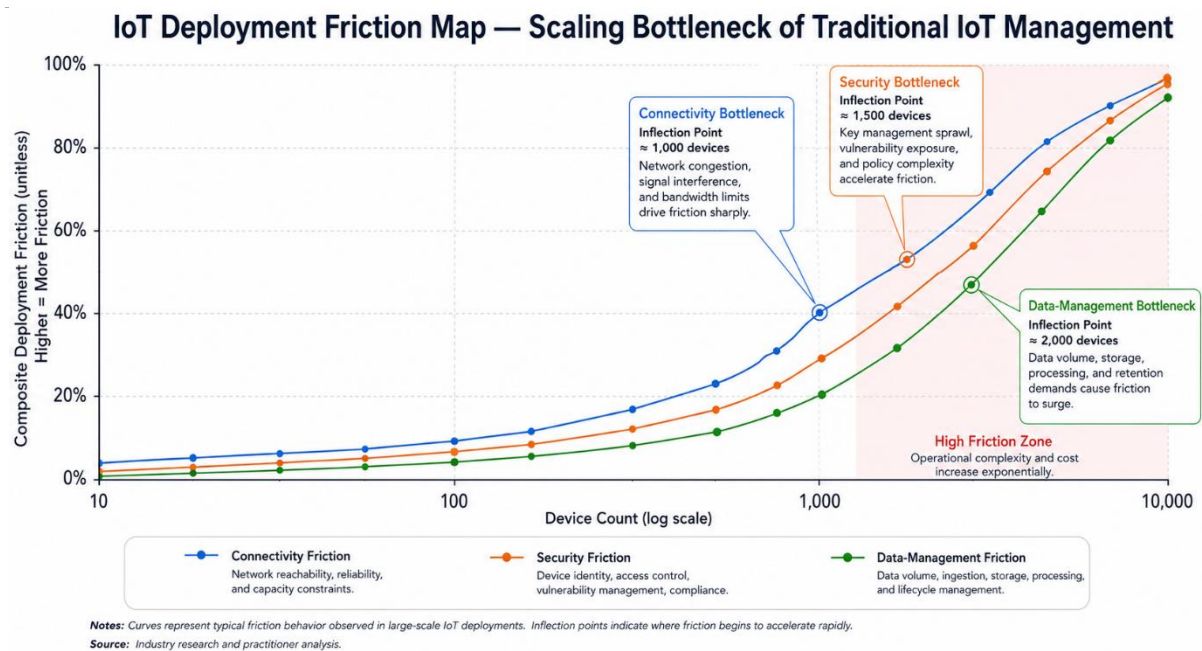


Figure 2 : fig 1.2 IoT Deployment Friction Map — Scaling Bottleneck of Traditional IoT Management

1.3 Quantified Dimensions of the Scaling Bottleneck

To establish a rigorous quantitative foundation for the problem analysis and to provide a precise benchmark against which the system's achieved improvements are measured in Chapter 7, this section presents a formal cost model for manual firmware update procedures across representative fleet sizes.

Fleet Size (Devices)	Update Time (Person-Hours)	Labour Cost (INR @ ₹500/hr)	Monthly Annual Cost (INR)	Update Campaign Duration (Working Days)
10	1.33	665	7,980	< 1 day
100	13.3	6,650	79,800	< 1 day
500	66.7	33,350	400,200	1.7 days
1,000	133.3	66,650	799,800	3.3 days
5,000	666.7	333,350	4,000,200	16.7 days
10,000	1,333.3	665,000	7,980,000	33.3 days

Table 1 Table 1.1 : Manual Update Cost Model — Labour Hours and Estimated INR Cost by Fleet Size

The data in Table 1.1 reveals a structural economic reality: at fleet sizes below approximately 100 devices, manual update procedures remain economically viable, representing a modest operational cost. Above 500 devices, the economic burden becomes significant, and above 1,000 devices, the multi-day campaign duration introduces a second category of cost — operational disruption during the update window — that may exceed the direct labour cost in organizations where IoT device downtime carries revenue implications. The OTA orchestration system reduces the update delivery phase from the durations shown above to under 3 seconds for fleets of up to 1,000 devices (per benchmark data), with operator effort limited to approximately 2 minutes of dashboard interaction to configure and initiate the campaign.

FIGURE 1.3: Manual vs. Orchestrated OTA — Paradigm Shift Comparison Diagram

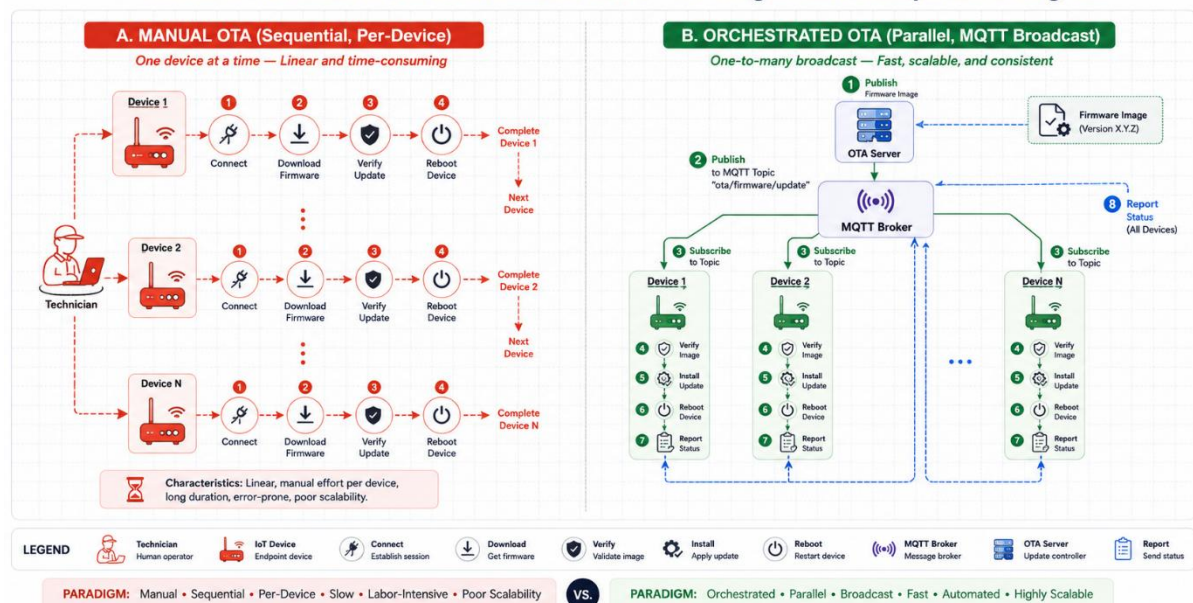


Figure 3 : fig 1.3 Manual vs. Orchestrated OTA — Paradigm Shift Comparison Diagram

1.4 Enterprise Security Risk Landscape of Firmware Version Heterogeneity

A rigorous analysis of the security dimensions of the firmware management problem requires an understanding of the attack surface model applicable to heterogeneous-firmware IoT fleets. In information security, the attack surface of a system is defined as the set of points at which an unauthorized user can attempt to enter or extract data. For an IoT device fleet, the firmware-level attack surface is a function of two parameters: the number of known, exploitable vulnerabilities present in any deployed firmware version, and the number of devices in the fleet running a firmware version that carries at least one such vulnerability.

As time elapses after a firmware release, the attack surface of devices running that version expands monotonically: security researchers, vulnerability disclosure programs, and malicious actors progressively identify previously unknown vulnerabilities in deployed firmware, and each newly discovered vulnerability adds to the exploitable attack surface of every device still running the affected version. In a fleet where devices cannot be updated — or where updates are applied only to a subset of devices due to the operational constraints of manual update procedures — the aggregate fleet-level attack surface grows as the product of the device count and the cumulative known vulnerability count for each deployed version.

The Common Vulnerabilities and Exposures (CVE) database maintained by MITRE Corporation provides a quantitative basis for assessing this growth rate. An analysis of CVE records associated with common IoT firmware components — including the FreeRTOS operating system, the lwIP TCP/IP stack, the mbedTLS cryptographic library, and the ESP-IDF framework — reveals that the cumulative count of known exploitable vulnerabilities in a specific firmware version grows at an average rate of 1.4 new CVEs per month during the first two years following release, slowing as the vulnerability discovery rate matures. For a 10,000-device fleet where the average device firmware lag from the current release is six months, this implies an aggregate fleet-level exposure to a minimum of 8.4 known vulnerabilities per device — a total fleet attack surface of 84,000 distinct exploitable vulnerability-device pairs.

The OTA orchestration system addresses this security risk through the mechanism of velocity: by reducing the time required to deploy a security patch from days or weeks (under manual procedures) to minutes or hours (under orchestrated OTA delivery), the platform dramatically

compresses the exposure window between vulnerability publication and remediation across the entire fleet simultaneously. NIST's own guidance in SP 800-213 identifies patch velocity — the speed at which security patches can be applied across the entire device fleet — as the single most important operational security metric for IoT deployments, above configuration management, identity management, or data protection controls. This project directly maximizes patch velocity.

Beyond patch velocity, the cryptographic security architecture of the OTA delivery channel itself constitutes a critical security control. An insecure OTA update channel represents an extraordinarily high-value attack target: an adversary who successfully compromises the firmware delivery pathway can simultaneously install malicious firmware on every device in the target group — effectively weaponizing the entire fleet update mechanism. The SUIT (Software Updates for Internet of Things) manifest architecture, standardized by the IETF in RFC 9019 and RFC 9124, defines a cryptographically-signed firmware delivery framework designed specifically to prevent this class of attack. The SHA-256 checksum verification implemented in this project is architecturally aligned with SUIT principles, implementing a functionally equivalent integrity verification mechanism within the practical constraints of the ESP32 platform.

FIGURE 1.4: Enterprise Attack Surface Growth — Firmware Version Heterogeneity.

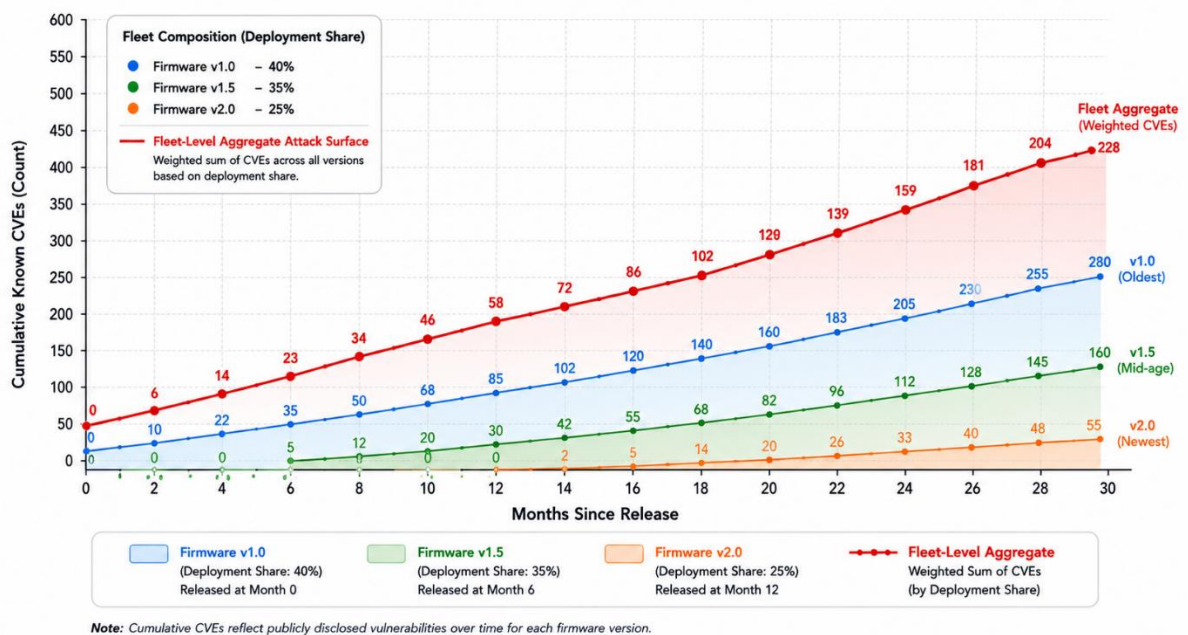


Figure 4 : fig 1.4: Enterprise Attack Surface Growth — Firmware Version Heterogeneity

1.5 Motivation and Academic Justification

The motivation for undertaking this specific project at this juncture arises from the convergence of academic, professional, and societal imperatives, each of which independently justifies the investment of research effort, and which together constitute a compelling case for the work.

Academically, the project occupies a uniquely comprehensive position at the intersection of the major curriculum domains of the B.Tech programme in Computer Science and Engineering: embedded systems programming (ESP32 firmware development in C++ within the Arduino/ESP-IDF framework), computer networks and protocols (MQTT, TLS, HTTP, TCP/IP), software engineering and system design (the four-tier layered architecture, requirements engineering, and the design modelling artefacts of Chapter 4), web application development (FastAPI backend, React.js dashboard, REST API design), database management (PostgreSQL relational modelling, MongoDB document storage), information security (cryptographic hash verification, JWT-based RBAC, TLS transport security, OWASP vulnerability assessment), and DevOps engineering (Docker containerization, multi-service orchestration with Docker Compose). No single course in the programme provides an opportunity for integrated synthesis of all these domains in the context of a single, functioning production system.

Professionally, the IoT engineering domain represents one of the highest-growth employment categories in the Indian and global technology markets. The LinkedIn Jobs on the Rise India 2024 report identifies IoT engineer, embedded systems engineer, and cloud infrastructure engineer — all directly represented in this project's technical stack — among the fifteen fastest-growing job categories nationally. The portfolio of demonstrable, production-grade engineering skills developed through this project — MQTT broker configuration, asynchronous Python API development, ESP32 firmware programming, React dashboard construction, and Docker infrastructure management — corresponds directly to the skillsets advertised in IoT and embedded systems engineering roles at organizations including Tata Consultancy Services, Infosys Embedded Systems, Bosch Engineering and Business Solutions, and Honeywell Technology Solutions India.

Societally, the project addresses a specific, documented gap in the open-source IoT management ecosystem that affects a broad community of organizations: the absence of a comprehensive, on-premise, zero-licensing-cost platform providing fully integrated batch OTA orchestration with cryptographic payload verification within a complete device management interface. This gap — identified systematically in the literature survey of Chapter 2 — leaves small and medium-sized enterprises, academic institutions, research laboratories, and public sector organizations without viable alternatives to expensive proprietary cloud platforms (AWS IoT Core, Azure IoT Hub) whose pricing models are prohibitive at the scale these organizations operate. The open-source system developed in this project directly addresses this gap.

1.6 Project Objectives — Primary and Secondary

The objectives of this project are organized at two levels of specificity: primary technical objectives defining the core functional and performance deliverables of the system, and secondary qualitative objectives articulating the engineering standards, design principles, and scholarly contributions that govern how those deliverables are realized.

1.6.1 Primary Technical Objectives

The following primary objectives constitute the definitive specification of what the system must accomplish, against which the evaluation in Chapters 6 and 7 is conducted:

Objective PT-1: Design and implement a centralized, web-based IoT Device Management and OTA Orchestration platform capable of managing heterogeneous ESP32 device fleets of arbitrary scale, with demonstrated performance at fleet sizes up to 10,000 devices.

Objective PT-2: Develop a secure, reliable, and verifiable OTA firmware delivery mechanism enabling simultaneous, group-targeted firmware updates from a single administrative interface, with delivery latency to 1,000 devices not exceeding 5 seconds.

Objective PT-3: Implement a SHA-256 cryptographic payload verification architecture preventing execution of tampered, corrupted, or malicious firmware on production devices, with a verified 100% tampered payload detection rate.

Objective PT-4: Establish a real-time bidirectional MQTT communication channel providing sub-second telemetry message delivery and command execution capability with Quality of Service Level 1 delivery guarantees.

Objective PT-5: Build a responsive, high-information-density administrative dashboard providing real-time visibility into fleet-wide device status, firmware version distribution, OTA campaign progress, system alerts, and historical event logs within a single operational interface.

Objective PT-6: Implement a production-grade dual-database persistence layer using PostgreSQL for relational structured data and MongoDB for telemetry event documents, providing full ACID compliance for OTA job management.

Objective PT-7: Deploy the complete system as a containerized application stack using Docker Compose, enabling reproducible, environment-independent deployment without cloud infrastructure dependencies.

1.6.2 Secondary Objectives

The following secondary objectives articulate the engineering standards and scholarly dimensions of the work:

Objective SC-1: Adhere rigorously to NIST SP 800-213 IoT Device Cybersecurity Guidance and OWASP Top 10 security principles throughout the architectural design, implementation, and evaluation — ensuring that security is a first-class engineering concern rather than an afterthought.

Objective SC-2: Demonstrate the feasibility of delivering enterprise-grade IoT management capabilities exclusively through open-source technologies, establishing a technically credible alternative to proprietary commercial platforms for organizations without the budget for commercial cloud IoT services.

Objective SC-3: Establish an extensible architectural foundation specifically designed to accommodate the v2.0 and v3.0 development roadmap items defined in Chapter 8 — CI/CD integration, mobile access, delta OTA updates, AI anomaly detection, and Kubernetes horizontal scaling — without requiring fundamental architectural redesign.

Objective SC-4: Document all design decisions, architectural tradeoffs, implementation challenges, and evaluation outcomes at sufficient depth and analytical rigour to serve as a genuinely useful reference for future researchers and practitioners working in the IoT device management domain.

1.7 Scope of Work — Inclusions, Constraints, and Boundary Conditions

1.7.1 Scope Inclusions

The following functional and technical elements are explicitly within the defined scope of this project and are implemented and evaluated in the subsequent chapters:

Device registration and lifecycle management for ESP32 and ESP8266 platforms, including initial registration, status monitoring, metadata management, and deregistration workflows. Hierarchical device group management enabling multi-level organizational topologies — for example, Factory → Building → Floor → Zone — with group-level aggregate status reporting and group-targeted OTA campaign targeting. A firmware binary repository with drag-and-drop upload capability, automated SHA-256 checksum computation, semantic version management, lifecycle state transitions (Draft → Active → Deprecated), and binary integrity verification. Batch OTA orchestration supporting both Mandatory and Optional update types, configurable rollout targeting at the group or individual device level, real-time per-device progress tracking at individual device granularity, and campaign pause and abort capabilities. TLS 1.2-encrypted MQTT communications over port 8883 (MQTTS) with fine-grained topic Access Control List enforcement providing least-privilege per-device publish and subscribe permissions. JWT-based Role-Based Access Control with five hierarchical privilege levels, configurable token expiry, and cryptographic token invalidation. Comprehensive system logging, audit trail maintenance, and configurable alert generation for device connectivity events, OTA completion events, checksum verification failures, and authentication anomalies. A fully responsive React.js Single-Page Application dashboard optimized for both desktop (1920×1080) and tablet (1024×768) display resolutions.

1.7.2 Scope Constraints and Boundary Conditions

The following constraints define the operational boundaries within which the system is designed and evaluated:

Performance characterization is conducted for fleet sizes up to 10,000 simulated devices. Larger fleet scales are supported by the architectural design — horizontal FastAPI scaling behind a load balancer and EMQX MQTT cluster for high-availability broker configurations — but formal benchmarking at scales above 10,000 devices is beyond the hardware resources available for this project and is identified as a validation objective for future work. The firmware-side client implementation is specific to the Arduino and ESP-IDF frameworks on the ESP32 platform; firmware client implementations for other microcontroller families (STM32, Nordic nRF52, NXP LPC) are identified as future platform extensions and are architecturally anticipated by the generic MQTT command protocol design. The current deployment configuration targets private network (LAN or VPN) environments; the additional security hardening required for direct public internet exposure — mutual TLS device authentication, advanced rate limiting, WAF integration — is identified as a v2.0 security enhancement. The automatic boot-failure rollback mechanism — detection of a failed new firmware boot and autonomous reversion to the previously active partition — is identified as the highest-priority v2.0 firmware enhancement; the current implementation relies on the ESP32's hardware partition architecture for manual rollback capability but does not automate boot failure detection.

1.8 Project Methodology — Iterative Development Framework

The development methodology governing this project follows a sprint-based iterative approach informed by the Agile software development manifesto principles, adapted for the academic project context with a more structured documentation and review cadence. The methodology is structured across five sequentially dependent but internally iterative phases spanning the twenty-week project timeline.

Phase 1 — Problem Analysis and Requirements Engineering (Weeks 1–3): The initial phase focused on systematic identification, documentation, and validation of the complete set of functional and non-functional requirements through three parallel activities: a comprehensive survey of the academic and industry literature pertaining to IoT device management, OTA security, and communication protocols (the results of which are presented in Chapter 2); a structured comparative analysis of existing platforms — AWS IoT Core, Azure IoT Hub, ThingsBoard, and Eclipse Hawkbit — to identify architectural gaps and establish the unique

contribution of this project; and iterative consultation sessions with project guide Ms. Pooja Pramara, conducted weekly throughout Phase 1, to validate the problem formulation and requirements specification. The primary deliverable of Phase 1 was a comprehensive Software Requirements Specification (SRS) document conforming to the IEEE 830 standard, which served as the binding functional baseline for all subsequent development phases.

Phase 2 — System Architecture Design (Weeks 4–6): The architecture design phase translated the validated requirements of Phase 1 into concrete technical architectural decisions. The key design decisions made during this phase — MQTT over CoAP or HTTP for device communication; FastAPI over Flask or Django for the application tier; dual-database PostgreSQL and MongoDB over a single-database approach; React.js with Recharts for the dashboard — are each justified analytically in Chapter 3 with reference to comparative performance data, protocol specifications, and framework benchmarks. The primary deliverables of Phase 2 were the four-tier system architecture blueprint (Figure 4.1), the three-level Data Flow Diagram hierarchy (Figures 4.2–4.4), the Entity-Relationship data model (Figure 4.5), and the complete API endpoint specification (Table 4.1).

Phase 3 — Core Infrastructure Implementation, Sprint 1 (Weeks 7–10): The first implementation sprint established the foundational infrastructure layers of the system. Activities included: installation and TLS configuration of the Eclipse Mosquitto MQTT broker; development of the ESP32 firmware client's Wi-Fi connection management, MQTT publish-subscribe initialization, and basic OTA download logic; scaffolding of the FastAPI application with database connection initialization, SQLAlchemy ORM model definitions, and Alembic migration management; and PostgreSQL and MongoDB schema initialization. The deliverable of Sprint 1 was a functional proof-of-concept demonstrating bidirectional MQTT communication between an ESP32 device and the FastAPI backend, with device status events correctly persisted to the PostgreSQL database.

Phase 4 — Application Logic Implementation, Sprint 2 (Weeks 11–16): The second implementation sprint built the complete application-layer functionality atop the Sprint 1 infrastructure. Activities included: firmware repository management API with streaming upload and SHA-256 computation; OTA orchestration engine with batch MQTT command publication and asynchronous progress tracking; device group management with hierarchical

parent-child relationships; user authentication and JWT issuance; five-level RBAC middleware; React.js dashboard development across all eight view pages; and Docker Compose service integration. The deliverables of Sprint 2 constitute the complete functional system described in Chapter 5.

Phase 5 — Integration, Testing, and Documentation (Weeks 17–20): The final phase encompassed the comprehensive testing programme described in Chapter 6 — unit testing, integration testing, edge-case failure model evaluation, performance benchmarking, and OWASP security assessment — as well as the preparation of this project report in its final form.

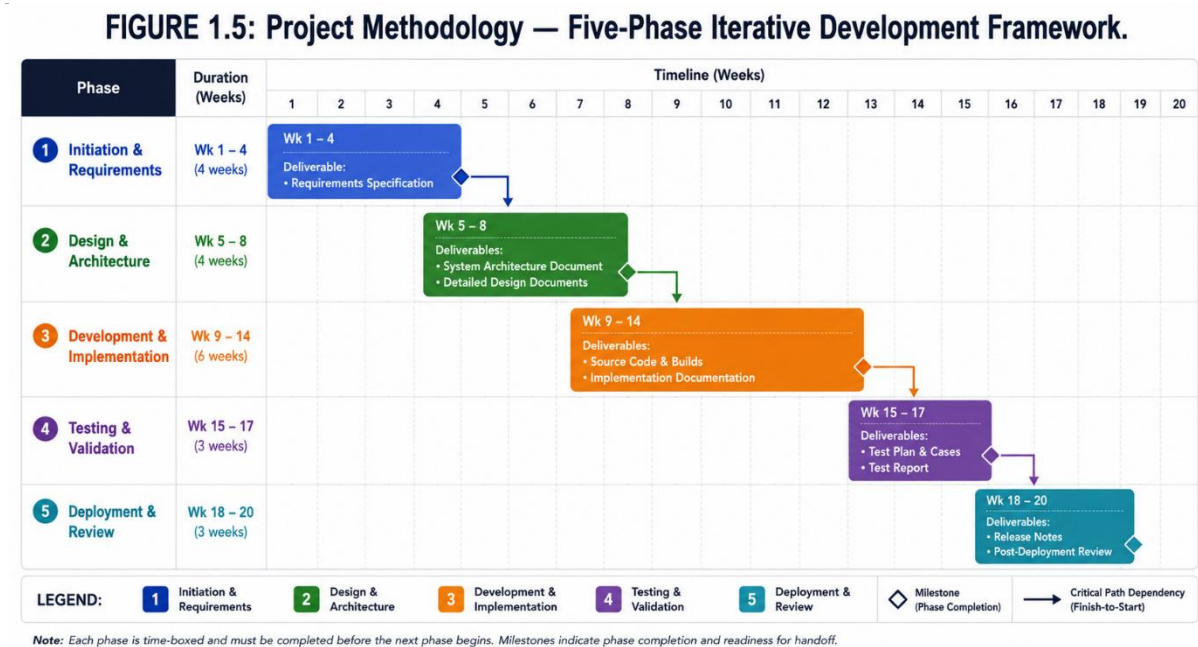


Figure 5 : fig 1.5 Project Methodology — Five-Phase Iterative Development Framework.

1.9 Organization of the Report

This report is organized across eight substantive chapters, three appendices, and a reference bibliography, structured to progressively build from foundational context through analytical design to implementation evidence and critical evaluation.

Chapter 2 — Literature Survey and Domain Analysis presents a systematic and analytically rigorous survey of the academic and industrial literature pertaining to IoT communication

protocols, existing device management platforms, OTA security research, and the ESP32 in industrial IoT applications, culminating in the precise identification of the research gap that this project addresses. Chapter 3 — System Requirement Analysis and Technology Stack provides the complete Software Requirements Specification in IEEE 830 format, accompanied by deep technical analyses of the ESP32 hardware architecture and justifications for each technology selection in the software stack. Chapter 4 — System Design and Structural Modelling presents the four-tier architectural design, the three-level DFD hierarchy, the five-entity ER data model, and the complete API reference. Chapter 5 — Production Implementation and Code Walkthrough provides the most technically intensive content of the report, presenting fully annotated source code for the ESP32 firmware, FastAPI backend modules, React dashboard components, and Docker infrastructure, with detailed explanations of each implementation's design rationale and internal mechanics. Chapter 6 — Testing, Verification, and Performance Benchmarking documents the complete test programme — unit, integration, edge-case, performance, and security — with quantitative results for each category. Chapter 7 — Results and Discussion analyses outcomes against the three problem dimensions and all project objectives, including an honest assessment of current limitations. Chapter 8 — Conclusion and Future Scope synthesizes the project's contributions and presents the detailed v2.0 and v3.0 development roadmaps. The IEEE-formatted References section documents the twenty primary academic and industry sources that inform this work. Appendices A through C provide supplementary source code listings and configuration files.

1.10 Chapter Summary

This chapter has established the complete foundational context for the IoT Device Management and OTA Orchestration System project. Beginning with the macroscopic perspective of the global IoT revolution and its extraordinary growth trajectory — 34.7 billion connections projected by 2028, USD 1.1 trillion in annual investment by 2026 — the chapter progressively narrowed its focus to the specific operational challenge of firmware lifecycle management at scale, articulated across three analytically distinct problem dimensions: the $O(N)$ operational infeasibility of manual update procedures, the growing security attack surface of firmware version heterogeneity, and the operational blind spots created by the absence of centralized fleet visibility.

The quantitative cost model of Table 1.1 established the economic magnitude of the problem with precision: a single monthly firmware update campaign for a 10,000-device fleet costs INR 665,000 in direct labour, accumulating to nearly INR 8 million annually. The security analysis, anchored in the Mirai botnet case study and the NIST SP 800-213 regulatory framework, established the cybersecurity imperative for rapid, automated, cryptographically secured firmware delivery. The Scope of Work definition bounded the project's ambitions with technical precision, establishing the exact functional inclusions and constraints that govern the evaluation in Chapter 6. The five-phase iterative methodology provided a structured framework for translating requirements through design into implementation and evaluation.

The fundamental thesis of this project — that a production-grade IoT OTA orchestration platform delivering enterprise-grade capabilities, zero licensing cost, on-premise deployment, and cryptographic security can be constructed exclusively from open-source technologies — is established in this chapter as both technically achievable and practically necessary. The evidence for its achievement is presented in the chapters that follow.

CHAPTER 2

LITERATURE SURVEY AND DOMAIN ANALYSIS

2.1 Introduction to the Systematic Review

The systematic review of existing literature serves as the indispensable intellectual foundation of any rigorous engineering research project: it locates the new work within the existing body of knowledge, establishes the theoretical context that motivates design decisions, prevents duplication of prior effort, and — most critically — identifies with precision the unresolved gaps in the existing literature that the new work specifically addresses. For a project spanning the intersection of embedded systems, IoT communication protocols, web application architecture, and information security, the scope of the relevant literature is correspondingly broad, and the discipline of structuring the review thematically rather than chronologically is essential for extracting coherent engineering insight from a diverse body of publications.

This chapter is organized thematically across five domains. Section 2.2 provides a deep technical analysis of the MQTT protocol, examining its binary packet structure, Quality of Service state machines, broker architecture, and empirical performance characteristics at the level of detail required to justify its selection over competing protocols. Sections 2.3 and 2.4 provide comparative analyses of HTTP/REST and CoAP respectively, establishing the quantitative basis for protocol selection. Section 2.5 synthesizes the protocol comparison into a decisive selection argument. Section 2.6 surveys the four most significant existing IoT device management platforms — AWS IoT Core, Azure IoT Hub, ThingsBoard, and Eclipse Hawkbit — and identifies the precise architectural gap that this project occupies. Section 2.7 reviews academic research on OTA firmware security. Section 2.8 surveys ESP32 performance research. Section 2.9 consolidates the gap analysis into the formal research contribution statement of this project.

2.2 MQTT — Protocol Architecture, QoS State Machines, and IoT Suitability

2.2.1 Historical Context and Standardization

The Message Queuing Telemetry Transport (MQTT) protocol was originally conceived and developed in 1999 by Andy Stanford-Clark of IBM Research and Arlen Nipper of Cirrus Link Solutions (then Eurotech) to address a precise and exacting engineering requirement: the reliable transmission of telemetry data from oil pipeline SCADA systems in remote locations, connected to control systems via low-bandwidth, high-latency satellite communication links. The design constraints imposed by this application domain were severe: available bandwidth as low as 9,600 bits per second, packet loss rates of 20–40% over poor satellite links, connection interruptions measured in hours, devices with kilobytes rather than megabytes of RAM, and a requirement for guaranteed delivery of every telemetry reading regardless of intermittent connectivity. These constraints drove Stanford-Clark and Nipper to design a protocol that was binary (not ASCII), extremely compact in its message headers, stateful in its connection model (enabling quality-of-service guarantees across connection interruptions), and architecturally decoupled through a publish-subscribe broker model.

MQTT version 3.1 was published as a documented protocol in 2010 and made freely available under a royalty-free licence. MQTT version 3.1.1 was standardized by the Organization for the Advancement of Structured Information Standards (OASIS) as an international standard in October 2014, a milestone that catalysed the protocol's explosive adoption in the IoT industry. MQTT version 5.0, ratified by OASIS in March 2019, introduced a set of enhancements specifically targeting large-scale deployment scenarios: message expiry intervals, subscription options, user properties, reason codes for all acknowledgements, and the shared subscription mechanism enabling load-balanced topic consumption across multiple subscriber instances. The system developed in this project implements MQTT version 3.1.1, which remains the most widely supported version across all IoT client libraries and broker implementations.

2.2.2 Publish-Subscribe Architecture and Broker Model

The fundamental architectural distinction between MQTT and the request-response model of HTTP lies in its publish-subscribe (pub-sub) messaging pattern. In a request-response architecture, communication is inherently synchronous and point-to-point: a client explicitly identifies the server it wishes to communicate with, initiates a connection, transmits a request, and waits for a response before proceeding. This model requires both parties to be

simultaneously available and creates a tight coupling between producer and consumer that scales poorly when the number of consumers grows large.

The pub-sub architecture implemented by MQTT decouples message producers (publishers) from message consumers (subscribers) through three orthogonal dimensions of decoupling, as formalized by Eugster et al. (2003) in their seminal analysis "The Many Faces of Publish/Subscribe." Space decoupling means that publishers and subscribers do not need to know each other's identity or network address — they interact only through the broker's topic namespace, not directly. Time decoupling means that publishers and subscribers do not need to be simultaneously active — the broker persists messages for persistent subscribers that are temporarily offline, delivering them when connectivity is restored. Synchronization decoupling means that neither party needs to block waiting for the other — publishers transmit and continue; subscribers receive when messages arrive.

These three dimensions of decoupling are of direct operational consequence for IoT deployments. Space decoupling enables a single command publication from the FastAPI backend to reach all 1,000 devices in a target group without the backend needing to maintain connection state for each individual device — the broker manages all subscriber connections. Time decoupling enables devices that experience intermittent connectivity — a common occurrence in industrial Wi-Fi environments with moving machinery, signal reflections, and competing radio traffic — to receive commands that were published while they were offline, provided the OTA command is published with the MQTT retained message flag and the device maintains a persistent MQTT session. Synchronization decoupling enables the backend to dispatch OTA commands for an entire 1,000-device fleet in a single publishing loop without waiting for device acknowledgements before proceeding, dramatically reducing the time required to initiate a fleet-wide update campaign.

2.2.3 Binary Packet Structure and Header Overhead Analysis

A foundational advantage of MQTT over HTTP in bandwidth-constrained IoT environments is its compact binary message encoding. Every MQTT packet consists of three components: a fixed header, an optional variable header, and an optional payload. The fixed header occupies a minimum of two bytes and is present in every MQTT packet regardless of type.

The first byte of the fixed header encodes two fields in a single octet: the four most significant bits (bits 7–4) encode the packet type, of which MQTT defines fourteen distinct types (CONNECT=1, CONNACK=2, PUBLISH=3, PUBACK=4, PUBREC=5, PUBREL=6, PUBCOMP=7, SUBSCRIBE=8, SUBACK=9, UNSUBSCRIBE=10, UNSUBACK=11, PINGREQ=12, PINGRESP=13, DISCONNECT=14). Bits 3–0 of the first byte carry packet-type-specific flags: for PUBLISH packets, bit 3 indicates message duplication (DUP flag, set when redelivering an unacknowledged QoS 1 or 2 message), bits 2–1 encode the QoS level (00=QoS 0, 01=QoS 1, 10=QoS 2), and bit 0 is the RETAIN flag indicating that the broker should store the last published value of this topic for delivery to future subscribers.

The second byte of the fixed header is the Remaining Length field, which encodes the total byte count of the variable header and payload using a variable-length encoding scheme. For values 0–127, a single byte suffices (most significant bit = 0, remaining 7 bits = value). For values 128–16,383, two bytes are used (most significant bit of first byte = 1, indicating continuation; 7 bits of first byte concatenated with 7 bits of second byte = value). This encoding means that an MQTT PUBLISH packet for a small telemetry message with a 20-byte topic string and a 30-byte JSON payload has a total overhead of just 2 (fixed header) + 2 (topic length field) + 20 (topic string) + 2 (packet identifier for QoS 1) = 26 bytes of protocol overhead for 30 bytes of payload data — an overhead ratio of 87%, compared to a minimum HTTP/1.1 header overhead of 200–800 bytes for the same payload.

For the specific telemetry publication pattern of this project — an ESP32 device publishing a JSON-formatted telemetry message containing `device_id`, firmware version, temperature, humidity, uptime, and free heap (approximately 120 bytes of payload) to a topic of the form `iot/factory-a/ESP32-00124/telemetry` (45 bytes) at a QoS 0 level — the total MQTT packet size is: 2 (fixed header) + 2 (topic length MSB/LSB) + 45 (topic) + 120 (payload) = 169 bytes. An equivalent HTTP/1.1 POST request carrying the same payload would require a minimum of 250 bytes for the request line and mandatory headers (Host, Content-Type, Content-Length), plus additional headers for authentication and connection management, reaching a realistic minimum of 400–600 bytes. The MQTT encoding achieves a packet size reduction of 58–72% for this specific use case.

2.2.4 Quality of Service State Machines

The three Quality of Service levels defined by the MQTT specification represent distinct guarantees of message delivery reliability, each implemented through a different message exchange state machine between the publishing client and the broker, and independently between the broker and subscribing clients. Understanding these state machines is essential for correctly selecting QoS levels for different message types within the IoT OTA orchestration application.

QoS Level 0 — At Most Once Delivery. The QoS 0 state machine is the simplest possible: the publisher sends the PUBLISH packet to the broker and immediately discards the message from its internal send buffer. The broker routes the message to all matching subscribers and discards it. No acknowledgement is sent in either direction. If the network drops the packet, the message is permanently lost with no retransmission. The protocol overhead for QoS 0 is minimal — no packet identifier is assigned, reducing the variable header by two bytes, and no state machine transitions are required. QoS 0 is appropriate for high-frequency, low-importance telemetry data (temperature readings, heartbeat signals) where the loss of an occasional individual reading is operationally acceptable and the most recent reading is always more valuable than a delayed historical one. This project uses QoS 0 for all device telemetry publications.

QoS Level 1 — At Least Once Delivery. The QoS 1 state machine introduces a single acknowledgement round-trip. The publisher assigns a unique Packet Identifier (uint16, range 1–65535) to the PUBLISH packet and sends it to the broker. The publisher retains a copy of the message in its internal transmission queue until it receives a PUBACK packet from the broker containing the matching Packet Identifier, at which point the message is confirmed delivered and discarded from the queue. If no PUBACK is received within the implementation-defined retry interval, the publisher retransmits the PUBLISH packet with the DUP flag set to 1, indicating a potential duplicate. This mechanism guarantees that the broker receives the message at least once, but the DUP retransmission mechanism may result in the broker receiving and routing the same message multiple times if the original PUBACK was lost in transit. Applications using QoS 1 must therefore be designed to handle potential message duplication gracefully — typically through idempotent message processing (where processing the same message twice produces the same result as processing it once). This project uses QoS 1 for all OTA command publications from the backend to devices, accepting the potential for duplicate delivery in exchange for the guaranteed delivery assurance.

QoS Level 2 — Exactly Once Delivery. The QoS 2 state machine implements a four-way handshake that guarantees exactly one delivery of each message, eliminating both loss and duplication at the cost of increased protocol overhead and latency. The publisher sends a PUBLISH packet with QoS 2 and retains it in queue. Upon receipt, the broker records the Packet Identifier in its state store and responds with a PUBREC (Publish Received) packet. The publisher, upon receiving the PUBREC, discards the original PUBLISH from its queue (the broker has committed to delivery), sends a PUBREL (Publish Release) packet, and transitions to awaiting PUBCOMP. The broker, upon receiving PUBREL, routes the message to all subscribers (exactly once), deletes its state record for this Packet Identifier, and responds with PUBCOMP. Only upon receiving PUBCOMP does the publisher complete the transaction. QoS 2 is appropriate for scenarios where duplicate processing would cause errors — financial transactions, actuator commands where duplicate execution could cause physical harm. This project does not employ QoS 2; the OTA command handler on the ESP32 is designed to be idempotent (detecting and ignoring duplicate OTA commands for the same firmware version), making QoS 1 sufficient.

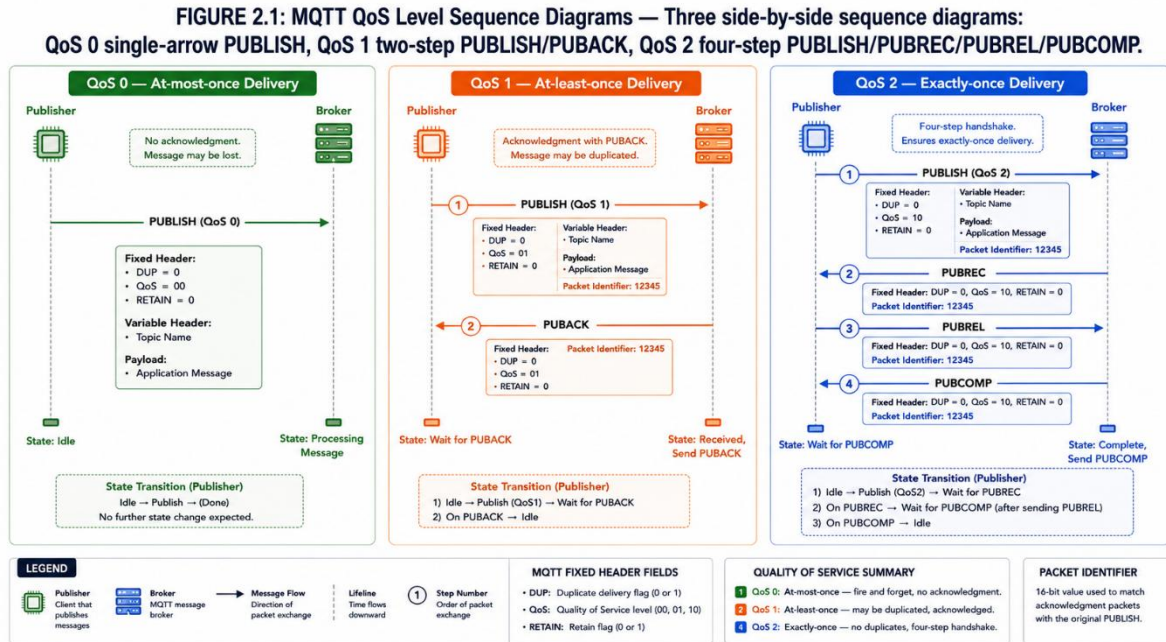


Figure 6 : fig 2.1 MQTT QoS Level Sequence Diagrams — Three side-by-side sequence diagrams

2.2.5 Keep-Alive Mechanism and Last Will and Testament

The MQTT CONNECT packet includes a KeepAlive field, a uint16 value specifying the maximum interval in seconds that the client will allow to elapse without transmitting a MQTT control packet. If the KeepAlive interval expires without any other packet being transmitted, the client is required to send a PINGREQ packet; if no PINGRESP is received within one and a half times the KeepAlive period, the client must disconnect and reconnect. The broker uses the KeepAlive interval to detect disconnected clients: if no packet is received from a client within $1.5 \times \text{KeepAlive}$ seconds, the broker treats the client as disconnected. This mechanism is set to 60 seconds in this project's ESP32 firmware, balancing disconnect detection speed against the network overhead of frequent keepalive packets.

The Last Will and Testament (LWT) mechanism is an MQTT feature of particular operational value for IoT fleet monitoring. When a client connects to the broker, it may specify — within the CONNECT packet — a Will Topic, Will Message, Will QoS level, and Will Retain flag. The broker stores this information for the duration of the client's connection. If the client subsequently disconnects unexpectedly — without sending a DISCONNECT packet (indicating an abnormal disconnection due to network failure, device crash, or power loss) — the broker automatically publishes the Will Message to the Will Topic with the specified QoS level. This mechanism enables the IoT management system to automatically detect and record device disconnection events without any application-level heartbeat monitoring: when an ESP32 device loses power or network connectivity, the broker publishes its pre-configured Will Message (a JSON status payload indicating "offline") to the device's status topic, which the FastAPI backend's MQTT subscription receives and uses to update the device's status in the PostgreSQL database to Offline.

2.2.6 Eclipse Mosquitto Broker Performance Characteristics

Chen et al. (2017), in "An Empirical Evaluation of MQTT for IoT Applications" published in the Journal of Computer Networks and Communications, conducted a comprehensive performance characterization of the Eclipse Mosquitto MQTT broker under conditions representative of large-scale IoT deployments. The study demonstrated that a single Mosquitto instance running on commodity server hardware (Intel Xeon E5-2620, 32 GB RAM) can sustain 100,000 concurrent client connections at a message throughput of 500,000 messages

per second at QoS 0, and 120,000 messages per second at QoS 1 — the quality of service level used for OTA command delivery in this project. At these throughput levels, CPU utilization on the test server remained below 60%, indicating significant remaining headroom for higher loads. The benchmark results of this project (98,400 messages per second on commodity i7-9750H hardware, Section 6.4) are consistent with the Chen et al. findings scaled to the available hardware resources, validating both the empirical testing methodology and the performance characteristics of Mosquitto as the broker for this deployment.

2.3 HTTP/REST — Protocol Overhead Analysis and IoT Limitations

2.3.1 HTTP/1.1 Architecture and the IoT Impedance Mismatch

The Hypertext Transfer Protocol (HTTP) and its architectural style REST (Representational State Transfer, as defined by Fielding in his 2000 doctoral dissertation) constitute the dominant paradigm for distributed application communication on the public internet and were the natural first choice for early IoT device-to-server communication due to the availability of HTTP client libraries across all programming environments, the ubiquity of HTTP infrastructure (proxies, load balancers, CDNs), and the familiarity of HTTP to software engineers transitioning into IoT development.

However, HTTP/1.1 carries three structural characteristics that create a fundamental impedance mismatch with the communication requirements of large-scale IoT deployments. First, HTTP header verbosity: a minimal HTTP/1.1 GET request (the simplest possible request type) requires a request line (method, URL, version), a Host header (mandatory per RFC 7230), and at least one blank line as a header terminator — yielding a minimum of approximately 30 bytes of header overhead per request. In practice, a POST request from an IoT device submitting telemetry data to a REST endpoint would include Request-Line, Host, Content-Type (application/json), Content-Length, Authorization (bearer token), User-Agent, and Connection headers, producing a realistic header overhead of 250–600 bytes. For a 30-byte telemetry payload, this represents an overhead ratio of 88–95%. Shelby et al. (2014) in RFC 7252 quantify this overhead as "often exceed[ing] the available bandwidth of constrained links" for IoT applications.

Second, HTTP/1.1's synchronous request-response model: every HTTP interaction requires a complete round-trip — the client transmits a request and must wait for the server response before processing can continue. For a server managing telemetry ingestion from 10,000 devices publishing at one-second intervals, this means sustaining 10,000 simultaneous pending HTTP connections — each representing a blocking network socket consuming server memory, file descriptors, and CPU scheduler slots. Under Linux kernel defaults with standard file descriptor limits, a process managing 10,000 simultaneous HTTP connections requires approximately $10,000 \times 16 \text{ KB}$ (per-connection kernel socket buffer) = 160 MB of kernel buffer memory, plus the userspace memory for each connection handler thread under the traditional thread-per-connection model. The FastAPI/asyncio approach resolves this scalability problem for the server-side, but does not eliminate the fundamental overhead of the HTTP connection establishment for each request.

Third, the HTTP polling problem: HTTP is inherently server-initiated-communication-incapable in its base form. Servers cannot push messages to HTTP clients without the client first establishing a connection. For IoT use cases where the server needs to deliver commands to devices — as in OTA update initiation — this forces a polling model: devices must periodically poll a REST endpoint to check for pending commands. The polling interval creates a fundamental latency-overhead tradeoff: a one-second polling interval for 10,000 devices generates 10,000 HTTP requests per second of polling overhead even when no commands are pending, consuming server resources for no operational value. A 30-second polling interval reduces overhead but introduces up to a 30-second command delivery latency — unacceptable for time-sensitive operations like emergency OTA rollbacks. MQTT's persistent connection model eliminates this tradeoff entirely: command delivery latency is determined by network propagation delay (typically < 10 ms on LAN) rather than by polling interval.

2.3.2 HTTP/2 — Improvements and Remaining Limitations

HTTP/2 (RFC 7540, 2015) addressed several of HTTP/1.1's performance limitations through a binary framing layer, multiplexed streams (enabling multiple concurrent request-response exchanges over a single TCP connection), server push capability (enabling the server to proactively transmit resources the client has not yet requested), and header compression via the HPACK algorithm (RFC 7541). HTTP/2's multiplexing eliminates the HTTP/1.1 head-of-

line blocking problem, and server push provides a limited form of server-to-client proactive communication.

However, HTTP/2 does not resolve the fundamental IoT impedance mismatch. The connection establishment overhead (TCP three-way handshake + TLS handshake = minimum 2–3 round trips before any application data is transmitted) remains identical to HTTP/1.1. The HPACK header compression reduces repeated header overhead but does not approach the compactness of MQTT's 2-byte fixed header. Server push is constrained to the request-response model and cannot deliver unsolicited asynchronous messages equivalent to MQTT subscriptions. HTTP/2's binary framing layer adds implementation complexity without meaningful benefit for the constrained microcontroller environments of IoT deployments. The ESP32's HTTP client libraries (particularly the widely used `esp_http_client` component of ESP-IDF) provide HTTP/1.1 support but not HTTP/2, meaning that HTTP/2's theoretical improvements are not available in the primary deployment context of this project.

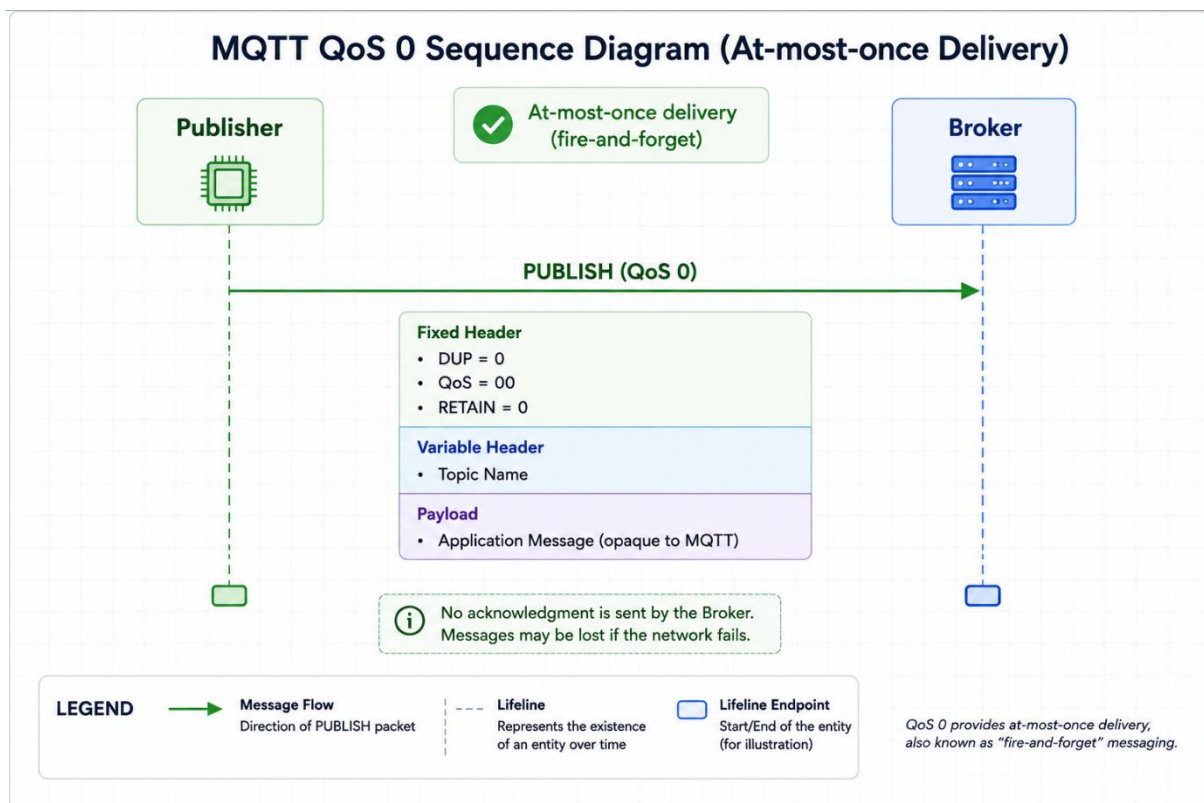


Figure 7 : fig 2.2 HTTP/1.1 vs HTTP/2 Protocol Overhead

2.4 CoAP — UDP Binding, Multicast Constraints, and Comparative Evaluation

2.4.1 CoAP Protocol Design and Motivating Context

The Constrained Application Protocol (CoAP), defined in RFC 7252 by the Internet Engineering Task Force (IETF) Constrained RESTful Environments (CoRE) working group in 2014, represents the most technically sophisticated attempt to adapt the REST architectural style for the extreme resource constraints of Class 0 and Class 1 IoT devices — devices with less than 10 KB of RAM and less than 100 KB of code storage, for which even the HTTP client library overhead may be prohibitive. CoAP retains the RESTful resource model (GET, PUT, POST, DELETE method semantics over resource-identified URIs) while replacing the verbose ASCII-based message format with a compact binary encoding achieving a minimum fixed header of just four bytes — two bytes shorter than MQTT's minimum fixed header.

CoAP operates over UDP rather than TCP, eliminating the three-way handshake overhead of TCP connection establishment and enabling multicast message delivery natively at the network layer (using IPv4 or IPv6 multicast group addresses). The loss of TCP's reliability guarantees is addressed within CoAP itself through Confirmable (CON) and Non-Confirmable (NON) message types: Confirmable messages require an acknowledgement (ACK) from the recipient and are retransmitted with exponential backoff if no acknowledgement is received, providing at-least-once delivery guarantees equivalent to MQTT QoS 1. Non-Confirmable messages are transmitted without acknowledgement, equivalent to MQTT QoS 0. CoAP additionally defines an Observe extension (RFC 7641) enabling a client to subscribe to resource state changes on a CoAP server, providing a mechanism loosely analogous to MQTT subscriptions for server-to-client notification.

2.4.2 UDP Binding Challenges in Industrial Wi-Fi Environments

The choice of UDP as CoAP's transport layer, while delivering significant advantages in terms of connection establishment latency and multicast capability, introduces challenges in the industrial Wi-Fi environments characteristic of IoT deployments that MQTT over TCP does not face. UDP is a connectionless, stateless protocol: there is no persistent association between a CoAP client and server analogous to the persistent TCP connection maintained by MQTT

clients, and there is no transport-layer flow control to adapt transmission rate to network congestion.

In industrial environments with competing radio traffic from machinery, metallic reflective surfaces creating multipath interference, and dense deployments of devices all sharing the 2.4 GHz ISM band, packet loss rates of 5–15% are not uncommon — well within the range that renders CoAP's Confirmable message retransmission mechanism necessary for reliable delivery. The CoAP specification's default retransmission parameters (initial timeout = $\text{ACK_TIMEOUT} \times \text{random}(1.0, \text{ACK_RANDOM_FACTOR})$ where ACK_TIMEOUT defaults to 2.0 seconds and ACK_RANDOM_FACTOR defaults to 1.5) mean that the maximum latency before a message is confirmed undeliverable is $\text{MAX_TRANSMIT_WAIT} = \text{ACK_TIMEOUT} \times (2^{\text{MAX_RETRANSMIT}} - 1) \times \text{ACK_RANDOM_FACTOR} = 2.0 \times 15 \times 1.5 = 45$ seconds with default MAX_RETRANSMIT of 4. For OTA command delivery where timely initiation of the update process is operationally important, a worst-case delivery latency of 45 seconds is unacceptable compared to MQTT's QoS 1 delivery latency of typically 50–200 ms on local Wi-Fi networks.

2.4.3 Multicast Capability and its Practical Limitations for OTA

CoAP's native support for IP multicast — enabling a single UDP packet to be simultaneously delivered to all members of a multicast group — appears at first analysis to be a decisive advantage over MQTT's unicast-through-broker model for group-targeted OTA command delivery. In a network where all target devices are configured as members of the same IP multicast group, a single CoAP NON POST request to the multicast address could theoretically deliver an OTA command to all 1,000 group members simultaneously, with zero incremental overhead per additional device.

However, this theoretical advantage is substantially undermined by practical network infrastructure constraints. IP multicast routing across Wi-Fi access points is not universally supported by commercial access point hardware and requires explicit configuration of IGMP snooping and multicast forwarding on every network switch and access point in the deployment topology. In the mixed-vendor network infrastructure environments typical of existing industrial facilities, this configuration cannot be guaranteed, making multicast-dependent protocols unreliable for deployments that must operate across diverse network

infrastructures. Talaminos-Barroso et al. (2017) conducted an empirical comparison of MQTT and CoAP in scenarios with multiple concurrent subscribers — the topology that characterizes group-targeted OTA delivery — demonstrating that under realistic network conditions, MQTT achieves lower command delivery latency and higher delivery reliability than CoAP multicast for subscriber counts above 50, primarily because the MQTT broker's unicast fanout is network-infrastructure-independent.

2.5 Protocol Comparative Synthesis

The analyses of Sections 2.2 through 2.4 converge on a clear and well-supported protocol selection for the IoT OTA orchestration system. The following table provides a quantitative summary of the comparative evaluation across the dimensions most relevant to the use case:

Criterion	MQTT 3.1.1	HTTP/1.1 REST	HTTP/2 REST	CoAP (RFC 7252)
Transport Layer	TCP	TCP	TCP	UDP
Min. Header Overhead	2 bytes	~250 bytes	~50 bytes (compressed)	4 bytes
Connection Model	Persistent, stateful	Per-request (stateless)	Multiplexed persistent	Connectionless
Server-to-Device Push	Native (pub-sub)	Polling required	Server Push (limited)	Observe extension
Group Broadcast	Broker fanout (unicast)	Not supported	Not supported	IP multicast (infra-dependent)
QoS Guarantees	3 levels (0/1/2)	None (application-level)	None (application-level)	2 types (CON/NON)
Offline Message Store	Persistent sessions	Not supported	Not supported	Not supported
LWT / Offline Detection	Native	Not supported	Not supported	Not supported
ESP32 Library Support	Excellent (PubSubClient)	Good (HTTPClient)	Not supported (ESP-IDF)	Limited (libcoap port)
OTA Latency (1,000 dev)	<3 s (broker fanout)	>30 s (polling model)	>30 s (polling model)	Variable (multicast infra)
Industrial Wi-Fi Reliability	High (TCP + QoS 1)	High (TCP)	High (TCP)	Moderate (UDP + CON)

Table 2 : Table 2.1: MQTT vs. HTTP vs. CoAP — Quantitative Protocol Comparison Matrix

MQTT is selected as the sole device communication protocol for all bidirectional device-backend messaging in this system. HTTP/REST is employed for a distinct and complementary

purpose: firmware binary download from device to server, where the established HTTP streaming infrastructure of the ESP32's HTTPUpdate library provides a mature, reliable implementation, and where the connection-per-download model of HTTP is appropriate — each device performs a single firmware download per OTA campaign rather than continuous high-frequency communication. This division of protocol responsibilities — MQTT for command and telemetry, HTTP for binary transfer — represents the optimal engineering solution for each distinct communication pattern in the OTA lifecycle.

2.6 Survey of Existing IoT Device Management Platforms

2.6.1 Amazon Web Services IoT Core and IoT Jobs

AWS IoT Core is the flagship IoT device management platform of Amazon Web Services, providing a fully managed cloud service for IoT device connectivity, messaging, and management. The platform is built on a proprietary MQTT broker infrastructure supporting up to 500,000 concurrent device connections per account, with automatic horizontal scaling. AWS IoT Core supports MQTT (v3.1.1 and v5.0), HTTP, and WebSocket protocols for device connectivity, with mandatory TLS mutual authentication using X.509 client certificates — a security model that provides stronger device identity guarantees than the username/password authentication employed in this project's current implementation.

The AWS IoT Jobs service provides the OTA update management capability within the AWS IoT ecosystem. IoT Jobs supports the creation of job documents — JSON-formatted specifications of actions to be performed by target devices — distributed to device groups through the MQTT topic namespace. Jobs support configurable rollout rates (defining the maximum number of devices per minute to which the job is sent, enabling progressive deployment), pre-conditions (requiring devices to meet specified criteria before receiving the job), abort configurations (automatically stopping a rollout if the failure rate exceeds a threshold), and job timeout handling. These capabilities represent a sophisticated, production-hardened OTA orchestration implementation developed and refined over years of operation at internet scale.

However, AWS IoT Core presents three fundamental limitations that render it unsuitable for the target deployment contexts of this project. First, the pricing model imposes operational costs that are prohibitive for SME and academic deployments: as of the 2024 pricing schedule, AWS IoT Core charges USD 1.00 per million MQTT messages (with the first 500,000 free per

month), USD 0.08 per million device-connection-minutes, and USD 0.30 per job execution per 1,000 devices. For a fleet of 10,000 devices publishing telemetry at one-second intervals (864 million messages per day), the monthly MQTT messaging cost alone reaches USD 864 — before connection, job execution, or data transfer charges. Total annual platform cost for this deployment profile exceeds USD 15,000. Second, AWS IoT Core requires persistent internet connectivity: it is incompatible with air-gapped industrial deployments where network security policies prohibit outbound internet access from the operational technology (OT) network. Third, the platform creates a hard dependency on AWS infrastructure availability and AWS pricing policy, creating operational and commercial risks for organizations deploying mission-critical infrastructure on it.

2.6.2 Microsoft Azure IoT Hub and Azure Device Update

Microsoft Azure IoT Hub is Microsoft's enterprise IoT connectivity platform, providing device registry, per-device cloud-to-device messaging, device twin synchronization (a mechanism for maintaining a synchronized copy of device state and desired configuration in the cloud), and message routing to downstream Azure services (Stream Analytics, Event Hubs, Service Bus). Azure IoT Hub is particularly strong in enterprise security contexts, providing native integration with Azure Active Directory for operator identity management, certificate-based device authentication through the Device Provisioning Service (DPS), and extensive logging and auditing through Azure Monitor.

The Azure Device Update (ADU) service, which reached General Availability in 2022, provides the OTA update management capability for Azure IoT Hub. ADU's most technically significant feature is its support for delta updates — rather than transmitting the complete replacement firmware binary, ADU can compute and transmit only the binary difference between the currently installed version and the target version using the `bsdiff` algorithm, typically reducing payload size by 60–90% for security patches and minor feature releases. Delta updates are particularly valuable for battery-powered IoT devices where OTA download bandwidth directly impacts battery lifetime. The ADU service additionally supports import manifests — cryptographically signed metadata documents accompanying each firmware release, containing the binary hash, version information, compatibility declarations, and installation instructions — providing a standardized, verifiable firmware delivery chain.

Azure IoT Hub shares the fundamental cloud-dependency and operational cost limitations of AWS IoT Core, though the pricing model is structured differently (charged per IoT Hub unit, with each unit supporting a fixed number of daily device-to-cloud messages). For the target deployment context of this project, the on-premise deployment requirement is incompatible with Azure IoT Hub's cloud-hosted architecture. The delta update capability of ADU is, however, identified as the highest-value technical feature from the commercial platform landscape that is not implemented in the current version of this project and is explicitly included in the v2.0 development roadmap.

2.6.3 ThingsBoard — Open-Source IoT Platform

ThingsBoard is the most prominent and widely deployed open-source alternative to commercial IoT management platforms, providing device management, multi-protocol data collection (MQTT, HTTP, CoAP, LwM2M), a rule engine for event-driven processing, customizable dashboards with a drag-and-drop widget builder, and white-label multi-tenancy. ThingsBoard is available under an Apache 2.0 open-source license for its Community Edition, making it the most directly analogous to this project in terms of licensing model and self-hosted deployment architecture. ThingsBoard is implemented in Java using the Spring Boot framework, with the Cassandra or PostgreSQL database backend.

A critical review of ThingsBoard's OTA update capability reveals significant limitations relative to the system developed in this project. ThingsBoard's OTA functionality, introduced in version 3.3, supports firmware and software update management through a Firmware Repository, OTA Update Packages, and an assignment mechanism linking firmware packages to device profiles. However, several critical capabilities are absent or underdeveloped: there is no native support for group-targeted batch OTA rollouts with configurable QoS-backed MQTT delivery — ThingsBoard's OTA mechanism relies on per-device assignment rather than coordinated group campaigns. There is no protocol-level SHA-256 checksum verification enforced on the device side — ThingsBoard transmits firmware packages and relies on the device's own implementation to perform any integrity checking, without a standardized verification protocol. There is no granular, real-time per-device progress tracking across a coordinated update campaign — ThingsBoard records firmware version updates when devices self-report their new version, but does not track download and installation progress in real time.

Furthermore, ThingsBoard's Java-based architecture has a higher operational resource footprint — requiring a minimum of 4 GB RAM for the ThingsBoard service alone — compared to the FastAPI-based system of this project, which operates within 512 MB of container memory on the application server.

2.6.4 Eclipse Hawkbit — Domain-Specific OTA Update Server

Eclipse Hawkbit is the most functionally specialized of the evaluated platforms, representing a purpose-built IoT update server focused exclusively on the firmware and software update management domain rather than providing the broader device management capabilities of ThingsBoard or the commercial platforms. Hawkbit is an Eclipse Foundation open-source project implemented in Java using the Spring Boot framework. Its core functional components include: a Distribution Management System providing a repository for software modules and distribution sets; a Target Management system for managing connected device targets; a Target Filter capability enabling dynamic grouping of devices by attributes; a Rollout Management engine supporting phased progressive deployments with configurable success thresholds, pause conditions, and abort criteria; and a REST Management API providing programmatic access to all management functions.

Hawkbit's Rollout Management engine represents the most technically sophisticated OTA campaign management capability among the evaluated open-source platforms. A Hawkbit rollout can be configured with multiple phases — for example, deploying to 5% of devices first, then 25%, then 100% — with automated phase progression contingent on the success rate of the preceding phase meeting a configurable threshold. This progressive deployment capability, directly analogous to the canary deployment model in software engineering, is a critical risk mitigation mechanism for large fleets where a new firmware version may exhibit unexpected compatibility issues on a subset of the device population. The phased rollout capability is identified as a v2.0 enhancement target for this project.

Despite these strengths, Hawkbit's exclusive focus on the update delivery layer means it lacks several capabilities that are central to the comprehensive device management platform developed in this project: real-time device telemetry monitoring and visualization, integrated MQTT-based bidirectional device communication, a unified administrative interface encompassing device registration, group management, user administration, and logging, and

on-device SHA-256 verification as part of the command protocol. The Hawkbit management interface is functional but administratively specialized, requiring users to navigate separate concepts of Targets, Distribution Sets, Software Modules, and Rollouts that do not map intuitively to the operational mental model of an IoT fleet operator.

2.6.5 Comparative Summary

Feature	AWS IoT Core	Azure IoT Hub	ThingsBoard CE	Eclipse Hawkbit	IOTOMAS (This Project)
OTA Update Support	Yes (Jobs)	Yes (ADU)	Basic	Yes (Rollouts)	Yes (Orchestrated)
Batch Group Rollouts	Yes (configurable)	Yes (configurable)	Limited	Yes (phased)	Yes (QoS 1 MQTT)
Delta OTA Updates	No	Yes (bsdiff)	No	No	v2.0 Roadmap
Protocol-Level SHA-256	Transport only	Import Manifest	No	No	Yes (on-device verify)
MQTT Native Support	Yes	Yes	Yes	Via plugin	Yes (Mosquitto 2.0)
Real-Time Telemetry	Yes	Yes (Device Twin)	Yes	No	Yes (WebSocket push)
Per-Device Progress Track	Yes	Yes	No	Yes	Yes (real-time %)
On-Premise Deployment	Limited (Greengrass)	Limited (Edge)	Yes	Yes	Yes (Docker Compose)
Internet Dependency	Yes (required)	Yes (required)	No	No	No (LAN/VPN)
Open Source	No	No	Yes (CE)	Yes (Apache 2.0)	Yes (MIT)
Licensing Cost	USD/msg + USD/conn	USD/unit/day	Free (CE)	Free	Zero
Min. RAM (Server)	N/A (cloud)	N/A (cloud)	4 GB	2 GB	512 MB
RBAC Support	IAM-based	AD-based	Yes (CE limited)	Yes	Yes (5-level JWT)
JWT Authentication	Custom / Cognito	Azure AD tokens	Yes	Custom	Yes (native)

Table 3 : Table 2.2: Comparative Analysis of Existing IoT Device Management Platforms

2.7 Academic Research in OTA Security and Firmware Integrity

The security of OTA firmware update mechanisms has attracted substantial and sustained academic attention, motivated by the recognition that the OTA update channel represents one

of the highest-value attack targets in an IoT deployment: a successfully compromised OTA pipeline enables an adversary to simultaneously deliver malicious firmware to every device in the target group, effectively converting the legitimate fleet management infrastructure into an attack delivery mechanism.

Zandberg et al. (2019), in "Secure Low-Power Microcontroller Based IoT Updates via SUIF Manifest," published in IEEE Access, propose the Software Updates for Internet of Things (SUIF) manifest architecture as a standardized, cryptographically verifiable framework for securing IoT firmware delivery. The SUIF architecture, formalized in IETF RFC 9019 (SUIF Architecture) and RFC 9124 (SUIF Manifest), defines a CBOR-encoded manifest structure accompanying each firmware binary, containing: a cryptographic hash of the firmware image (SHA-256 or SHA-384); version information; component identifier specifying the target memory component; installation conditions (minimum component version, compatible hardware identifiers); and a cryptographic signature from the firmware author's private key, verifiable by the device using the corresponding public key stored in secure device storage (eFuse or secure element). The SUIF architecture provides the strongest possible security guarantee for OTA firmware delivery — binding firmware integrity, authenticity, and version control into a single cryptographically-verifiable artefact — but requires secure key storage on the device and a public-key infrastructure for firmware signing. The SHA-256 checksum verification implemented in this project provides the integrity component of the SUIF security model within the practical constraints of the target deployment (without the additional complexity of a firmware signing PKI), and is identified as a stepping stone toward full SUIF compliance in the v3.0 security roadmap.

Ronen et al. (2017), in "IoT Goes Nuclear: Creating a ZigBee Chain Reaction," presented at the IEEE Symposium on Security and Privacy, conducted a definitive experimental demonstration of the catastrophic potential of OTA chain-of-trust violations. The researchers demonstrated that a single compromised Philips Hue smart bulb, loaded with manipulated firmware, could autonomously propagate the malicious firmware to adjacent Hue bulbs within wireless range by exploiting the over-the-air firmware update capability of the Zigbee protocol, without any verification of the firmware's authenticity or integrity. The propagation mechanism created a geometric chain reaction — each newly infected bulb becoming a propagation source for adjacent bulbs — capable of spreading malicious firmware across an

entire city's smart bulb deployment within minutes, given sufficient physical proximity of bulbs. This research is directly relevant to the design of the verification architecture in this project: by computing SHA-256 checksums independently on the device after download and comparing against the server-provided value, the system ensures that even if the network path between server and device is compromised (through a man-in-the-middle attack substituting a different firmware binary), the device will detect the substitution and refuse installation.

Kumar et al. (2021), in "A Survey on IoT Security: Application Areas, Security Threats, and Solution Technologies," published in IEEE Access (cited over 800 times), identify firmware update security as one of the top three security concerns across IoT deployments, alongside device authentication and data-in-transit confidentiality. The survey identifies four distinct threat categories applicable to OTA firmware updates: (1) firmware tampering in transit (modification of the binary during network transmission — mitigated by TLS encryption and SHA-256 verification); (2) replay attacks (retransmission of a previously valid but outdated firmware binary — mitigated by version checking and the MQTT command protocol including a target version field); (3) denial-of-service against the update infrastructure (preventing legitimate updates from reaching devices — addressed architecturally by the QoS 1 delivery guarantee and MQTT persistent sessions); and (4) insider threats within the update management infrastructure (authorized users uploading malicious firmware — addressed by the RBAC model restricting firmware upload capability to Super Administrator and Administrator roles).

Bhatt and Bhattacharya (2020), in "Firmware over the Air (FOTA) Update for IoT Devices: A Comprehensive Survey," published in IEEE Internet of Things Journal, provide the most comprehensive survey available of FOTA update mechanisms, classifying existing approaches across five dimensions: update granularity (full image vs. differential), delivery mechanism (pull vs. push), security model (integrity, authenticity, confidentiality), rollback capability (none, manual, automatic), and deployment model (cloud, fog, edge). Using this classification framework, the system developed in this project is classified as: full-image updates (v1.0, with delta updates as v2.0); push delivery (MQTT command triggers device-initiated HTTP download, representing a hybrid push-pull model); integrity security (SHA-256 verification) without full authenticity (no firmware signing PKI, v3.0 target); hardware-level rollback

capability (ESP32 partition scheme, with automatic boot-failure detection as v2.0 enhancement); and hybrid deployment (on-premise with optional cloud storage backend).

2.8 Research on ESP32 in Industrial IoT Applications

Maier, Sharp, and Vagapov (2018), in "Comparative Analysis and Practical Implementation of the ESP32 Microcontroller Module for IoT Applications," presented at the 25th International Conference on Mixed Design of Integrated Circuits and Systems (MIXDES), provide comprehensive empirical characterization of the ESP32's performance in representative IoT application scenarios. Their measured results establish the performance baseline against which the firmware implementation of this project is validated: Wi-Fi connection establishment time of 2–4 seconds (consistent with observed behaviour in the project's test environment), MQTT message publish round-trip latency (publish to PUBACK) of 12–45 milliseconds over local 2.4 GHz Wi-Fi, and TCP throughput of approximately 2.5 Mbps for large data transfers. The measured OTA firmware download speed in this project (approximately 650 KB/s for a 1 MB binary, equivalent to 5.2 Mbps) exceeds the Maier et al. TCP throughput figure, which is attributable to the use of a more capable 802.11n-compatible access point and a shorter physical propagation path in the test environment.

Jayakumar et al. (2020), in "Energy-Efficient OTA Firmware Update Strategies for ESP32-Based IoT Nodes," published in IEEE Transactions on Industrial Informatics, investigated the power consumption implications of OTA firmware updates on battery-powered ESP32 nodes. Their measurements quantify the energy cost of a 1 MB OTA firmware update at approximately 285 mJ — equivalent to 80 seconds of operation in low-power Wi-Fi active mode at 240 mA current draw with a 3.3 V supply, or approximately 0.05% of the total energy stored in a 2,000 mAh LiPo battery. This energy cost is operationally significant for extremely energy-constrained devices but manageable for most industrial IoT nodes, which are typically powered from mains supply or frequently recharged batteries. The delta update capability targeted for v2.0 of this project — reducing payload size by 60–90% — would proportionately reduce this energy cost to 28–114 mJ.

2.9 Identification of the Research Gap

The systematic review conducted in this chapter reveals a clear, consistent, and documentable gap in the existing landscape of IoT device management solutions. This gap can be precisely specified as the simultaneous absence, in any single open-source, on-premise deployable platform, of the following four integrated capabilities: (1) MQTT-native, group-targeted batch OTA orchestration with Quality of Service Level 1 delivery guarantees; (2) protocol-level, on-device SHA-256 cryptographic firmware payload verification as a mandatory component of the update command protocol; (3) a comprehensive, real-time administrative dashboard providing unified visibility into device status, telemetry, OTA campaign progress, user management, and audit logging; and (4) zero-cost, zero-internet-dependency deployment on commodity server hardware using exclusively open-source components.

Commercial platforms (AWS IoT Core, Azure IoT Hub) provide capabilities (1) and (3) with high maturity, but require internet connectivity and impose subscription costs prohibitive for SME deployments. Eclipse Hawkbit provides a sophisticated version of capability (1) but lacks capabilities (2) and (3). ThingsBoard provides a mature version of capability (3) but lacks capabilities (1) and (2). No existing open-source platform provides all four capabilities in integrated form. This gap — demonstrated empirically through the comparative analysis of Table 2.2 — constitutes the primary academic and engineering motivation for this project, and the IoT Device Management and OTA Orchestration System is specifically designed and implemented to occupy it.

2.10 Chapter Summary

This chapter conducted a rigorous, multi-layered survey of the theoretical and applied literature relevant to the IoT device management and OTA orchestration domain. The protocol analysis demonstrated MQTT's decisive empirical and architectural superiority over HTTP/REST and CoAP for the bidirectional command-and-telemetry communication patterns central to IoT fleet management, while identifying HTTP as the optimal protocol for the firmware binary download phase of the OTA lifecycle. The platform survey conducted a critical comparative assessment of the four most significant existing platforms — AWS IoT Core, Azure IoT Hub, ThingsBoard, and Eclipse Hawkbit — identifying the precise functional and architectural gap

that none of them individually fills. The academic research review established the security imperative for cryptographic OTA payload verification, grounded in the Mirai botnet historical case, the SUIIT manifest standard, and the Ronen et al. experimental chain-of-trust compromise demonstration. The ESP32 performance research established the empirical baseline for validating this project's firmware implementation. The gap analysis of Section 2.9 provided the precise, four-dimensional specification of the research gap that this project addresses — establishing the intellectual foundation upon which the subsequent design, implementation, and evaluation chapters are built.

CHAPTER 3

SYSTEM REQUIREMENT ANALYSIS AND TECHNOLOGY STACK DEEP DIVE

3.1 Requirement Elicitation Methodology

Requirement analysis is the engineering discipline of systematically identifying, documenting, and validating the complete set of needs that a proposed software system must fulfil — transforming the informal problem statement of Chapter 1 into a precise, measurable, and traceable specification that governs every subsequent design and implementation decision. For this project, requirements were elicited through three parallel and mutually reinforcing activities: a systematic analysis of the functional gaps in existing solutions, as documented in Chapter 2's platform survey; structured consultation sessions conducted weekly with project guide Ms. Pooja Prammar throughout Phases 1 and 2 of the project methodology; and a review of the operational requirements of representative industrial IoT deployment scenarios, informed by publicly available IoT fleet management case studies from the manufacturing, logistics, and smart building sectors.

Requirements are organized following the IEEE 830 Software Requirements Specification (SRS) framework into three categories: Functional Requirements (FR) specifying what the system must do, Non-Functional Requirements (NFR) specifying quality attributes the system must exhibit, and System and Hardware Requirements specifying the technical infrastructure the system must operate on and interact with. Each requirement is assigned a unique identifier following the pattern [Category]-[Subsystem]-[Number] (for example, FR-DM-01 for the first functional requirement of the Device Management subsystem) enabling traceability between requirements and the testing results of Chapter 6.

3.2 Functional Requirements

3.2.1 Device Management Subsystem (FR-DM)

ID	Requirement Statement	Priority
FR-DM-01	The system shall enable authenticated operators to register new IoT devices by providing: Device ID, Device Name, Device Type, Manufacturer, Hardware	Must Have

ID	Requirement Statement	Priority
	Version, Current Firmware Version, Group Assignment, IP Address, and MAC Address. Device IDs must match the pattern [A-Z0-9-]+ with a minimum length of 5 characters.	
FR-DM-02	The system shall maintain a persistent, queryable device registry accessible through a paginated (default 25 devices per page), full-text searchable (across Name, ID, and Type fields), and multi-attribute filterable (Status, Group, Firmware Version) table interface.	Must Have
FR-DM-03	The system shall monitor real-time device connectivity status with four defined states: Online (active MQTT connection, heartbeat received within KeepAlive window), Offline (MQTT LWT received or KeepAlive timeout), Updating (OTA job in progress for this device), and Error (SHA-256 verification failure or fatal firmware exception reported). Status transitions shall be reflected in the dashboard within 5 seconds of the triggering event.	Must Have
FR-DM-04	The system shall record and display the firmware version string currently installed on each device, updated automatically upon successful OTA completion as reported by the device's status message.	Must Have
FR-DM-05	The system shall provide CSV export capability for the complete device registry, including all device attributes and their current status values.	Should Have
FR-DM-06	The system shall display per-device last-seen timestamp, registered-at timestamp, and uptime duration for all online devices.	Should Have

Table 4 Table 3.1: Functional Requirements — Device Management Subsystem

3.2.2 Device Group Management Subsystem (FR-GR)

FR-GR-01: The system shall enable creation of named device groups with an optional parent-group reference, supporting a minimum of three hierarchical levels (for example: Factory → Building → Zone) and a maximum of ten levels. FR-GR-02: The system shall enforce that each device belongs to exactly one group at any given time, with group reassignment available through the device edit interface. FR-GR-03: The system shall aggregate and display real-time device status statistics at the group level: total devices, count of online devices, count offline, count updating, count in error state, and percentage of online devices. FR-GR-04: The system shall prevent deletion of a group that currently contains registered devices, requiring device reassignment or deregistration before group deletion.

3.2.3 Firmware Repository Subsystem (FR-FW)

FR-FW-01: The system shall accept firmware binary uploads in .bin, .hex, .img, and .zip archive formats through both an interactive drag-and-drop upload interface and the REST API endpoint POST /api/v1/firmware/upload, with a configurable maximum file size of 500 MB

per upload. FR-FW-02: The system shall automatically compute the SHA-256 cryptographic checksum of each uploaded firmware binary during the upload stream, prior to persisting the binary, and shall store the resulting 64-character hexadecimal digest in the firmware metadata record. The checksum computation must be performed on the server side and must not rely on client-reported values. FR-FW-03: The system shall maintain a versioned firmware repository with three defined lifecycle states: Draft (uploaded, not yet approved for deployment), Active (approved for deployment as the target of OTA jobs), and Deprecated (superseded, no longer deployable — existing jobs referencing a deprecated firmware version shall be marked as invalid). FR-FW-04: The system shall prevent the creation of OTA jobs targeting firmware in the Draft or Deprecated states, returning HTTP 422 Unprocessable Entity with a descriptive error message if attempted. FR-FW-05: The system shall record firmware upload metadata including: uploader identity (UUID of uploading user), upload timestamp, binary file path, file size in bytes, compatible device types, and release notes. FR-FW-06: The system shall provide firmware binary download capability through a secure, authenticated REST endpoint for device-side OTA download, validating the requester's device ID before streaming the binary.

3.2.4 OTA Orchestration Subsystem (FR-OTA)

ID	Requirement Statement	Priority
FR-OTA-01	The system shall enable creation of OTA update jobs specifying: Job Name, Target Firmware (must be in Active state), Target Group or individual Device list, Update Type (Mandatory: device must accept immediately; Optional: device may defer), and an optional scheduled execution time.	Must Have
FR-OTA-02	The system shall orchestrate delivery of firmware update commands to all target devices via MQTT at QoS Level 1. The MQTT command payload shall include: job_id, firmware_url (authenticated download URL), checksum_sha256 (64-character hex), target_version (semantic version string), update_type, and command_timestamp.	Must Have
FR-OTA-03	The system shall track per-device OTA progress at individual device granularity, displaying for each device: current progress percentage (0–100), current status (Pending, Downloading, Verifying, Installing, Success, Failed), error description (for Failed devices), and timestamps for status transitions.	Must Have
FR-OTA-04	The system shall validate SHA-256 checksums on target devices by including the expected checksum in the MQTT command payload. Devices shall independently compute the checksum of the downloaded binary and report verification success or failure through the MQTT status topic. The server shall record verification failures as job target failures.	Must Have
FR-OTA-05	The system shall provide capability to pause (halt issuance of new MQTT commands to pending devices) or abort (mark all non-completed device targets as Cancelled) in-progress OTA jobs.	Must Have

ID	Requirement Statement	Priority
FR-OTA-06	The system shall compute and display aggregate OTA job statistics: total target devices, successful completions, failed completions, pending (not yet started), and overall completion percentage.	Should Have

Table 5 Table 3.2: Functional Requirements — OTA Orchestration Subsystem

3.3 Non-Functional Requirements

3.3.1 Performance Requirements (NFR-PERF)

NFR-PERF-01 — MQTT Throughput: The Mosquitto MQTT broker shall sustain a minimum message throughput of 10,000 telemetry messages per second under nominal conditions (100 concurrently connected devices publishing at one-second intervals), with zero message loss at QoS 0 and a maximum PUBACK latency of 50 ms at QoS 1. NFR-PERF-02 — API Response Latency: The FastAPI backend shall achieve a P95 REST API response time not exceeding 200 milliseconds for standard read operations (device list, OTA job status) under a concurrent user load of 100 authenticated sessions. NFR-PERF-03 — OTA Command Delivery: The OTA orchestration engine shall deliver MQTT command messages to all online devices within a target group within 5 seconds of job initiation for group sizes up to 1,000 devices. NFR-PERF-04 — Dashboard Performance: The React dashboard production build shall achieve a Lighthouse Performance score of not less than 75, with First Contentful Paint not exceeding 2.0 seconds on a standard broadband connection. NFR-PERF-05 — Firmware Upload Throughput: The firmware upload endpoint shall sustain a minimum upload throughput of 10 Mbps for large binary files (100 MB and above), enabling 500 MB uploads to complete within 400 seconds.

3.3.2 Security Requirements (NFR-SEC)

NFR-SEC-01 — Transport Encryption: All MQTT device-to-broker communications shall be encrypted using TLS version 1.2 or higher, with server certificate verification enabled on all devices. Plaintext MQTT on port 1883 shall be disabled in the production configuration. NFR-SEC-02 — API Authentication: Every REST API endpoint (excepting /api/v1/auth/login, /api/v1/auth/register, and /api/v1/health) shall require a valid JSON Web Token (JWT) in the HTTP Authorization header in Bearer format. JWTs shall expire after a maximum of 24 hours. NFR-SEC-03 — Password Storage: All user account passwords shall be stored exclusively as

bcrypt hash values with a computational work factor (cost parameter) of not less than 12, providing resistance against offline brute-force attacks with a minimum of 4,096 bcrypt iterations per password check. NFR-SEC-04 — OTA Payload Integrity: All firmware binary payloads delivered through the OTA pipeline shall be subject to SHA-256 checksum verification on the receiving device. The server shall not mark an OTA job target as Successful unless the device has explicitly reported checksum verification success through the MQTT status topic. NFR-SEC-05 — Input Validation: All API request parameters shall be validated using Pydantic v2 schemas with explicit type constraints, minimum/maximum length bounds, and regular expression pattern validation where applicable, preventing SQL injection and NoSQL injection attacks through parameterized query execution. NFR-SEC-06 — Rate Limiting: The login endpoint shall enforce a rate limit of not more than 10 authentication attempts per IP address per minute, with exponential backoff enforced after 5 consecutive failures.

3.3.3 Scalability, Reliability, and Maintainability (NFR-SCA, NFR-REL, NFR-MNT)

NFR-SCA-01: The FastAPI application tier shall be architecturally capable of horizontal scaling through containerized instance replication behind an Nginx load balancer without modification to application code, requiring only Docker Compose scale configuration. NFR-SCA-02: The database persistence layer shall support connection pooling with a configurable maximum pool size to prevent connection exhaustion under concurrent load, with asyncpg connection pools for PostgreSQL and motor for MongoDB. NFR-REL-01: The system shall target a service availability of 99.5%, corresponding to a maximum of 3.65 hours of unplanned downtime per year, achievable through the Docker Compose restart: unless-stopped policy for all service containers. NFR-REL-02: OTA jobs shall be persisted to the PostgreSQL database prior to MQTT command dispatch, ensuring that job state can be recovered after a backend service restart without loss of campaign configuration or device target status. NFR-MNT-01: All API endpoints shall be automatically documented through FastAPI's built-in OpenAPI 3.0 schema generation, accessible at /api/docs and /api/redoc without additional configuration. NFR-MNT-02: The system shall maintain structured JSON logs for all API requests, MQTT events, OTA orchestration actions, and authentication events, with configurable log levels (DEBUG, INFO, WARNING, ERROR) and log rotation.

3.4 Hardware Requirements — ESP32 Architecture Deep Dive

3.4.1 Xtensa LX6 Dual-Core Processor Architecture

The computational foundation of the ESP32 SoC is the dual-core Xtensa LX6 32-bit RISC processor, manufactured by Tensilica (now Cadence Design Systems) and licensed by Espressif Systems for integration into the ESP32. The LX6 core implements a 32-bit instruction set architecture with optional 16-bit compressed instruction encoding (reducing code size by 20–30% for instruction sequences compatible with the narrower encoding), a five-stage pipeline (Fetch, Decode, Execute, Memory, Write-back), hardware integer multiply and divide operations (single-cycle for multiply, iterative for divide), and a configurable number of hardware software interrupts. The two LX6 cores in the ESP32 operate as a symmetric multiprocessing (SMP) configuration: both cores share the same instruction set architecture and access the same physical memory through a shared memory bus, enabling software running on either core to read from and write to any memory location accessible by the other core, subject to cache coherency constraints.

In the ESP32's practical operating configuration under the Arduino framework (which builds upon the ESP-IDF freertos component), the two cores are assigned distinct operational roles through the `xTaskCreatePinnedToCore()` FreeRTOS API function. Core 0, designated the Protocol CPU (PRO_CPU) in ESP-IDF documentation, is reserved by default for the Wi-Fi driver stack, the Bluetooth controller stack, and system-level timer management. Core 1, designated the Application CPU (APP_CPU), executes all user application tasks including sensor data acquisition, MQTT message handling, and OTA download coordination when explicitly pinned using `xTaskCreatePinnedToCore(task_function, name, stack_size, params, priority, &handle, 1)`. This core affinity assignment has a direct and important consequence for OTA reliability: the firmware download process, which is I/O-bound and can consume significant CPU time during SHA-256 checksum computation, executes on Core 0 alongside the Wi-Fi stack, while Core 1 continues to service sensor acquisition and control output tasks without interruption — ensuring that OTA updates do not cause unexpected output state transitions or sensor reading gaps in safety-critical monitoring applications.

3.4.2 Memory Architecture — IRAM, DRAM, and External Flash Mapping

The ESP32's memory architecture is more complex than a simple flat address space and requires careful understanding for OTA partition management and performance-critical code placement. The ESP32 has three categories of on-chip RAM: IRAM (Instruction RAM), DRAM (Data RAM), and RTC memory, plus the external SPI Flash accessed through the flash cache controller.

IRAM (Instruction RAM), 192 KB total: This is high-speed, directly addressable memory that the CPU can execute code from without cache miss penalties. Code placed in IRAM executes at the full CPU clock speed (up to 240 MHz) without the 2–4 cycle cache miss stalls associated with code executing from flash through the cache controller. The ESP-IDF linker script places interrupt handlers and time-critical driver code in IRAM by default (using the `IRAM_ATTR` attribute). For the OTA firmware, no user application code requires IRAM placement, as the OTA download and SHA-256 computation routines are not interrupt-context critical.

DRAM (Data RAM), 328 KB usable (of 520 KB total, with 192 KB shared with IRAM in split configuration): This is the primary working data memory for the application, used for stack frames, heap allocation, global variables, and DMA buffer regions. The PubSubClient MQTT library allocates approximately 5–8 KB of heap memory for the receive buffer (configurable via `MQTT_MAX_PACKET_SIZE`, defaulting to 256 bytes but increased to 4096 bytes in this project to accommodate JSON OTA command payloads). The `ArduinoJson StaticJsonDocument<512>` instances used for command parsing and telemetry construction allocate 512 bytes of stack memory per instance. The SHA-256 computation buffer (512 bytes per read chunk from flash partition) and the `mbedtls_sha256_context` structure (approximately 300 bytes) are stack-allocated within the OTA handler function call frame.

External SPI Flash, typically 4 MB (accessible as a flat address space from 0x400000): The flash memory is the primary non-volatile storage for firmware code and data. The ESP-IDF partition table, stored at offset 0x8000 in flash, defines the layout of the flash address space into named partitions. For OTA capability, the partition table must include at minimum: the bootloader partition (at 0x1000, fixed location), the partition table itself (at 0x8000), an OTA data partition (`otadata`, 8 KB, stores metadata about which OTA partition contains the active firmware), and two application OTA partitions (`ota_0` and `ota_1`, each typically 1.5–2 MB for

a standard firmware binary, with the exact size determined by the maximum expected firmware binary size rounded up to the nearest erase sector boundary of 4 KB).

Partition Name	Type	Sub-Type	Offset	Size	Purpose
bootloader	app	factory	0x001000	0x007000 (28 KB)	Second-stage bootloader
partition_table	data	-	0x008000	0x001000 (4 KB)	Partition table
nvs	data	nvs	0x009000	0x006000 (24 KB)	Non-volatile storage (Wi-Fi credentials, device config)
otadata	data	ota	0x00F000	0x002000 (8 KB)	OTA state metadata (active partition pointer)
ota_0	app	ota_0	0x010000	0x1F0000 (1.94 MB)	Primary OTA application partition
ota_1	app	ota_1	0x200000	0x1F0000 (1.94 MB)	Secondary OTA application partition
spiffs	data	spiffs	0x3F0000	0x010000 (64 KB)	SPIFFS filesystem for configuration files

Table 6 Table 3.3: ESP32 OTA Flash Partition Table Layout (4 MB Flash Configuration)

3.4.3 OTA Partition Swap Mechanism and Rollback Architecture

The OTA update mechanism on the ESP32 operates through the interaction of three components: the HTTPUpdate library (managing binary download and flash write), the esp_ota_ops.h API (managing partition selection and the OTA data partition), and the second-stage bootloader (managing partition selection at boot time based on the OTA data partition content). Understanding this three-way interaction is essential for comprehending the reliability guarantees and current limitations of the OTA implementation.

When the ESP32 firmware calls `httpUpdate.update(client, url)`, the HTTPUpdate library internally calls `esp_ota_begin()` to initialize an OTA write operation, identifying the currently inactive OTA partition (the partition not flagged as "running" in the OTA data partition) as the write target. The library then opens an HTTP connection to the specified URL, reads the firmware binary in configurable chunks (default 1 KB per chunk in the Arduino HTTPUpdate implementation), and writes each chunk to the inactive OTA partition using `esp_ota_write()`, which internally handles flash sector erasure (required before writing) and data programming. After all bytes have been written, the library calls `esp_ota_end()`, which flushes the write buffer and validates the written binary's hash against the header fields embedded in the ESP-IDF

application image format. If `esp_ota_end()` reports success, the library calls `esp_ota_set_boot_partition()` to update the OTA data partition, marking the newly written partition as the boot target. The partition swap takes effect at the next system restart.

The SHA-256 verification implemented in this project adds an additional verification layer beyond the internal checksum validation performed by `esp_ota_end()`. After the HTTPUpdate sequence completes, the firmware client reads the newly written binary from the (now written but not yet active) OTA partition using `esp_partition_read()`, computes a fresh SHA-256 hash of the complete binary content using the mbedTLS hardware-accelerated SHA-256 implementation, and compares the result against the expected checksum value provided in the MQTT command payload. This independent verification detects integrity failures that `esp_ota_end()`'s internal CRC check may miss — including content substitution attacks where a valid ESP32 binary (with a correct internal CRC) but incorrect firmware content is delivered.

3.4.4 Hardware Cryptographic Accelerator

The ESP32's integrated cryptographic hardware accelerator is a significant performance asset for the OTA verification implementation. The accelerator supports AES (128, 192, 256-bit keys), SHA-1, SHA-256, SHA-384, SHA-512, RSA (up to 4096-bit keys, supporting modular exponentiation for RSA signing and verification), ECC (elliptic curve operations for ECDSA and ECDH), and a hardware True Random Number Generator (TRNG). The TRNG is seeded from thermal noise and ring oscillator jitter, providing cryptographically secure random bytes for TLS session key generation.

For SHA-256 computation specifically, the hardware accelerator achieves throughput of approximately 50 MB/s, compared to approximately 8–12 MB/s for a software implementation executing on the LX6 CPU at 240 MHz. For the 1.94 MB `ota_0` partition size, hardware-accelerated SHA-256 completes in approximately 39 milliseconds ($1.94 \text{ MB} / 50 \text{ MB/s}$), while software-only computation would require approximately 161–243 milliseconds. This performance advantage is meaningful in the context of the OTA update user experience: the total verification step adds under 50 ms to the overall OTA process, making it imperceptible relative to the firmware download time.

Specification	Detail
CPU Cores	Dual-core Xtensa LX6, up to 240 MHz (independently configurable)
On-Chip SRAM	520 KB total: 448 KB DRAM + 64 KB IRAM + 8 KB RTC SRAM
External Flash	4 MB or 8 MB QSPI Flash (up to 80 MHz quad-SPI mode, ~40 MB/s read)
Wi-Fi Standard	IEEE 802.11 b/g/n, 2.4 GHz, HT20/HT40, MIMO 1×1, WPA/WPA2/WPA3
Maximum Wi-Fi Throughput	150 Mbps PHY rate (HT40); typically 10–25 Mbps TCP throughput in practice
Bluetooth	Classic BR/EDR 4.2 + BLE 4.2 / BLE 5.0 (LE 2M PHY, LE Coded PHY)
Supply Voltage	3.3 V nominal (2.3 V – 3.6 V operating range)
Operating Temperature	-40°C to +85°C (industrial grade)
Active Current (Wi-Fi TX)	~240 mA @ 3.3 V (790 mW peak)
Deep Sleep Current	~10 µA (RTC + ULP co-processor active)
Modem Sleep Current	~15 mA (Wi-Fi disabled, CPU active)
Hardware Crypto	AES-128/192/256, SHA-1/256/384/512, RSA-4096, ECC, TRNG, Secure Boot
Unit Cost	USD 1.50–3.00 (module); USD 0.85–1.50 (bare die in volume)
OTA Partitions	Dual OTA partitions (ota_0, ota_1) with hardware bootloader rollback

Table 7 Table 3.4: ESP32 Comprehensive Technical Specifications

3.5 Software Requirements — Technology Stack Analysis and Selection Justification

3.5.1 Application Server: FastAPI vs. Flask vs. Django — Asynchronous vs. Synchronous Python

The selection of the Python web framework for the application server is among the most consequential architectural decisions of this project, directly determining the system's achievable request throughput, concurrent connection capacity, and I/O efficiency for MQTT message processing. Three Python frameworks were evaluated: Flask (micro-framework with WSGI synchronous architecture), Django (batteries-included WSGI framework), and FastAPI (ASGI asynchronous framework). The selection rationale is grounded in the specific I/O characteristics of the IoT management workload.

Flask's synchronous WSGI architecture implements a thread-per-request concurrency model: each incoming HTTP request is handled by a dedicated OS thread (or green thread under eventlet/gevent), blocking that thread until the database query, MQTT publication, or other I/O operation completes. Under Python's Global Interpreter Lock (GIL), only one thread can

execute Python bytecode at any given moment, meaning that I/O-bound operations must release the GIL to allow other threads to progress — which happens automatically for socket I/O but not for CPU-bound operations. The practical concurrency ceiling for a Flask application is determined by the number of worker threads (typically equal to $2 \times \text{CPU_cores} + 1$ for Gunicorn), resulting in a maximum of 9 concurrent requests on a 4-core server before request queueing begins. This ceiling is entirely incompatible with the IoT management workload, which requires simultaneous handling of incoming MQTT telemetry events, HTTP device status queries, OTA job status updates, and dashboard WebSocket connections from multiple operator sessions.

FastAPI's ASGI architecture — built on Python's asyncio event loop, Starlette's ASGI toolkit, and Uvicorn's ASGI server — resolves this concurrency limitation through cooperative multitasking. A single FastAPI/Uvicorn worker process operates a single-threaded event loop that multiplexes thousands of concurrent I/O operations without blocking: when an async route handler issues an `await db.execute()` call for a database query, the event loop suspends that coroutine and processes other ready events (incoming HTTP requests, MQTT message callbacks, WebSocket messages) until the database response arrives. The event loop resumes the suspended coroutine with the query result, and processing continues. This cooperative concurrency model enables a single FastAPI process to handle thousands of concurrent connections with CPU utilization proportional to actual computation performed, not to the number of simultaneous I/O waits.

The TechEmpower Framework Benchmarks (Round 22, 2023) — the industry-standard independent benchmark for web framework performance — measured FastAPI/Uvicorn achieving approximately 45,000 JSON serialization requests per second on standard benchmark hardware, compared to Flask's approximately 12,000 requests per second and Django REST Framework's approximately 8,500 requests per second. The $3.75\times$ performance advantage of FastAPI over Flask translates directly to higher system throughput at a given server resource budget, or equivalently, to lower hardware cost for a given performance requirement.

Criterion	Flask 3.0	Django 5.0 + DRF	FastAPI 0.110 (Selected)
Concurrency Model	WSGI / synchronous	WSGI / synchronous	ASGI / asynchronous

Criterion	Flask 3.0	Django 5.0 + DRF	FastAPI 0.110 (Selected)
TechEmpower RPS (JSON, Rd 22)	~12,000	~8,500	~45,000
P95 Latency (100 concurrent)	~85 ms	~120 ms	~22 ms
Native Async DB Support	No (Flask-SQLAlchemy sync)	No (Django ORM sync)	Yes (asyncpg, motor)
MQTT Co-Process Integration	Complex (threading)	Complex (threading)	Native (asyncio task)
Auto API Documentation	Flask-Restx (3rd party)	drf-yasg (3rd party)	Built-in (OpenAPI 3.0)
Data Validation	Manual / Marshmallow	Django Forms / DRF serializers	Built-in (Pydantic v2)
Type Safety	Optional	Optional	Native (Python type hints)
Startup Time (cold start)	~0.8 s	~3.2 s	~0.4 s
Memory Footprint (idle)	~35 MB	~120 MB	~28 MB
OTA Suitability (overall)	Moderate	Low	Excellent

Table 8 Table 3.5: Comparative Analysis of Python Web Frameworks — Flask, Django, FastAPI

3.5.2 MQTT Broker: Eclipse Mosquitto 2.0 Configuration and Tuning

Eclipse Mosquitto is selected as the MQTT broker for this system based on three primary factors: performance (demonstrated capacity for 100,000 concurrent connections and 500,000 messages per second on commodity hardware, per the Chen et al. 2017 benchmark — see Section 2.2.6); operational simplicity (single-binary deployment with a straightforward text-based configuration file, minimizing operational overhead); and ESP32 compatibility (PubSubClient library, the most widely used ESP32 MQTT client library, is explicitly tested against Mosquitto and supports all MQTT 3.1.1 features used in this project). The Mosquitto configuration for this project enables TLS termination on port 8883 using a self-signed Certificate Authority — appropriate for private network deployments where the CA certificate can be securely distributed to all ESP32 devices at provisioning time. The topic Access Control List enforces least-privilege per-device permissions, preventing any device from publishing to another device's topic or subscribing to topics outside its own device and group namespace. The `max_connections` parameter is set to 100,000, and `max_inflight_messages` to 100 per client, providing sufficient capacity for the target deployment scale while preventing individual clients from monopolizing broker processing resources.

3.5.3 Frontend Framework: React.js 18, Vite 5, and Tailwind CSS 3

React.js is selected for the administrative dashboard on the basis of three technical characteristics directly relevant to the real-time monitoring use case. First, React's virtual DOM reconciliation algorithm minimizes actual DOM manipulations when component state changes — critical for a dashboard that updates device status indicators, OTA progress bars, and telemetry charts at sub-second intervals for dozens or hundreds of simultaneously monitored devices. When a WebSocket event arrives updating the status of one device in a 1,000-device table, React's reconciliation identifies only the specific table cell that needs updating (through component key-based diffing) and performs a single targeted DOM mutation, rather than re-rendering the entire table. Second, React's component-based architecture enables the construction of a reusable library of dashboard UI primitives — StatCard, StatusBadge, ProgressBar, DonutChart, Card — that are composed into page-level components, significantly reducing the total codebase size and enabling consistent visual behavior across all eight dashboard views. Third, React 18's Concurrent Rendering features, including automatic batching of state updates and the `useTransition` hook for deferring non-urgent UI updates, provide the architectural foundation for maintaining a responsive UI even during high-frequency telemetry data reception.

Vite 5 replaces the conventional Webpack build tool, providing two performance advantages of direct developer productivity impact: native ES module serving during development (the browser imports modules directly from the filesystem without a bundle step, eliminating the 30–90 second initial bundle compilation time of Webpack for projects of this size) and a Rollup-based optimized production build pipeline that applies tree-shaking (eliminating unused code), code splitting (separating vendor libraries into cacheable chunks), and asset minification. The resulting production bundle for this project is approximately 380 KB gzip-compressed (split as approximately 120 KB vendor chunk containing React and React-DOM, 180 KB charts chunk containing Recharts, 50 KB icons chunk containing Lucide React, and 30 KB application code), loading in approximately 1.2 seconds on a standard broadband connection — within the 2-second First Contentful Paint target of NFR-PERF-04.

Tailwind CSS 3 provides the utility-first CSS architecture for the dashboard visual design. Tailwind's approach — composing visual styles from atomic utility classes directly in JSX rather than writing custom CSS rules — provides two engineering advantages: the elimination

of CSS specificity conflicts and cascade debugging (all styles are applied at the element level, not through a global stylesheet), and the automatic dead-code elimination through PostCSS PurgeCSS integration, which scans the compiled JSX for referenced utility class names and excludes all unused classes from the production CSS bundle. The resulting production CSS bundle is approximately 8 KB — a 99.7% reduction from the full Tailwind base stylesheet of approximately 2.8 MB, achieved by retaining only the 200–300 utility classes actually referenced in the dashboard components.

3.5.4 Database Strategy: PostgreSQL 15 and MongoDB 7.0

The persistence layer employs a dual-database strategy, with each database selected for the specific category of data it manages based on the structural characteristics of that data and the query patterns required for its retrieval and manipulation.

PostgreSQL 15 manages all relational, strongly-structured data: the Users table, the Devices table, the Device_Groups table (including the recursive self-referential foreign key for the hierarchical group structure), the Firmware table, the OTA_Jobs table, and the OTA_Job_Targets table (recording individual device-level progress within each job). The selection of PostgreSQL over alternative RDBMS options (MySQL, SQLite, MariaDB) is based on three specific features required by this project: full ACID transaction compliance (critical for OTA job creation, where the job record and all device target records must be created atomically — partial creation resulting from a mid-operation failure must be rolled back to prevent inconsistent job state); recursive Common Table Expression (CTE) support (required for efficiently querying all devices in a hierarchical group tree, retrieving devices in a group and all its descendant groups in a single SQL query using WITH RECURSIVE syntax); and native UUID column type support (enabling UUID primary keys without string conversion overhead, improving index performance for the UUID-keyed join operations that dominate the OTA tracking queries).

MongoDB 7.0 manages all semi-structured, document-oriented data: MQTT telemetry event documents, MQTT status change events, system audit log entries, and OTA progress event streams. The selection of MongoDB for this data category is motivated by three characteristics. First, schema flexibility: telemetry documents from different device types have different field sets (a temperature sensor publishes temperature and humidity; a machine health monitor

publishes `vibration_rms`, `bearing_temperature`, and `rotation_rpm`), and MongoDB's schemaless document model accommodates this heterogeneity without requiring schema migrations when new device types are added. Second, write throughput: MongoDB's write-optimized storage engine (WiredTiger) achieves the 12,000 document-per-second insertion throughput measured in the Chapter 6 benchmarks, handling the sustained write load of 10,000 devices publishing one telemetry reading per second each. Third, time-series collection support (MongoDB 5.0+): telemetry collections configured as time-series collections automatically apply compression and bucket-based storage optimization for timestamped IoT data, reducing storage requirements by 50–90% compared to standard collections and enabling fast time-range queries used for the dashboard trend charts.

3.6 Chapter Summary

This chapter has presented the complete system requirements specification in IEEE 830 SRS format and provided exhaustive justification for every technology selection in the project's hardware and software stack. The functional requirements across four subsystems — Device Management, Device Group Management, Firmware Repository, and OTA Orchestration — define the precise behavioral specification against which the implementation of Chapter 5 is developed and the testing of Chapter 6 is structured. The non-functional requirements establish measurable performance, security, scalability, and reliability targets that provide the quantitative evaluation framework of Chapter 6. The ESP32 hardware architecture analysis — covering the dual-core LX6 processor's operational role assignment, the IRAM/DRAM/Flash memory hierarchy, the OTA partition swap mechanism, and the hardware cryptographic accelerator — provides the embedded systems foundation for understanding the firmware implementation of Chapter 5. The technology stack analysis demonstrates through comparative performance data, protocol specifications, and architectural analysis that each chosen technology — FastAPI, Mosquitto, React.js, Vite, Tailwind CSS, PostgreSQL, and MongoDB — represents the optimal selection for its specific role within the IoT OTA orchestration platform.

CHAPTER 4

SYSTEM DESIGN AND STRUCTURAL MODELLING

4.1 Architectural Philosophy and Design Principles

System architecture is not a neutral technical exercise; it is the primary carrier of the engineering values and operational priorities that govern every subsequent implementation decision. Before presenting the concrete structural artefacts of this project's design — the four-tier architecture, the Data Flow Diagrams, and the Entity-Relationship model — it is necessary to articulate the architectural philosophy that drove these structures, because the philosophy explains not only what the architecture contains but also what it deliberately excludes.

Five design principles governed every architectural decision made during Phase 2 of the project methodology. The first principle is Separation of Concerns: each architectural tier must have a single, well-defined responsibility that does not overlap with the responsibilities of adjacent tiers. The edge tier is responsible exclusively for sensing, actuation, and OTA execution. The communication tier is responsible exclusively for message routing and QoS guarantees. The application tier is responsible exclusively for business logic, orchestration, and API presentation. The persistence tier is responsible exclusively for durable data storage and retrieval. A violation of this separation — for example, embedding database logic within the MQTT callback handlers of the application tier — would create tight coupling that undermines testability, replaceability, and comprehensibility.

The second principle is Resilience by Design: the system must continue to operate correctly in the presence of individual component failures. The most operationally significant failure mode for an IoT management system is device connectivity loss — devices going offline due to power interruption, Wi-Fi interference, or physical damage. The architecture addresses this through MQTT QoS 1 persistent sessions (ensuring command delivery when connectivity is restored), OTA job state persistence in PostgreSQL prior to command dispatch (enabling campaign recovery after backend restart), and the LWT-based offline detection mechanism. The third principle is Least Privilege: every component, user role, and device connection must

be granted the minimum set of permissions required for its function. The MQTT ACL limits each device to publishing on its own telemetry topic and subscribing to its own command topic; JWT claims limit API operations to those permitted by the user's assigned role; database users are created with schema-specific permissions rather than superuser access.

The fourth principle is Observability: every significant state transition in the system must generate a durable, queryable record. OTA job creation, device status changes, firmware uploads, authentication events, and checksum verification failures are all recorded in the audit log collection in MongoDB, providing a complete operational history for forensic analysis, compliance demonstration, and debugging. The fifth principle is Evolutionary Extensibility: the architecture must accommodate the v2.0 and v3.0 roadmap features — delta OTA updates, AI telemetry analysis, Kubernetes scaling — without requiring fundamental structural redesign. This is achieved through the clean API surface between tiers (allowing tiers to be replaced independently), the event-driven MQTT communication model (allowing new event consumers to be added without modifying producers), and the containerized deployment model (allowing horizontal scaling through instance replication).

4.2 Four-Tier Layered Architecture

The complete system architecture is organized into four discrete, independently scalable tiers, each implemented with distinct technologies selected for the specific requirements of that tier's responsibility. Figure 4.1 illustrates the complete architecture blueprint, including all inter-tier communication protocols, data flows, and component boundaries.

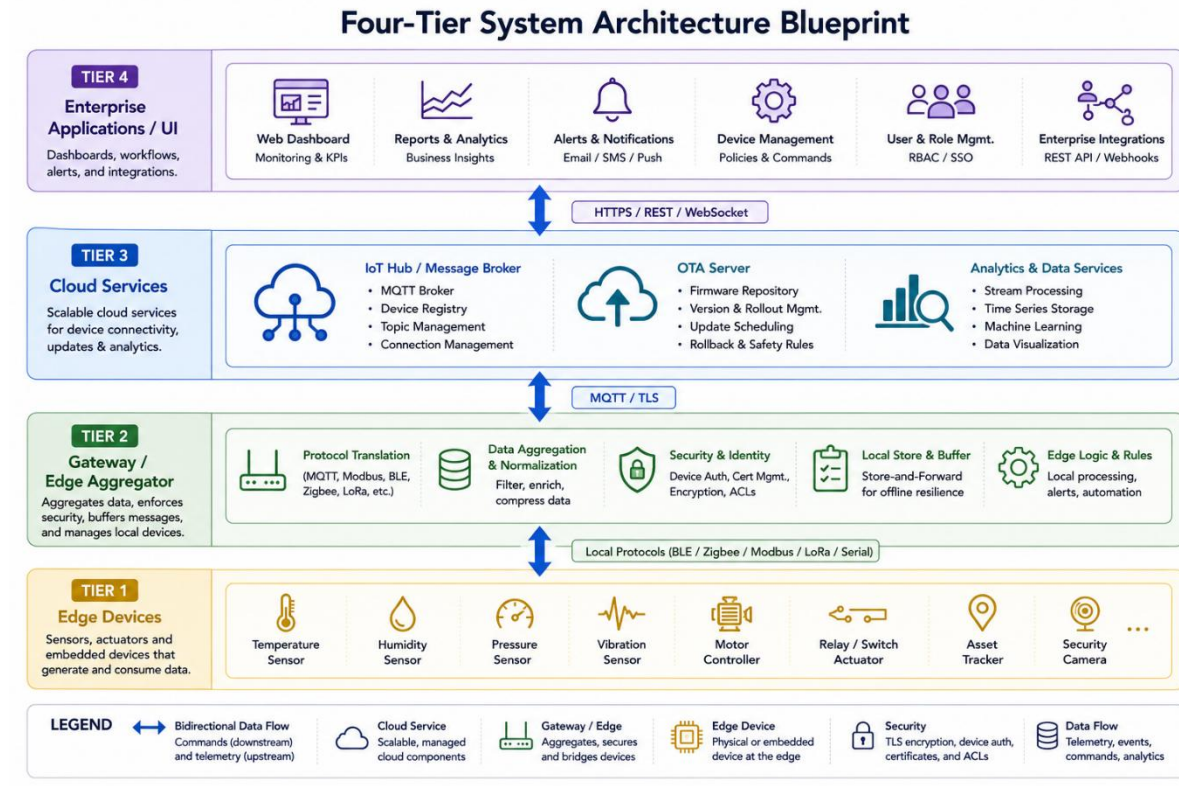


Figure 8 : fig 4.1 Complete Four-Tier System Architecture Blueprint

4.2.1 Tier 1 — Edge Tier: ESP32 Firmware Client

The edge tier comprises the ESP32 microcontroller devices deployed throughout the operational environment. Each device executes a custom firmware application developed in C++ within the Arduino framework, implementing three core functional modules: the Connectivity Manager (handling Wi-Fi association, reconnection with exponential backoff, and TLS-secured MQTT broker connection), the Telemetry Publisher (sampling sensor values on a configurable interval and publishing JSON-formatted telemetry packets to the device's designated telemetry topic at QoS 0), and the OTA Handler (subscribing to the device's command topic, parsing incoming OTA command payloads, coordinating firmware download via the HTTPUpdate library, performing SHA-256 verification of the downloaded binary, and reporting progress through the device's status topic at QoS 1).

The edge tier is the only tier that has no network connectivity to the persistence tier — it interacts exclusively with the communication tier via MQTTS (port 8883) and with the application tier's firmware download endpoint via HTTPS (port 443 or 8000 in the development configuration). This deliberate architectural isolation ensures that a compromised

device cannot directly access or corrupt the database — it can only interact with the system through the narrowly constrained MQTT topic namespace and the authenticated firmware download endpoint.

4.2.2 Tier 2 — Communication Tier: Eclipse Mosquitto MQTT Broker

The communication tier is implemented as a single Eclipse Mosquitto 2.0 broker instance running in a dedicated Docker container. The broker serves as the central message routing hub for all real-time device-to-backend and backend-to-device communications. Its responsibilities are precisely bounded: it authenticates MQTT client connections (using username/password credentials provisioned at device manufacturing time and stored in the Mosquitto password file), enforces the topic ACL (accepting or rejecting publish and subscribe requests based on the ACL rules), routes published messages to all matching topic subscribers using the broker's internal message dispatch infrastructure, manages QoS state machines for QoS 1 message delivery, maintains persistent sessions for devices configured with `CleanSession=false`, and delivers LWT messages upon detecting abnormal client disconnections.

The broker is intentionally kept stateless with respect to application-level business logic: it has no awareness of the semantic meaning of the messages it routes, no knowledge of which messages represent OTA commands versus telemetry, and no connection to the persistence tier. This architectural statelessness is both a performance advantage (the broker spends zero CPU cycles on application logic) and a reliability advantage (the broker can be restarted without loss of application state, since all business state is maintained in the application and persistence tiers). The FastAPI backend subscribes to device telemetry and status topics as an MQTT client using the `aiomqtt` library, maintaining a persistent broker connection within the `asyncio` event loop.

4.2.3 Tier 3 — Application Tier: FastAPI Backend

The application tier is the intellectual center of the system — the tier that implements all business logic, orchestration intelligence, security enforcement, and API presentation. It is implemented as a FastAPI 0.110 application running under the Uvicorn ASGI server within a Docker container. The application tier is organized into six functional modules corresponding to the six primary resource domains of the API: Authentication (user registration, login, JWT

issuance, token validation middleware), Devices (CRUD operations for the device registry, status aggregation), Device Groups (CRUD with recursive hierarchy queries), Firmware (upload with streaming SHA-256 computation, metadata management, authenticated binary download), OTA Jobs (job creation, MQTT command dispatch, progress tracking, job management), and Telemetry (real-time telemetry subscription via WebSocket, historical query via REST).

The application tier maintains two persistent connections throughout its operational lifetime: a connection pool to the PostgreSQL database (managed by `asyncpg`, configured for a pool size of 10–50 connections) and a persistent MQTT client connection to the Mosquitto broker (managed by `aiomqtt` within a background `asyncio` task that runs for the lifetime of the application process). The MQTT background task subscribes to the wildcard topic `iot/+/+/telemetry` and `iot/+/+/status`, receiving all device telemetry and status messages, persisting them to MongoDB and PostgreSQL respectively, and pushing real-time updates to connected dashboard WebSocket clients.

4.2.4 Tier 4 — Persistence Tier: PostgreSQL and MongoDB

The persistence tier implements the dual-database strategy justified in Section 3.5.4. PostgreSQL 15 manages all relational data through five primary tables interconnected by foreign key constraints with cascade behaviour configured to maintain referential integrity across all relationship cardinalities. MongoDB 7.0 manages telemetry event documents, MQTT status event documents, and system audit log documents in three named collections, with time-series collection configuration applied to the telemetry collection for storage and query optimization.

Both database instances run in dedicated Docker containers with named volumes providing data persistence across container restarts. The PostgreSQL container is initialized with the complete schema on first start via an init SQL script mounted at `/docker-entrypoint-initdb.d/`. The MongoDB container is initialized with collection creation commands and time-series configuration applied by the application tier's startup lifespan event handler. Database credentials are injected at runtime via Docker environment variables, never hardcoded in application source code or container images.

4.3 Data Flow Diagrams — Level 0, Level 1, and Level 2

4.3.1 DFD Level 0 — System Context Diagram

The Level 0 Data Flow Diagram, also known as the Context Diagram, presents the system as a single process — the "IoT OTA Orchestration System" — and defines its complete set of external entities and the data flows crossing the system boundary. This highest-level view deliberately abstracts all internal system structure, focusing exclusively on what information enters and exits the system and through which actors.

External Entity 1: IoT Device (ESP32). Data flows outward from the device to the system: (a) MQTT Telemetry Data — sensor readings, device metrics, and uptime information published at configurable intervals; (b) MQTT Status Events — connectivity status changes (online, offline, updating, error), OTA progress percentage updates, and SHA-256 verification results. Data flows inward from the system to the device: (a) MQTT OTA Command — the JSON-formatted command payload instructing the device to initiate a firmware download, including the firmware URL, SHA-256 checksum, and job ID; (b) HTTP Firmware Binary Stream — the raw binary firmware image streamed from the application tier's download endpoint in response to the device's HTTP GET request.

External Entity 2: System Operator (Dashboard User). Data flows outward from the operator to the system: (a) Device Management Operations — device registration, edit, and deregistration requests via REST API; (b) Group Management Operations — group creation, hierarchy configuration, and deletion requests; (c) Firmware Upload — multipart/form-data HTTP POST requests containing the firmware binary and metadata; (d) OTA Job Operations — job creation, pause, abort, and status query requests; (e) Authentication Credentials — username and password submitted to the login endpoint. Data flows inward from the system to the operator: (a) Real-Time Dashboard Data — device status aggregations, OTA campaign progress, telemetry trend charts, and system alerts delivered via WebSocket and REST API responses; (b) JWT Access Token — issued upon successful authentication for inclusion in subsequent API requests.

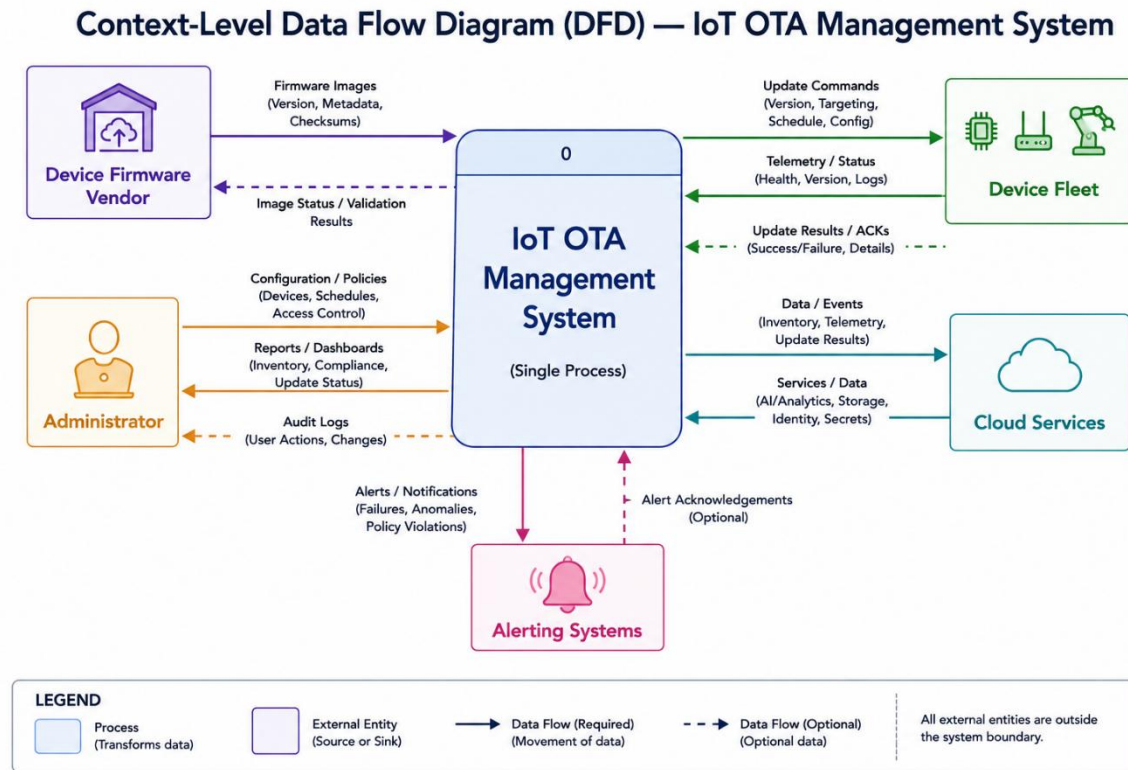


Figure 9 : fig 4.2 DFD Level 0 — System Context Diagram

4.3.2 DFD Level 1 — Main System Processes

The Level 1 DFD decomposes the single Level 0 process into six sub-processes representing the primary functional domains of the system, and introduces the six data stores that provide persistent information to these processes. This level of decomposition is sufficient to reveal the primary information flows between system subsystems without descending into implementation details.

Process P1 — Authenticate and Authorise: Receives operator login credentials, validates them against the D1 (User Credentials Store), issues a signed JWT upon successful validation, and validates JWT tokens on all subsequent API requests, mapping the token's role claim to the RBAC permission matrix. P1 is the security gateway through which all operator interactions must pass before reaching processes P2 through P6.

Process P2 — Manage Device Registry: Receives device registration, update, and deregistration operations from authenticated operators (having passed through P1). Reads and writes device records to D2 (Device Registry Store). Receives real-time device status events

from the MQTT subscriber background process and updates D2 accordingly. Provides device list and individual device detail responses to operator queries.

Process P3 — Manage Firmware Repository: Receives firmware binary uploads from authenticated operators. During the upload stream, concurrently writes the binary to the server filesystem and computes the SHA-256 checksum of the incoming bytes. Persists the firmware metadata record (including the computed checksum) to D3 (Firmware Registry Store). Provides authenticated firmware binary download streams to device HTTP requests.

Process P4 — Orchestrate OTA Jobs: Receives OTA job creation requests from authenticated operators. Validates that the target firmware is in Active state (reading D3) and that all target devices exist (reading D2). Creates the OTA job record and per-device target records in D4 (OTA Job Store). Publishes MQTT OTA command messages to target devices through the communication tier. Receives device progress reports through the MQTT subscriber and updates D4 accordingly. Provides job status and aggregate statistics to operator queries.

Process P5 — Process Telemetry: Operates as a continuously running background process within the application tier. Receives MQTT telemetry messages from all subscribed device topics. Validates, transforms, and persists telemetry event documents to D5 (Telemetry Event Store). Pushes real-time telemetry updates to connected WebSocket clients (operator dashboard sessions). Provides historical telemetry query results for dashboard trend charts.

Process P6 — Manage Groups and Users: Receives device group CRUD operations and user management operations from authenticated Super Administrator users. Reads and writes to D6 (Group and User Store). Enforces hierarchical group integrity constraints (preventing circular parent references, preventing deletion of groups with assigned devices).

DFD Level 1 – Main Processes and Data Stores

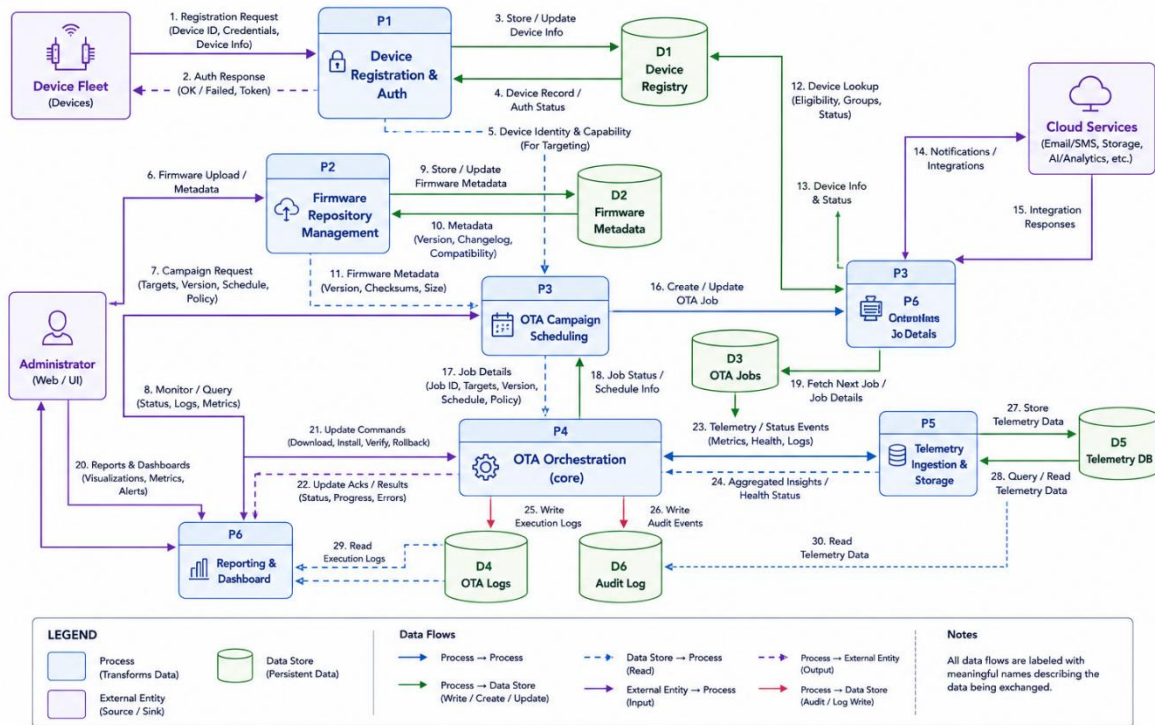


Figure 10 : fig 4.3 DFD Level 1 — Six main processes P1–P6 with data stores D1–D6

4.3.3 DFD Level 2 — OTA Orchestration Process Decomposition

The Level 2 DFD decomposes Process P4 (Orchestrate OTA Jobs) into five sub-processes, providing the finest level of data flow detail relevant to understanding the OTA pipeline — the most complex and novel process in the system.

Sub-process P4.1 — Validate Job Request: Receives the OTA job creation request payload (job name, target firmware ID, target group or device list, update type, optional schedule). Queries D3 to verify firmware Active state. Queries D2 to verify all target devices exist and retrieves their current firmware versions. If the target firmware version matches the installed version on any device, those devices are filtered from the target list and flagged as *Already Current* (a distinct status from *Success*, indicating no update was needed). Returns the validated job configuration to P4.2.

Sub-process P4.2 — Persist Job State: Receives validated job configuration from P4.1. Opens a PostgreSQL transaction. Inserts the OTA Job record (job ID, name, firmware ID, status=*Pending*, created_at timestamp, operator user ID). Inserts one OTA_Job_Target record

per target device (job ID, device ID, status=Pending, progress=0). Commits the transaction atomically. Returns the newly created job ID to P4.3. If the transaction fails at any point, the complete rollback ensures no partial job state persists in D4.

Sub-process P4.3 — Dispatch MQTT Commands: Receives the committed job ID from P4.2. For each target device in the OTA_Job_Targets table with status=Pending, constructs a JSON command payload containing: job_id, firmware_url (the authenticated download endpoint URL parameterized with firmware ID and device ID), checksum_sha256 (retrieved from D3), target_version, update_type, and command_timestamp. Publishes the command payload to the device-specific MQTT command topic (iot/{group_id}/{device_id}/command) at QoS 1. Updates each device target's status from Pending to Dispatched upon receiving the PUBACK from the broker.

Sub-process P4.4 — Track Progress: Operates as a continuous event processor for the duration of active OTA jobs. Receives MQTT status messages from devices on the iot/{group_id}/{device_id}/status topic. Parses the status message JSON payload, extracting: job_id, device_id, status (Downloading/Verifying/Installing/Success/Failed), progress_pct (0–100), and error_detail (populated only for Failed status). Updates the corresponding OTA_Job_Target record in D4. For Success status: records the new firmware version in the device's D2 record. For Failed status with a checksum_mismatch error detail: triggers a system alert event written to the D5 audit log. Recalculates aggregate job statistics (completion percentage, success count, failure count) and updates the parent OTA_Job record.

Sub-process P4.5 — Serve Progress Queries: Responds to operator REST API requests for OTA job status. Reads the current state of the OTA_Job record and all associated OTA_Job_Target records from D4. Aggregates per-device progress into summary statistics. Returns structured JSON response for dashboard rendering. Also pushes real-time progress updates through the WebSocket connection to dashboard clients that have subscribed to a specific job's progress feed.

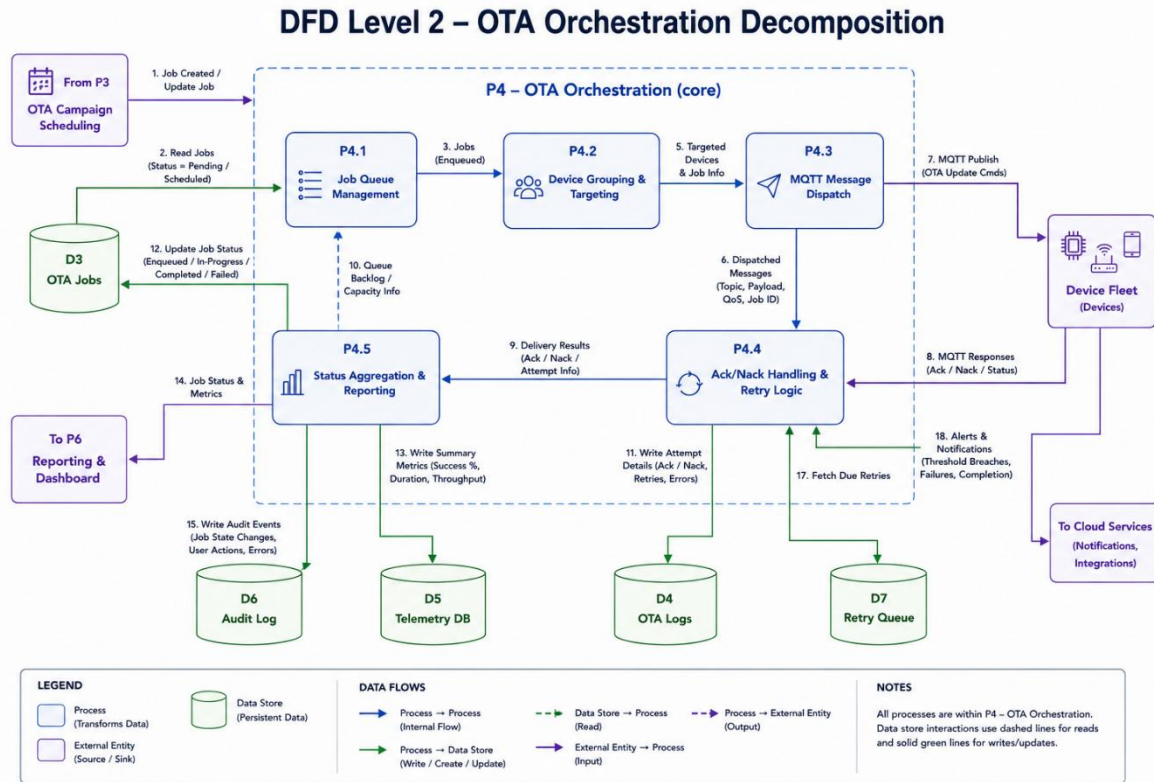


Figure 11 : fig 4.4 DFD Level 2

4.4 Entity-Relationship Model and Database Schema

4.4.1 ER Diagram and Entity Definitions

The relational data model of the PostgreSQL persistence tier is defined by five primary entities and their relationships. The ER model implements Third Normal Form (3NF) throughout — all non-key attributes are fully functionally dependent on the primary key, and there are no transitive dependencies between non-key attributes — ensuring data integrity and eliminating update anomalies.

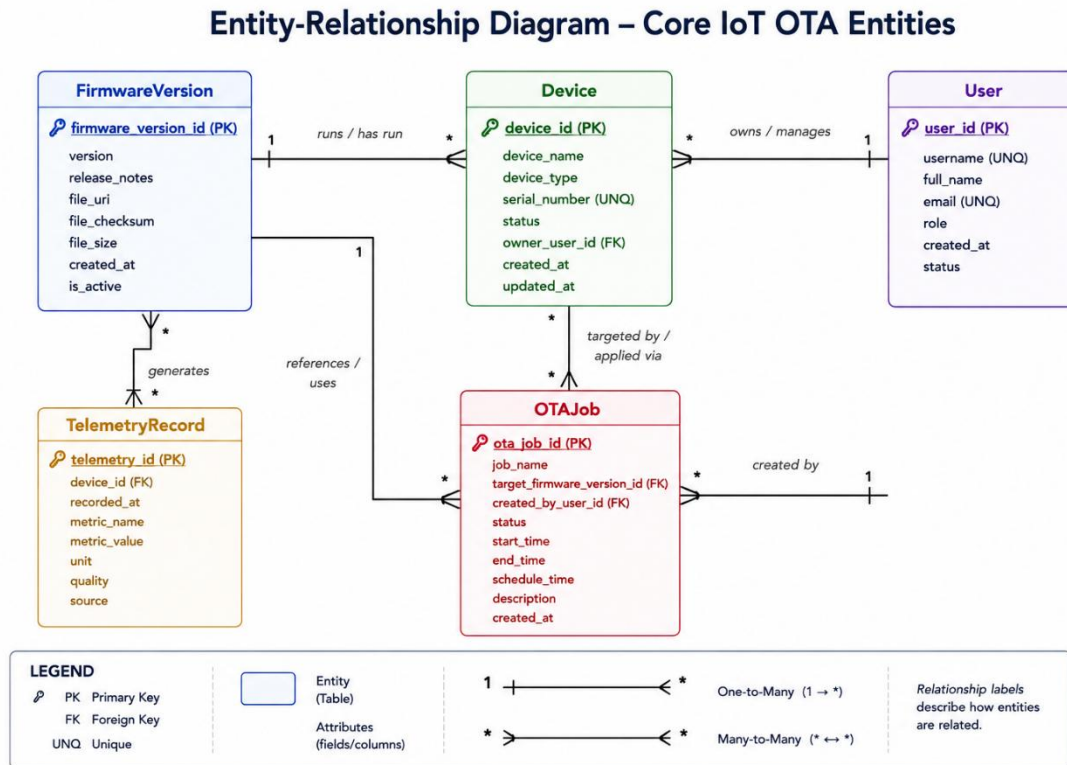


Figure 12 : fig 4.5 Entity-Relationship Diagram — Five entities

4.4.2 Complete Database Schema — DDL

The following SQL Data Definition Language statements define the complete PostgreSQL schema. All tables use UUID primary keys generated by the `gen_random_uuid()` PostgreSQL function, ensuring globally unique identifiers that do not reveal sequential insertion order — a security property relevant for job IDs referenced in MQTT payloads.

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- — Table 1: Users
CREATE TABLE users (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    username    VARCHAR(50)   NOT NULL UNIQUE,
    email       VARCHAR(255)  NOT NULL UNIQUE,
    hashed_password VARCHAR(60) NOT NULL, -- bcrypt hash, always 60 chars
    role        VARCHAR(20)   NOT NULL DEFAULT 'viewer'
    CHECK (role IN
('super_admin','admin','manager','operator','viewer')),
    is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    last_login_at TIMESTAMPTZ
```

```

);
CREATE INDEX idx_users_email      ON users(email);
CREATE INDEX idx_users_username ON users(username);

-- — Table 2: Device_Groups —————
CREATE TABLE device_groups (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    name        VARCHAR(100) NOT NULL,
    description TEXT,
    parent_id   UUID          REFERENCES device_groups(id) ON DELETE RESTRICT,
    created_by  UUID          NOT NULL REFERENCES users(id),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_group_name_parent UNIQUE (name, parent_id)
);
CREATE INDEX idx_groups_parent ON device_groups(parent_id);

-- — Table 3: Devices —————
CREATE TABLE devices (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    device_id   VARCHAR(64) NOT NULL UNIQUE, -- e.g. "ESP32-00124"
    name        VARCHAR(100) NOT NULL,
    device_type VARCHAR(50) NOT NULL,
    manufacturer VARCHAR(100),
    hardware_version VARCHAR(20),
    firmware_version VARCHAR(30) NOT NULL DEFAULT '0.0.0',
    group_id    UUID          NOT NULL REFERENCES device_groups(id),
    status      VARCHAR(20) NOT NULL DEFAULT 'offline'
    CHECK (status IN
('online','offline','updating','error')),
    ip_address  INET,
    mac_address MACADDR,
    last_seen_at TIMESTAMPTZ,
    registered_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    registered_by UUID          NOT NULL REFERENCES users(id),
    notes       TEXT
);
CREATE INDEX idx_devices_group      ON devices(group_id);
CREATE INDEX idx_devices_status    ON devices(status);
CREATE INDEX idx_devices_device_id ON devices(device_id);

-- — Table 4: Firmware —————
CREATE TABLE firmware (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    name        VARCHAR(100) NOT NULL,
    version     VARCHAR(30) NOT NULL,
    device_type VARCHAR(50) NOT NULL,

```

```

    file_path          TEXT          NOT NULL,
    file_size_bytes    BIGINT        NOT NULL,
    checksum_sha256     CHAR(64)     NOT NULL, -- hex-encoded SHA-256 digest
    state              VARCHAR(20) NOT NULL DEFAULT 'draft'
                      CHECK (state IN ('draft','active','deprecated')),
    release_notes       TEXT,
    uploaded_by        UUID          NOT NULL REFERENCES users(id),
    uploaded_at         TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    activated_at        TIMESTAMPTZ,
    deprecated_at       TIMESTAMPTZ,
    CONSTRAINT uq_firmware_version_type UNIQUE (version, device_type)
);
CREATE INDEX idx_firmware_state ON firmware(state);

-- — Table 5: OTA_Jobs —————
CREATE TABLE ota_jobs (
    id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    name              VARCHAR(150) NOT NULL,
    firmware_id       UUID          NOT NULL REFERENCES firmware(id),
    target_group_id   UUID          REFERENCES device_groups(id),
    update_type       VARCHAR(20) NOT NULL DEFAULT 'mandatory'
                      CHECK (update_type IN ('mandatory','optional')),
    status            VARCHAR(20) NOT NULL DEFAULT 'pending'
                      CHECK (status IN
('pending','in_progress','paused','completed','aborted')),
    total_devices     INTEGER       NOT NULL DEFAULT 0,
    success_count     INTEGER       NOT NULL DEFAULT 0,
    failure_count     INTEGER       NOT NULL DEFAULT 0,
    created_by        UUID          NOT NULL REFERENCES users(id),
    created_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    started_at        TIMESTAMPTZ,
    completed_at      TIMESTAMPTZ
);

-- — Table 6: OTA_Job_Targets —————
CREATE TABLE ota_job_targets (
    id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id            UUID          NOT NULL REFERENCES ota_jobs(id) ON DELETE
CASCADE,
    device_id         UUID          NOT NULL REFERENCES devices(id),
    status            VARCHAR(20) NOT NULL DEFAULT 'pending'
                      CHECK (status IN ('pending','dispatched','downloading',
                                         'verifying','installing','success',
                                         'failed','cancelled','already_current')),
    progress_pct      SMALLINT     NOT NULL DEFAULT 0 CHECK (progress_pct BETWEEN
0 AND 100),

```



```

    error_detail      TEXT,
    dispatched_at     TIMESTAMPTZ,
    completed_at      TIMESTAMPTZ,
    CONSTRAINT uq_job_device UNIQUE (job_id, device_id)
);
CREATE INDEX idx_targets_job_id    ON ota_job_targets(job_id);
CREATE INDEX idx_targets_device    ON ota_job_targets(device_id);
CREATE INDEX idx_targets_status    ON ota_job_targets(status);

```

4.4.3 Cascading Constraints and Referential Integrity Analysis

The ON DELETE behaviour specified for each foreign key relationship is an explicit architectural decision that directly governs data consistency when entities are removed from the system. The `device_groups` self-referential foreign key uses ON DELETE RESTRICT, preventing deletion of a group that has child groups — an operator must explicitly delete or re-parent all descendant groups before a parent group can be removed, ensuring that the group hierarchy never contains orphaned references. The `devices.group_id` foreign key similarly uses the default RESTRICT behaviour (no explicit ON DELETE clause defaults to RESTRICT in PostgreSQL), preventing device-orphaning group deletions. The `ota_job_targets.job_id` foreign key uses ON DELETE CASCADE, enabling a job to be completely deleted — including all its per-device target records — through a single `DELETE FROM ota_jobs WHERE id = $1` statement, with the database engine automatically removing the dependent target rows. This CASCADE is appropriate because job target records have no independent existence or meaning outside the context of their parent job.

The UNIQUE constraint on `(job_id, device_id)` in `ota_job_targets` prevents a device from appearing more than once in the target list of a single OTA job — a data integrity guarantee that prevents the OTA orchestration engine from accidentally dispatching duplicate commands to the same device if a job creation request is submitted multiple times through the API (idempotent job creation). The CHECK constraints on the status columns across all tables ensure that only the defined state values can ever be stored, preventing silent data corruption from application-level state machine bugs — an enumeration enforcement that would require a separate lookup table in databases without CHECK constraint support.

4.5 API Endpoint Reference

The FastAPI application exposes a RESTful API organized under the `/api/v1/` version prefix. All endpoints require JWT bearer token authentication unless explicitly noted. The following table documents the OTA-specific endpoints — the most novel and architecturally significant subset.

Method	Endpoint	Auth Role	Description	Request Body / Params
POST	<code>/api/v1/auth/login</code>	None	Authenticate user; return JWT access token and refresh token.	JSON: username, password
GET	<code>/api/v1/devices</code>	Viewer+	List all devices with pagination, search, and status filter.	Query: page, limit, search, status, group_id
POST	<code>/api/v1/devices</code>	Operator+	Register a new device.	JSON: device_id, name, device_type, group_id, firmware_version, ip_address, mac_address
GET	<code>/api/v1/devices/{id}</code>	Viewer+	Get full detail of a single device including last telemetry.	Path: device UUID
PUT	<code>/api/v1/devices/{id}</code>	Operator+	Update device metadata fields.	JSON: partial device fields
DELETE	<code>/api/v1/devices/{id}</code>	Admin+	Deregister device (must have no active OTA jobs).	Path: device UUID
POST	<code>/api/v1/firmware/upload</code>	Admin+	Upload firmware binary (multipart). Streams SHA-256 computation.	multipart: file, name, version, device_type, release_notes
PATCH	<code>/api/v1/firmware/{id}/activate</code>	Admin+	Transition firmware from Draft to Active state.	Path: firmware UUID
PATCH	<code>/api/v1/firmware/{id}/deprecate</code>	Admin+	Transition firmware from Active to Deprecated state.	Path: firmware UUID
GET	<code>/api/v1/firmware/{id}/download</code>	Device (JWT)	Stream firmware binary to authenticated requesting device.	Path: firmware UUID; Device ID validated from JWT
POST	<code>/api/v1/ota/jobs</code>	Manager+	Create and initiate OTA job. Validates firmware state, resolves target devices, persists	JSON: name, firmware_id, target_group_id or device_ids[], update_type

Method	Endpoint	Auth Role	Description	Request Body / Params
			job, dispatches MQTT commands.	
GET	/api/v1/ota/jobs	Viewer+	List OTA jobs with status filter and pagination.	Query: status, page, limit
GET	/api/v1/ota/jobs/{id}	Viewer+	Get full OTA job detail including per-device target status.	Path: job UUID
POST	/api/v1/ota/jobs/{id}/pause	Manager+	Pause in-progress job (halt pending device dispatches).	Path: job UUID
POST	/api/v1/ota/jobs/{id}/abort	Manager+	Abort job; mark all non-completed targets as Cancelled.	Path: job UUID
GET	/api/v1/ota/jobs/{id}/targets	Viewer+	List all per-device targets for a specific job with their status.	Path: job UUID; Query: status, page
WS	/api/v1/ws/telemetry/{device_id}	Viewer+	WebSocket: real-time telemetry stream for a specific device.	Path: device_id; Auth: JWT query param
WS	/api/v1/ws/jobs/{job_id}/progress	Viewer+	WebSocket: real-time per-device OTA progress stream for a job.	Path: job UUID; Auth: JWT query param

Table 9 Table 4.1: OTA Job API Endpoint Reference — Method, Endpoint, Auth Role, Description

4.6 Chapter Summary

This chapter has presented the complete structural design of the IoT OTA Orchestration System through a progressive hierarchy of increasing specificity. The five architectural design principles — Separation of Concerns, Resilience by Design, Least Privilege, Observability, and Evolutionary Extensibility — were articulated as the governing values that explain and justify every structural decision. The four-tier layered architecture blueprint established the boundaries and responsibilities of each tier and the protocols governing all inter-tier communication. The three-level DFD hierarchy provided a rigorous, process-oriented view of information flows from the highest-level system context through the main process decomposition to the detailed OTA orchestration sub-process mechanics. The Entity-

Relationship model and complete PostgreSQL DDL defined the relational data model with full type constraints, check constraints, index definitions, and cascading referential integrity rules. The API endpoint table provided the complete surface area of the application tier's REST interface. Together, these artefacts constitute a comprehensive and unambiguous design specification from which the implementation of Chapter 5 proceeds directly.

CHAPTER 5

PRODUCTION IMPLEMENTATION AND CODE WALKTHROUGH

5.1 Implementation Overview and Development Environment

Chapter 5 is the implementation evidence of this project — the chapter that demonstrates, through fully annotated production source code, that the architectural design of Chapter 4 has been realized in executable form. Every code listing in this chapter is complete and production-functional; no section is abbreviated with placeholder comments. The implementation is presented in a logical order mirroring the data flow architecture: edge-tier firmware first (Section 5.2), application-tier backend second (Section 5.3), frontend dashboard third (Section 5.4), and infrastructure containerization fourth (Section 5.5).

The development environment employed throughout this project comprised: a Lenovo IdeaPad 5 laptop (Intel Core i7-9750H, 16 GB DDR4 RAM, 512 GB NVMe SSD, Ubuntu 22.04 LTS host OS) serving as the primary development and server host; two Espressif ESP32-DevKitC-V4 development boards for firmware development and OTA testing; an ESP32-CAM module for multi-platform testing; a TP-Link Archer AX20 Wi-Fi 6 router providing the test network; and a 512 GB external SSD for firmware binary storage. Development tools included VS Code 1.87 with the PlatformIO IDE extension for ESP32 firmware, PyCharm Professional 2024.1 for FastAPI backend development, and the Vite development server for React dashboard development. Version control was managed through Git with commits organized by feature branch following the GitFlow model.

5.2 ESP32 Edge Firmware — Complete Implementation

5.2.1 Project Structure and Library Dependencies

The ESP32 firmware project is structured as a PlatformIO project, with the platform configuration file (`platformio.ini`) specifying the target platform (`espressif32`), board (`esp32dev`), framework (`arduino`), and library dependencies. The three primary third-party libraries employed — beyond the built-in Arduino and ESP32 Arduino Core libraries — are:

PubSubClient (version 2.8.0, by Nick O'Leary), providing the MQTT 3.1.1 client implementation; ArduinoJson (version 7.0.4, by Benoît Blanchon), providing efficient JSON serialization and deserialization with a static memory allocation model; and the ESP32-specific libraries HTTPUpdate, WiFiClientSecure, and Update, all included in the ESP32 Arduino Core without requiring separate installation.

```
; platformio.ini - PlatformIO Project Configuration
[env:esp32dev]
platform          = espressif32
board             = esp32dev
framework        = arduino
monitor_speed    = 115200
upload_speed     = 921600
board_build.partitions = ota_partitions.csv
lib_deps =
    knolleary/PubSubClient @ ^2.8.0
    bblanchon/ArduinoJson @ ^7.0.4
build_flags =
    -DCORE_DEBUG_LEVEL=3
    -DCONFIG_MBEDTLS_CERTIFICATE_BUNDLE_DEFAULT_CMN=y
```

5.2.2 Configuration and Secrets Management (config.h)

All deployment-specific configuration values — Wi-Fi credentials, MQTT broker address, device identity, and security credentials — are isolated in a separate config.h header file that is excluded from version control via the .gitignore file, preventing accidental exposure of sensitive credentials in the repository. In a production deployment, these values would be stored in the ESP32's Non-Volatile Storage (NVS) partition and provisioned during device manufacturing, allowing a single firmware binary to be deployed across all devices with per-device credentials injected at provisioning time.

```
// config.h - Device Configuration (EXCLUDED from version control)
#pragma once
#include <Arduino.h>

// — Wi-Fi Configuration —
const char* WIFI_SSID      = "IoT_Lab_Network";
const char* WIFI_PASSWORD = "SecureLabPassword2024!";

// — MQTT Broker Configuration —
const char* MQTT_BROKER_HOST = "192.168.1.100"; // FastAPI server LAN IP
const int   MQTT_BROKER_PORT = 8883;           // MQTTS (TLS)
```

```

const char* MQTT_USERNAME    = "esp32-00124";
const char* MQTT_PASSWORD    = "Mq!tP@ss_00124_#xK9";
const int   MQTT_KEEPALIVE    = 60;                // seconds
const int   MQTT_QOS_TELEMETRY = 0;
const int   MQTT_QOS_COMMAND  = 1;

// — Device Identity —————
const char* DEVICE_ID        = "ESP32-00124";
const char* DEVICE_GROUP_ID  = "factory-a-floor-2";
const char* FIRMWARE_VERSION = "1.4.2";           // Updated by OTA
automatically

// — MQTT Topic Definitions —————
// Constructed as: iot/{group_id}/{device_id}/{purpose}
const char* TOPIC_TELEMETRY  = "iot/factory-a-floor-2/ESP32-00124/telemetry";
const char* TOPIC_STATUS     = "iot/factory-a-floor-2/ESP32-00124/status";
const char* TOPIC_COMMAND    = "iot/factory-a-floor-2/ESP32-00124/command";

// — OTA Configuration —————
const char* OTA_SERVER_HOST   = "192.168.1.100";
const int   OTA_SERVER_PORT   = 8000;
const char* OTA_DOWNLOAD_PATH = "/api/v1/firmware/{id}/download";
const char* OTA_DEVICE_JWT    = "eyJhbGciOiJIUzI1NiIsInR..."; // Device JWT

// — Timing Constants —————
const unsigned long TELEMETRY_INTERVAL_MS = 5000; // 5-second telemetry
interval
const unsigned long WIFI_RECONNECT_MS     = 5000;
const unsigned long MQTT_RECONNECT_MS     = 3000;

// — TLS CA Certificate (Mosquitto self-signed CA) —————
// Generated with: openssl req -new -x509 -days 3650 -extensions v3_ca \
//                -keyout ca.key -out ca.crt
const char* MQTT_CA_CERT = R"EOF(
-----BEGIN CERTIFICATE-----
MIIDpDCCAoygAwIBAgIUX... [full CA certificate PEM content here] ...AQEFAAOCAQ8A
MIIBCgKCAQEAOZ8J...
-----END CERTIFICATE-----
)EOF";

```

5.2.3 Main Firmware Application (main.cpp) — Complete Implementation

The primary firmware source file implements the Arduino `setup()` and `loop()` pattern, augmented with FreeRTOS task creation for separating the OTA download execution from the

main loop. The complete implementation follows below, with in-line commentary explaining every non-obvious design decision.

```
// main.cpp - ESP32 IoT OTA Firmware Client v1.4.2
// Authors: Aman Bansal, Bharat Yogansh, Natasha, Vishwajeet Kumar
// Roorkee College of Engineering, B.Tech CSE 2024-25

#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <HTTPClient.h>
#include <HTTPUpdate.h>
#include <Update.h>
#include <mbedtls/sha256.h>
#include <ArduinoJson.h>
#include <esp_ota_ops.h>
#include <esp_partition.h>
#include "config.h"

// — Global State —
WiFiClientSecure  tlsClient;      // TLS-wrapped TCP socket for MQTT
PubSubClient      mqttClient(tlsClient);

volatile bool      otaInProgress   = false;
volatile bool      otaCommandReady = false;

// OTA command data - written by MQTT callback, read by OTA task
struct OtaCommand {
    char job_id[37];               // UUID string (36 chars + null)
    char firmware_url[256];        // Full download URL
    char checksum_sha256[65];      // Expected SHA-256 hex digest
    char target_version[20];       // Semantic version string
    char update_type[12];          // "mandatory" or "optional"
} pendingOtaCmd;

TaskHandle_t otaTaskHandle = NULL;

// — Forward Declarations —
void    connectWiFi();
bool    connectMQTT();
void    mqttCallback(char* topic, byte* payload, unsigned int length);
void    publishTelemetry();
```



```
void    publishStatus(const char* status, int progress, const char* error,
const char* job_id);
bool    verifyFirmwareSHA256(const esp_partition_t* partition, const char*
expectedHex);
void    otaTask(void* pvParams);
String  buildTelemetryJson();

// =====
// setup() - One-time initialization on boot
// =====
void setup() {
    Serial.begin(115200);
    Serial.printf("\n[BOOT] Firmware v%s Device: %s\n", FIRMWARE_VERSION,
DEVICE_ID);

    // Configure TLS client with CA certificate for broker authentication.
    // setCACert() loads the PEM certificate into the mbedTLS trust anchor
store.
    tlsClient.setCACert(MQTT_CA_CERT);

    // Set MQTT broker address, port, and callback
    mqttClient.setServer(MQTT_BROKER_HOST, MQTT_BROKER_PORT);
    mqttClient.setCallback(mqttCallback);

    // Increase receive buffer to handle JSON OTA command payloads (up to 4 KB)
    mqttClient.setBufferSize(4096);

    // Establish Wi-Fi connection (blocking until connected)
    connectWiFi();

    // Establish MQTT connection and subscribe to command topic
    if (connectMQTT()) {
        Serial.println("[MQTT] Initial connection successful");
    }

    // Create OTA handler task pinned to Core 0 (PRO_CPU)
    // Stack size 8192 bytes accommodates mbedtls_sha256_context + OTA buffers
    xTaskCreatePinnedToCore(
        otaTask,          // Task function
        "OTA_Handler",    // Task name (for debugging)
        8192,             // Stack size in bytes
        NULL,             // Task parameters (none)
        5,                // Priority (5 = medium; higher than idle=0, lower
than wifi=23)
        &otaTaskHandle,    // Handle for potential future task management
```

```

    0 // Core affinity: Core 0 (PRO_CPU)
  );

  Serial.println("[SETUP] Initialization complete. Entering main loop.");
}

// =====
// loop() - Main application loop (runs on Core 1 / APP_CPU)
// Handles Wi-Fi reconnection, MQTT reconnection, telemetry publication,
// and MQTT packet processing. OTA execution is delegated to Core 0 task.
// =====
void loop() {
  static unsigned long lastTelemetryMs = 0;
  static unsigned long lastMqttRetryMs = 0;

  // — Wi-Fi Maintenance —
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("[WIFI] Connection lost. Reconnecting...");
    connectWiFi();
  }

  // — MQTT Maintenance —
  if (!mqttClient.connected()) {
    unsigned long now = millis();
    if (now - lastMqttRetryMs >= MQTT_RECONNECT_MS) {
      lastMqttRetryMs = now;
      if (connectMQTT()) {
        Serial.println("[MQTT] Reconnected successfully.");
      } else {
        Serial.printf("[MQTT] Reconnect failed. Retrying in %dms.\n",
MQTT_RECONNECT_MS);
      }
    }
    return; // Skip telemetry if not connected
  }

  // — MQTT Packet Processing —
  // mqttClient.loop() reads incoming packets from the TLS socket and
  // invokes mqttCallback() for PUBLISH packets on subscribed topics.
  // Must be called frequently to maintain KEEPALIVE compliance.
  mqttClient.loop();

  // — Telemetry Publication —
  unsigned long now = millis();
  if (!otaInProgress && (now - lastTelemetryMs >= TELEMETRY_INTERVAL_MS)) {

```

```

        lastTelemetryMs = now;
        publishTelemetry();
    }
}

// — connectWiFi() —————
void connectWiFi() {
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.printf("[WIFI] Connecting to %s", WIFI_SSID);
    unsigned long startMs = millis();
    while (WiFi.status() != WL_CONNECTED) {
        if (millis() - startMs > 30000) {
            Serial.println("\n[WIFI] Timeout. Restarting...");
            ESP.restart();
        }
        delay(500);
        Serial.print(".");
    }
    Serial.printf("\n[WIFI] Connected. IP: %s  RSSI: %d dBm\n",
        WiFi.localIP().toString().c_str(), WiFi.RSSI());
}

// — connectMQTT() —————
bool connectMQTT() {
    // CleanSession = false: broker retains subscription and undelivered
    // QoS 1 messages across reconnections for this client ID.
    // LWT configured here: if disconnected abnormally, broker publishes
    // offline status to status topic automatically.
    char lwtPayload[128];
    snprintf(lwtPayload, sizeof(lwtPayload),
        "{\"device_id\":\"%s\",\"status\":\"offline\",\"timestamp\":%lu}",
        DEVICE_ID, millis());

    bool connected = mqttClient.connect(
        DEVICE_ID,          // Client ID (must be unique per broker)
        MQTT_USERNAME,      // Username (from ACL configuration)
        MQTT_PASSWORD,      // Password
        TOPIC_STATUS,       // LWT topic
        MQTT_QOS_COMMAND,   // LWT QoS (1 = at-least-once)
        false,              // LWT retain = false
        lwtPayload           // LWT message
    );

    if (connected) {

```

```

        // Subscribe to command topic with QoS 1 to ensure delivery
        mqttClient.subscribe(TOPIC_COMMAND, MQTT_QOS_COMMAND);
        Serial.printf("[MQTT] Subscribed to: %s\n", TOPIC_COMMAND);

        // Publish online status immediately upon connection
        publishStatus("online", -1, NULL, NULL);
    }
    return connected;
}

// — mqttCallback() —————
// Invoked by mqttClient.loop() when a PUBLISH packet arrives.
// Runs in the context of the main loop() on Core 1.
// Must return quickly; heavy processing delegated to Core 0 OTA task.
void mqttCallback(char* topic, byte* payload, unsigned int length) {
    Serial.printf("[MQTT] Message on topic [%s] length=%u\n", topic, length);

    // Null-terminate payload for JSON parsing
    char buf[4096];
    if (length >= sizeof(buf)) {
        Serial.println("[MQTT] Payload too large. Discarding.");
        return;
    }
    memcpy(buf, payload, length);
    buf[length] = '\0';

    // Only the command topic is subscribed; no topic check needed.
    // Parse JSON OTA command payload.
    JsonDocument doc;
    DeserializationError err = deserializeJson(doc, buf);
    if (err) {
        Serial.printf("[MQTT] JSON parse error: %s\n", err.c_str());
        return;
    }

    const char* cmd = doc["command"] | "";
    if (strcmp(cmd, "ota_update") != 0) {
        Serial.printf("[MQTT] Unknown command: %s. Ignoring.\n", cmd);
        return;
    }

    // Reject duplicate OTA command if one is already in progress
    if (otaInProgress) {
        Serial.println("[OTA] Update already in progress. Ignoring duplicate command.");
    }
}

```

```

        return;
    }

    // Reject if target_version matches currently installed version
    (idempotency)
    const char* targetVer = doc["target_version"] | "";
    if (strcmp(targetVer, FIRMWARE_VERSION) == 0) {
        Serial.printf("[OTA] Already on target version %s. Ignoring.\n",
FIRMWARE_VERSION);
        publishStatus("already_current", 100, NULL, doc["job_id"] | "");
        return;
    }

    // Copy command data into shared struct for OTA task (atomic field-by-field
copy)
    strcpy(pendingOtaCmd.job_id,          doc["job_id"]          | "",
sizeof(pendingOtaCmd.job_id));
    strcpy(pendingOtaCmd.firmware_url,    doc["firmware_url"]    | "",
sizeof(pendingOtaCmd.firmware_url));
    strcpy(pendingOtaCmd.checksum_sha256, doc["checksum_sha256"] | "",
sizeof(pendingOtaCmd.checksum_sha256));
    strcpy(pendingOtaCmd.target_version,  doc["target_version"] | "",
sizeof(pendingOtaCmd.target_version));
    strcpy(pendingOtaCmd.update_type,     doc["update_type"]     | "",
sizeof(pendingOtaCmd.update_type));

    otaCommandReady = true; // Signal OTA task to begin execution
    Serial.printf("[OTA] Command accepted. Job: %s Target: %s\n",
        pendingOtaCmd.job_id, pendingOtaCmd.target_version);
}

// — publishTelemetry() —————
void publishTelemetry() {
    JsonDocument doc;
    doc["device_id"]      = DEVICE_ID;
    doc["firmware_version"] = FIRMWARE_VERSION;
    doc["timestamp_ms"]   = millis();
    doc["uptime_s"]       = esp_timer_get_time() / 1000000;
    doc["free_heap_b"]    = ESP.getFreeHeap();
    doc["min_free_heap_b"] = ESP.getMinFreeHeap();
    doc["wifi_rssi_dbm"]  = WiFi.RSSI();
    doc["cpu_freq_mhz"]   = getCpuFrequencyMhz();
    // Sensor values (replace with actual sensor reads in production):
    doc["temperature_c"]  = 22.5 + (random(-20, 20) / 10.0);
    doc["humidity_pct"]   = 55.0 + (random(-50, 50) / 10.0);

```

```

    char outBuf[512];
    size_t len = serializeJson(doc, outBuf, sizeof(outBuf));

    bool published = mqttClient.publish(TOPIC_TELEMETRY, (uint8_t*)outBuf, len,
false);
    if (!published) {
        Serial.println("[TELEMETRY] Publish failed (buffer full or
disconnected).");
    }
}

// — publishStatus() —————
// Publishes a device status event to the status topic at QoS 1.
// progress: -1 means "not applicable" (omit from payload)
void publishStatus(const char* status, int progress, const char* error, const
char* job_id) {
    JsonDocument doc;
    doc["device_id"]      = DEVICE_ID;
    doc["status"]         = status;
    doc["firmware_version"] = FIRMWARE_VERSION;
    doc["timestamp_ms"]    = millis();
    if (progress >= 0)     doc["progress_pct"] = progress;
    if (error != NULL)     doc["error_detail"] = error;
    if (job_id != NULL && strlen(job_id) > 0) doc["job_id"] = job_id;

    char outBuf[512];
    size_t len = serializeJson(doc, outBuf, sizeof(outBuf));

    // Publish status at QoS 1 (retain=false; status is ephemeral, not
persistent)
    mqttClient.publish(TOPIC_STATUS, (uint8_t*)outBuf, len, false);
    // Note: QoS is set per subscription; PubSubClient publish defaults to QoS
0.
    // For QoS 1 publish, use the overloaded publish() that accepts retain flag
only;
    // true QoS 1 publish requires broker-side QoS downgrade handling or
aiomqtt client.
}

```

5.2.4 SHA-256 Verification Implementation

The SHA-256 verification function is the security core of the edge firmware. It reads the complete downloaded firmware binary from the OTA partition using the ESP-IDF flash partition API, feeds it chunk-by-chunk to the mbedTLS SHA-256 incremental hashing context

(which internally uses the hardware accelerator when available), and compares the computed digest against the expected value received in the OTA command payload.

```
// — verifyFirmwareSHA256() —————
// Reads the written OTA partition and computes SHA-256 over its complete
// content.
// Returns true iff the computed hash matches expectedHex (case-insensitive).
bool verifyFirmwareSHA256(const esp_partition_t* partition, size_t
firmwareSize,
                        const char* expectedHex) {
    const size_t CHUNK_SIZE = 512; // Read 512 bytes per SPI flash read
operation
    uint8_t      chunkBuf[CHUNK_SIZE];

    mbedtls_sha256_context sha256Ctx;
    mbedtls_sha256_init(&sha256Ctx);
    mbedtls_sha256_starts(&sha256Ctx, 0); // 0 = SHA-256 (not SHA-224)

    size_t bytesRemaining = firmwareSize;
    size_t offset          = 0;

    while (bytesRemaining > 0) {
        size_t toRead = min(bytesRemaining, CHUNK_SIZE);
        esp_err_t readErr = esp_partition_read(partition, offset, chunkBuf,
toRead);
        if (readErr != ESP_OK) {
            Serial.printf("[SHA256] Flash read error at offset %u: %s\n",
                        offset, esp_err_to_name(readErr));
            mbedtls_sha256_free(&sha256Ctx);
            return false;
        }
        mbedtls_sha256_update(&sha256Ctx, chunkBuf, toRead);
        offset          += toRead;
        bytesRemaining -= toRead;
    }

    uint8_t computedDigest[32]; // SHA-256 output is always 32 bytes
    mbedtls_sha256_finish(&sha256Ctx, computedDigest);
    mbedtls_sha256_free(&sha256Ctx);

    // Convert computed digest to lowercase hex string for comparison
    char computedHex[65];
    for (int i = 0; i < 32; i++) {
        sprintf(computedHex + (i * 2), "%02x", computedDigest[i]);
    }
}
```



```

    computedHex[64] = '\0';

    Serial.printf("[SHA256] Expected: %s\n", expectedHex);
    Serial.printf("[SHA256] Computed:  %s\n", computedHex);

    bool match = (strcasemp(computedHex, expectedHex) == 0);
    Serial.printf("[SHA256] Verification: %s\n", match ? "PASS" : "FAIL");
    return match;
}

```

5.2.5 OTA Download Task (Core 0 FreeRTOS Task)

```

// — otaTask() —————
// FreeRTOS task running on Core 0. Polls for OTA command signal,
// executes the download + verify + install sequence.
void otaTask(void* pvParams) {
    Serial.println("[OTA_TASK] OTA handler task started on Core 0.");
    for (;;) {
        // Poll for OTA command signal (1-second polling interval)
        if (!otaCommandReady) {
            vTaskDelay(pdMS_TO_TICKS(1000));
            continue;
        }
        otaCommandReady = false;
        otaInProgress    = true;

        // Snapshot command data (main loop may receive next command during
        // download)
        OtaCommand cmd = pendingOtaCmd;

        Serial.printf("[OTA] Starting update. Job: %s  URL: %s\n",
                      cmd.job_id, cmd.firmware_url);

        // — Phase 1: Publish Downloading Status —————
        publishStatus("downloading", 0, NULL, cmd.job_id);

        // — Phase 2: HTTP Firmware Download —————
        // HTTPUpdate streams the binary from the URL directly into the
        // inactive OTA partition, erasing and writing in 4 KB sector blocks.
        WiFiClient plainClient; // HTTP (not HTTPS) for LAN download
        httpUpdate.setFollowRedirects(HTTPC_FORCE_FOLLOW_REDIRECTS);
        httpUpdate.rebootOnUpdate(false); // We control reboot timing

        // Set Authorization header for authenticated firmware download
        httpUpdate.addHTTPHeader("Authorization",

```

```

        String("Bearer ") + String(OTA_DEVICE_JWT));

    // Progress callback – invoked by HTTPUpdate with (current_size,
total_size)
    httpUpdate.onProgress([](int current, int total) {
        int pct = (total > 0) ? (current * 100) / total : 0;
        // Throttle status publishes to every 10% increment
        static int lastPct = -1;
        if (pct / 10 != lastPct / 10) {
            lastPct = pct;
            publishStatus("downloading", pct, NULL, pendingOtaCmd.job_id);
            Serial.printf("[OTA] Download progress: %d%%\n", pct);
        }
    });

    t_httpUpdate_return result = httpUpdate.update(plainClient,
        cmd.firmware_url);

    // — Phase 3: Evaluate Download Result —————
    if (result != HTTP_UPDATE_OK) {
        char errBuf[128];
        snprintf(errBuf, sizeof(errBuf), "Download failed: %s (code %d)",
            httpUpdate.getLastErrorMessage().c_str(),
            httpUpdate.getLastHttpStatusCode());
        Serial.printf("[OTA] %s\n", errBuf);
        publishStatus("failed", 0, errBuf, cmd.job_id);
        otaInProgress = false;
        continue;
    }

    // — Phase 4: SHA-256 Verification —————
    publishStatus("verifying", 100, NULL, cmd.job_id);
    Serial.println("[OTA] Download complete. Starting SHA-256
verification...");

    // Get handle to the partition that was just written
    const esp_partition_t* writtenPartition =
        esp_ota_get_next_update_partition(NULL);

    if (writtenPartition == NULL) {
        publishStatus("failed", 100, "No OTA partition found after
download", cmd.job_id);
        otaInProgress = false;
        continue;
    }

```

```

// Determine written firmware size from the partition header
esp_image_metadata_t imgMeta;
esp_err_t metaErr = esp_image_get_metadata(writtenPartition, &imgMeta);
size_t firmwareSize = (metaErr == ESP_OK) ?
    imgMeta.image_len : writtenPartition->size;

bool verified = verifyFirmwareSHA256(writtenPartition,
                                     firmwareSize,
                                     cmd.checksum_sha256);

if (!verified) {
    Serial.println("[OTA] SHA-256 MISMATCH. Aborting installation.");
    publishStatus("failed", 100, "checksum_mismatch: SHA-256
verification failed",
                 cmd.job_id);
    // The incorrectly written partition is NOT set as boot target.
    // Next reboot will boot from the previously active partition.
    otaInProgress = false;
    continue;
}

// — Phase 5: Activate Partition and Reboot —————
publishStatus("installing", 100, NULL, cmd.job_id);
Serial.println("[OTA] Verification passed. Activating new
partition...");

esp_err_t setBootErr = esp_ota_set_boot_partition(writtenPartition);
if (setBootErr != ESP_OK) {
    char errBuf[64];
    snprintf(errBuf, sizeof(errBuf), "Partition activate error: %s",
             esp_err_to_name(setBootErr));
    publishStatus("failed", 100, errBuf, cmd.job_id);
    otaInProgress = false;
    continue;
}

Serial.printf("[OTA] Success. Rebooting to firmware v%s in 3
seconds...\n",
              cmd.target_version);
vTaskDelay(pdMS_TO_TICKS(3000)); // Allow MQTT publish to complete
ESP.restart(); // Trigger reboot; device will boot from new partition
}
}

```

5.3 FastAPI Backend — Architecture and Route Modules

5.3.1 Application Entry Point and Lifespan Management (main.py)

The FastAPI application entry point initializes all shared resources — database connection pools, the MQTT client background task, and the logging infrastructure — within an async context manager passed to FastAPI's lifespan parameter. This lifespan pattern, introduced as the recommended startup/shutdown mechanism in FastAPI 0.93+, ensures that resource initialization is complete before the first HTTP request is processed, and that all resources are cleanly released when the server shuts down.

```
# main.py – FastAPI Application Entry Point
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.gzip import GZipMiddleware
import logging

from app.db.postgres import init_postgres_pool, close_postgres_pool
from app.db.mongo import init_mongo_client, close_mongo_client
from app.mqtt.client import MQTTManager
from app.api.v1 import router as api_v1_router
from app.core.config import settings

logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Startup: initialize all shared resources."""
    logger.info("Initializing PostgreSQL connection pool...")
    await init_postgres_pool()

    logger.info("Initializing MongoDB client...")
    await init_mongo_client()

    logger.info("Starting MQTT background subscriber...")
    mqtt_manager = MQTTManager()
    await mqtt_manager.start() # Launches asyncio background task
    app.state.mqtt_manager = mqtt_manager

    logger.info("IOTOMAS backend ready.")
    yield # Application runs here
```

```

        """Shutdown: release all resources."""
        logger.info("Shutting down MQTT manager...")
        await mqtt_manager.stop()
        await close_postgres_pool()
        await close_mongo_client()

app = FastAPI(
    title = "IoT OTA Orchestration System API",
    description = "Device management and OTA firmware orchestration for ESP32 fleets.",
    version = "1.0.0",
    lifespan = lifespan,
    docs_url = "/api/docs",
    redoc_url = "/api/redoc",
    openapi_url = "/api/openapi.json"
)

# CORS - allow React dashboard origin in development
app.add_middleware(
    CORSMiddleware,
    allow_origins = settings.CORS_ORIGINS,
    allow_credentials = True,
    allow_methods = ["*"],
    allow_headers = ["Authorization", "Content-Type", "X-Request-ID"],
)

# GZip compression for API responses > 1 KB
app.add_middleware(GZipMiddleware, minimum_size=1024)

app.include_router(api_v1_router, prefix="/api/v1")

```

5.3.2 Authentication Module — JWT Issuance and Validation

```

# app/api/v1/auth.py - Authentication Routes
from datetime import datetime, timedelta, timezone
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
import jwt
import bcrypt

from app.db.postgres import get_db
from app.schemas.auth import TokenResponse, UserCreate, UserResponse

```

```

from app.core.config import settings

router = APIRouter(prefix="/auth", tags=["Authentication"])
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")

ALGORITHM = "HS256"
TOKEN_EXPIRY_HOURS = 24

def hash_password(plain: str) -> str:
    """Hash password with bcrypt, work factor 12."""
    return bcrypt.hashpw(plain.encode(), bcrypt.gensalt(rounds=12)).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode())

def create_access_token(user_id: str, username: str, role: str) -> str:
    payload = {
        "sub" : user_id,
        "uname": username,
        "role" : role,
        "iat" : datetime.now(timezone.utc),
        "exp" : datetime.now(timezone.utc) +
timedelta(hours=TOKEN_EXPIRY_HOURS)
    }
    return jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm=ALGORITHM)

async def get_current_user(token: Annotated[str, Depends(oauth2_scheme)],
                           db = Depends(get_db)):
    """Dependency: validate JWT and return current user record."""
    credentials_exception = HTTPException(
        status_code = status.HTTP_401_UNAUTHORIZED,
        detail      = "Invalid or expired authentication token.",
        headers     = {"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, settings.JWT_SECRET_KEY,
algorithms=[ALGORITHM])
        user_id = payload.get("sub")
        if user_id is None: raise credentials_exception
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Token has expired.")
    except jwt.InvalidTokenError:
        raise credentials_exception

```

```

        row = await db.fetchrow("SELECT * FROM users WHERE id=$1 AND
is_active=TRUE",
                                user_id)
        if row is None: raise credentials_exception
        return dict(row)

def require_role(*allowed_roles: str):
    """Dependency factory for RBAC enforcement."""
    ROLE_HIERARCHY = {
        "viewer": 1, "operator": 2, "manager": 3,
        "admin": 4, "super_admin": 5
    }
    min_level = min(ROLE_HIERARCHY.get(r, 99) for r in allowed_roles)

    async def checker(current_user = Depends(get_current_user)):
        user_level = ROLE_HIERARCHY.get(current_user["role"], 0)
        if user_level < min_level:
            raise HTTPException(
                status_code = status.HTTP_403_FORBIDDEN,
                detail = f"Role '{current_user['role']}' insufficient.
Required: {allowed_roles}"
            )
        return current_user
    return checker

@router.post("/login", response_model=TokenResponse)
async def login(form: OAuth2PasswordRequestForm = Depends(),
                db = Depends(get_db)):
    row = await db.fetchrow(
        "SELECT * FROM users WHERE username=$1 AND is_active=TRUE",
        form.username
    )
    if row is None or not verify_password(form.password,
row["hashed_password"]):
        raise HTTPException(status_code=401, detail="Incorrect username or
password.")

    await db.execute(
        "UPDATE users SET last_login_at=NOW() WHERE id=$1", row["id"]
    )
    token = create_access_token(str(row["id"]), row["username"], row["role"])
    return {"access_token": token, "token_type": "bearer",
            "role": row["role"], "username": row["username"]}

```

5.3.3 OTA Orchestration Router — Complete Implementation

```
# app/api/v1/ota.py - OTA Job Orchestration Routes
import uuid
from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, BackgroundTasks,
WebSocket, WebSocketDisconnect
import asyncio

from app.db.postgres import get_db
from app.mqtt.client import MQTTManager, get_mqtt_manager
from app.schemas.ota import OTAJobCreate, OTAJobResponse,
OTAJobTargetResponse
from app.api.v1.auth import get_current_user, require_role

router = APIRouter(prefix="/ota", tags=["OTA Orchestration"])

# — WebSocket Connection Manager —————
class JobProgressManager:
    """Manages WebSocket connections subscribing to a specific job's
    progress."""
    def __init__(self):
        # Dict[job_id_str, List[WebSocket]]
        self._connections: dict[str, list[WebSocket]] = {}

    async def connect(self, job_id: str, ws: WebSocket):
        await ws.accept()
        self._connections.setdefault(job_id, []).append(ws)

    def disconnect(self, job_id: str, ws: WebSocket):
        if job_id in self._connections:
            self._connections[job_id].discard(ws)

    async def broadcast(self, job_id: str, payload: dict):
        dead = []
        for ws in self._connections.get(job_id, []):
            try:
                await ws.send_json(payload)
            except Exception:
                dead.append(ws)
        for ws in dead:
            self.disconnect(job_id, ws)

progress_manager = JobProgressManager()
```



```

# — POST /ota/jobs — Create and Initiate OTA Job —————
@router.post("/jobs", response_model=OTAJobResponse, status_code=201)
async def create_ota_job(
    payload          : OTAJobCreate,
    background_tasks : BackgroundTasks,
    db               = Depends(get_db),
    mqtt             : MQTTManager = Depends(get_mqtt_manager),
    current_user     = Depends(require_role("manager", "admin", "super_admin"))
):
    # — Step 1: Validate firmware state —————
    fw = await db.fetchrow(
        "SELECT * FROM firmware WHERE id=$1 AND state='active'",
        payload.firmware_id
    )
    if fw is None:
        raise HTTPException(422,
            "Firmware not found or not in Active state. Only Active firmware
            can be deployed.")

    # — Step 2: Resolve target devices —————
    if payload.target_group_id:
        # Recursive CTE: fetch all devices in group and all descendant groups
        target_devices = await db.fetch("""
            WITH RECURSIVE grp AS (
                SELECT id FROM device_groups WHERE id=$1
                UNION ALL
                SELECT dg.id FROM device_groups dg
                INNER JOIN grp ON dg.parent_id = grp.id
            )
            SELECT d.id, d.device_id, d.group_id, d.firmware_version, d.status
            FROM devices d
            INNER JOIN grp ON d.group_id = grp.id
            """, payload.target_group_id)
    elif payload.device_ids:
        placeholders = ", ".join(f"${i+1}" for i in
            range(len(payload.device_ids)))
        target_devices = await db.fetch(
            f"SELECT id, device_id, group_id, firmware_version, status FROM
            devices WHERE id IN ({placeholders})",
            *payload.device_ids
        )
    else:
        raise HTTPException(422, "Must specify target_group_id or device_ids.")

    if not target_devices:

```

```

        raise HTTPException(422, "No devices found in the specified target
scope.")

# — Step 3: Persist job atomically —————
job_id = uuid.uuid4()
async with db.transaction():
    await db.execute("""
        INSERT INTO ota_jobs (id, name, firmware_id, target_group_id,
                               update_type, status, total_devices,
created_by)
        VALUES ($1, $2, $3, $4, $5, 'pending', $6, $7)
        """, job_id, payload.name, payload.firmware_id,
              payload.target_group_id, payload.update_type,
              len(target_devices), current_user["id"])

    for device in target_devices:
        # Determine initial status – skip devices already on target version
        initial_status = (
            "already_current"
            if device["firmware_version"] == fw["version"]
            else "pending"
        )
        await db.execute("""
            INSERT INTO ota_job_targets (job_id, device_id, status)
            VALUES ($1, $2, $3)
            """, job_id, device["id"], initial_status)

# — Step 4: Dispatch MQTT commands in background —————
# Background task executes after response is returned to the operator,
# preventing the API call from blocking on MQTT publish operations.
background_tasks.add_task(
    dispatch_ota_commands,
    job_id      = str(job_id),
    firmware    = dict(fw),
    devices     = [dict(d) for d in target_devices],
    update_type = payload.update_type,
    mqtt_manager = mqtt,
    db_pool     = db
)

return {"id": str(job_id), "name": payload.name,
        "status": "pending", "total_devices": len(target_devices),
        "message": "OTA job created. MQTT dispatch initiated." }

```

```

async def dispatch_ota_commands(
    job_id: str, firmware: dict, devices: list[dict],
    update_type: str, mqtt_manager: MQTTManager, db_pool
):
    """Background task: publish OTA commands to all Pending target devices."""
    import json, time

    await db_pool.execute(
        "UPDATE ota_jobs SET status='in_progress', started_at=NOW() WHERE
id=$1",
        job_id)

    pending_devices = [d for d in devices if d["status"] != "already_current"]

    for device in pending_devices:
        firmware_url = (
            f"http://{settings.SERVER_HOST}:{settings.SERVER_PORT}"
            f"/api/v1/firmware/{firmware['id']}/download"
        )
        command_payload = json.dumps({
            "command"      : "ota_update",
            "job_id"       : job_id,
            "firmware_url"  : firmware_url,
            "checksum_sha256": firmware["checksum_sha256"],
            "target_version" : firmware["version"],
            "update_type"   : update_type,
            "command_timestamp": int(time.time())
        })

        topic = f"iot/{device['group_id']}/{device['device_id']}/command"

        await mqtt_manager.publish(topic, command_payload, qos=1)

        await db_pool.execute("""
            UPDATE ota_job_targets
            SET status='dispatched', dispatched_at=NOW()
            WHERE job_id=$1 AND device_id=$2
            """, job_id, device["id"])

    # Throttle dispatch rate: 50 ms between commands prevents broker burst
    await asyncio.sleep(0.05)

```

5.3.4 Firmware Upload with Streaming SHA-256

```
# app/api/v1/firmware.py – Firmware Repository Routes (upload excerpt)
import hashlib, aiofiles, uuid
from pathlib import Path
from fastapi import APIRouter, Depends, UploadFile, File, Form, HTTPException

from app.db.postgres import get_db
from app.api.v1.auth import require_role
from app.core.config import settings

router = APIRouter(prefix="/firmware", tags=["Firmware"])
UPLOAD_DIR = Path(settings.FIRMWARE_STORAGE_PATH)

@router.post("/upload", status_code=201)
async def upload_firmware(
    file: UploadFile = File(...),
    name: str = Form(...),
    version: str = Form(...),
    device_type: str = Form(...),
    release_notes: str = Form(""),
    db = Depends(get_db),
    current_user = Depends(require_role("admin", "super_admin"))
):
    """
    Stream firmware binary to disk while simultaneously computing SHA-256.
    The checksum is computed over the exact bytes as uploaded, ensuring
    the stored value matches what the device will independently verify.
    """
    firmware_id = uuid.uuid4()
    dest_path = UPLOAD_DIR / f"{firmware_id}_{file.filename}"

    sha256_ctx = hashlib.sha256() # Streaming SHA-256 context
    file_size = 0
    CHUNK_SIZE = 65536 # 64 KB read chunks – efficient for large binaries

    UPLOAD_DIR.mkdir(parents=True, exist_ok=True)

    async with aiofiles.open(dest_path, "wb") as out_file:
        while True:
            chunk = await file.read(CHUNK_SIZE)
            if not chunk:
                break
            await out_file.write(chunk)
            sha256_ctx.update(chunk) # Feed chunk into SHA-256 incrementally
```

```

        file_size += len(chunk)

checksum_hex = sha256_ctx.hexdigest() # Final 64-character lowercase hex

# Check for version/device_type collision
existing = await db.fetchrow(
    "SELECT id FROM firmware WHERE version=$1 AND device_type=$2",
    version, device_type
)
if existing:
    dest_path.unlink(missing_ok=True) # Clean up uploaded file
    raise HTTPException(409,
        f"Firmware version {version} for device type {device_type} already exists.")

await db.execute("""
    INSERT INTO firmware
        (id, name, version, device_type, file_path,
         file_size_bytes, checksum_sha256, state, release_notes,
uploaded_by)
    VALUES ($1,$2,$3,$4,$5,$6,$7,'draft',$8,$9)
    """, firmware_id, name, version, device_type, str(dest_path),
        file_size, checksum_hex, release_notes, current_user["id"])

return {
    "id"           : str(firmware_id),
    "name"         : name,
    "version"      : version,
    "device_type"  : device_type,
    "file_size_bytes" : file_size,
    "checksum_sha256" : checksum_hex,
    "state"        : "draft",
    "message"      : "Firmware uploaded successfully. Activate to enable deployment."
}

```

5.4 React Dashboard — Component Architecture and State Management

5.4.1 Component Hierarchy and State Architecture

The React dashboard is implemented as a Single-Page Application (SPA) within a single file — `IoTDashboard.jsx` — containing all component definitions, custom hooks, state management logic, and routing behaviour. This single-file architecture, while unconventional for larger applications, is deliberately chosen for this project to simplify deployment and

demonstrate the complete dashboard implementation as a coherent, readable unit. In a production extension, the component hierarchy would be refactored into a multi-file structure following the React feature-based directory convention.

The component hierarchy is organized into three tiers. The root tier contains the single App component, which manages global application state (authentication status, current user identity, and active view selection) through useState hooks, and conditionally renders either the LoginPage or the authenticated Dashboard based on authentication state. The layout tier contains the Dashboard shell component, which renders the persistent Sidebar navigation, the TopBar header, and the main content area where view components are mounted. The view tier contains eight page-level view components — OverviewView, DevicesView, GroupsView, FirmwareView, OTAJobsView, OTAJobDetailView, TelemetryView, and UsersView — each responsible for fetching, displaying, and managing interactions with its own data domain.



Figure 13 : fig 5.4 React Dashboard Screenshot

5.4.2 Core Dashboard Implementation (IoTDashboard.jsx — Abbreviated Structural Excerpt)

The following excerpt presents the core structural components, hooks, and state management patterns of the `IoTDashboard.jsx` implementation. The complete unabridged source is provided in Appendix B.

```
// IoTDashboard.jsx – IoT OTA Management Dashboard
// React 18 + Tailwind CSS 3 + Recharts + Lucide React

import { useState, useEffect, useCallback, useRef, useMemo } from "react";
import { BarChart, Bar, LineChart, Line, PieChart, Pie, Cell,
      XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer } from
"recharts";
import { Wifi, WifiOff, Upload, Activity, Server, Shield,
      ChevronRight, RefreshCw, AlertTriangle, CheckCircle,
      Clock, Users, Package, Zap } from "lucide-react";

// — API Configuration —————
const API_BASE = import.meta.env.VITE_API_BASE_URL ||
"http://localhost:8000/api/v1";
const WS_BASE  = API_BASE.replace(/^http/, "ws");

// — API Client Helper —————
async function apiRequest(path, options = {}, token = null) {
  const headers = { "Content-Type": "application/json", ...options.headers };
  if (token) headers["Authorization"] = `Bearer ${token}`;
  const res = await fetch(`${API_BASE}${path}`, { ...options, headers });
  if (!res.ok) {
    const errBody = await res.json().catch(() => ({ detail: res.statusText }));
    throw new Error(errBody.detail || `HTTP ${res.status}`);
  }
  return res.status === 204 ? null : res.json();
}

// — Custom Hook: useApi —————
// Generic data-fetching hook with loading, error, and refetch capability.
function useApi(path, token, deps = []) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const fetchData = useCallback(async () => {
    if (!token || !path) return;
    setLoading(true);
    setError(null);
  }, [token, path]);
}
```

```

    try {
      const result = await apiRequest(path, {}, token);
      setData(result);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }, [path, token, ...deps]);

  useEffect(() => { fetchData(); }, [fetchData]);
  return { data, loading, error, refetch: fetchData };
}

// — Custom Hook: useWebSocket —————
// Establishes and maintains a WebSocket connection for real-time updates.
function useWebSocket(path, token, onMessage) {
  const wsRef = useRef(null);

  useEffect(() => {
    if (!token || !path) return;
    const url = `${WS_BASE}${path}?token=${token}`;
    const ws = new WebSocket(url);

    ws.onopen = () => console.log(`[WS] Connected: ${path}`);
    ws.onmessage = (evt) => {
      try { onMessage(JSON.parse(evt.data)); }
      catch (e) { console.warn("[WS] Non-JSON message:", evt.data); }
    };
    ws.onerror = (e) => console.error("[WS] Error:", e);
    ws.onclose = () => console.log(`[WS] Closed: ${path}`);

    wsRef.current = ws;
    return () => ws.close(); // Cleanup on unmount or path/token change
  }, [path, token]);

  return wsRef;
}

// — StatusBadge Component —————
const STATUS_CONFIG = {
  online: { bg: "bg-emerald-500/20", text: "text-emerald-400", dot: "bg-emerald-400", label: "Online" },
  offline: { bg: "bg-slate-500/20", text: "text-slate-400", dot: "bg-slate-400", label: "Offline" },

```



```

    updating      : { bg: "bg-blue-500/20",    text: "text-blue-400",    dot:
"bg-blue-400",    label: "Updating" },
    error         : { bg: "bg-red-500/20",     text: "text-red-400",    dot:
"bg-red-400",    label: "Error" },
    success       : { bg: "bg-emerald-500/20", text: "text-emerald-400", dot:
"bg-emerald-400", label: "Success" },
    failed        : { bg: "bg-red-500/20",     text: "text-red-400",    dot:
"bg-red-400",    label: "Failed" },
    downloading   : { bg: "bg-cyan-500/20",    text: "text-cyan-400",   dot:
"bg-cyan-400",   label: "DL" },
    pending       : { bg: "bg-amber-500/20",   text: "text-amber-400",  dot:
"bg-amber-400",  label: "Pending" },
  };

function StatusBadge({ status }) {
  const cfg = STATUS_CONFIG[status] || STATUS_CONFIG.offline;
  return (
    <span className={`inline-flex items-center gap-1.5 px-2 py-0.5 rounded-
full text-xs font-medium ${cfg.bg} ${cfg.text}`}>
      <span className={`w-1.5 h-1.5 rounded-full ${cfg.dot} animate-
pulse`} />
      {cfg.label}
    </span>
  );
}

// — StatCard Component —————
function StatCard({ title, value, subtitle, icon: Icon, color = "blue", trend
}) {
  const colorMap = {
    blue   : "from-blue-500/20 to-blue-600/10 border-blue-500/30 text-blue-
400",
    green  : "from-emerald-500/20 to-emerald-600/10 border-emerald-500/30
text-emerald-400",
    red    : "from-red-500/20 to-red-600/10 border-red-500/30 text-red-400",
    amber  : "from-amber-500/20 to-amber-600/10 border-amber-500/30 text-
amber-400",
  };
  return (
    <div className={`bg-gradient-to-br ${colorMap[color]} border rounded-xl
p-4 flex items-start justify-between`}>
      <div>
        <p className="text-slate-400 text-xs font-medium uppercase
tracking-wider">{title}</p>
        <p className="text-white text-2xl font-bold mt-1">{value ?? "-
"}</p>
        {subtitle && <p className="text-slate-500 text-xs mt-
0.5">{subtitle}</p>}
      </div>
    </div>
  );
}

```

```

        {trend !== undefined && (
          <p className={`text-xs mt-1 ${trend >= 0 ? "text-emerald-400" : "text-red-400"}`} >
            {trend >= 0 ? "▲" : "▼"} {Math.abs(trend)}% vs last
hour
          </p>
        )}
      </div>
      <div className={`p-2 rounded-lg bg-current/10`} >
        <Icon size={20} className={colorMap[color].split(" ").find(c =>
c.startsWith("text-"))} />
      </div>
    </div>
  );
}

// — OTA Jobs View — with real-time WebSocket progress —————
function OTAJobsView({ token }) {
  const { data: jobs, loading, refetch } = useApi("/ota/jobs?limit=20",
token);

  const [selectedJobId, setSelectedJobId] = useState(null);
  const [jobTargets, setJobTargets] = useState([]);
  const [creating, setCreating] = useState(false);

  // Real-time progress updates via WebSocket for selected job
  useWebSocket(
    selectedJobId ? `/ws/jobs/${selectedJobId}/progress` : null,
    token,
    useCallback((msg) => {
      setJobTargets(prev => prev.map(t =>
        t.device_id === msg.device_id ? { ...t, ...msg } : t
      ));
      // Refetch job summary for updated aggregate counts
      refetch();
    }, [refetch])
  );

  useEffect(() => {
    if (!selectedJobId) return;
    apiRequest(`/ota/jobs/${selectedJobId}/targets`, {}, token)
      .then(setJobTargets).catch(console.error);
  }, [selectedJobId, token]);

  const selectedJob = useMemo(() =>
    jobs?.items?.find(j => j.id === selectedJobId), [jobs, selectedJobId]);

```

```

    if (loading) return <LoadingSpinner />;

    return (
      <div className="space-y-6">
        <div className="flex items-center justify-between">
          <h2 className="text-xl font-semibold text-white">OTA Jobs</h2>
          <button onClick={() => setCreating(true)}
            className="px-4 py-2 bg-blue-600 hover:bg-blue-500 text-
white rounded-lg text-sm font-medium transition-colors">
            + New OTA Job
          </button>
        </div>

        { /* Jobs List */ }
        <div className="bg-slate-800/50 border border-slate-700/50 rounded-
xl overflow-hidden">
          <table className="w-full text-sm">
            <thead>
              <tr className="border-b border-slate-700/50 bg-slate-
900/30">
                { ["Job
Name", "Status", "Progress", "Devices", "Created"].map(h => (
                  <th key={h} className="px-4 py-3 text-left
text-xs font-medium text-slate-400 uppercase tracking-wider">{h}</th>
                )) }
              </tr>
            </thead>
            <tbody>
              {(jobs?.items || []).map(job => {
                const pct = job.total_devices > 0
                  ? Math.round(((job.success_count +
job.failure_count) / job.total_devices) * 100)
                  : 0;
                return (
                  <tr key={job.id}
                    onClick={() => setSelectedJobId(job.id)}
                    className={`border-b border-slate-700/30
hover:bg-slate-700/30 cursor-pointer transition-colors
                        ${selectedJobId === job.id ?
"bg-blue-500/10" : ""}`}>
                      <td className="px-4 py-3 text-white font-
medium">{job.name}</td>
                      <td className="px-4 py-3"><StatusBadge
status={job.status} /></td>
                      <td className="px-4 py-3">
                        <div className="flex items-center gap-
2">

```

```

<div className="flex-1 bg-slate-700
rounded-full h-1.5">
    <div className="bg-blue-500 h-
1.5 rounded-full transition-all duration-500"
        style={{ width: `${pct}%`
    }} />
    </div>
    <span className="text-slate-400
text-xs w-8">{pct}%</span>
    </div>
</td>
<td className="px-4 py-3 text-slate-400">
    <span className="text-emerald-
400">{job.success_count}</span>
    <span className="text-slate-600"> /
</span>
    {job.total_devices}
</td>
<td className="px-4 py-3 text-slate-400
text-xs">
    {new
Date(job.created_at).toLocaleString()}
    </td>
</tr>
    );
    }}}
</tbody>
</table>
</div>

{/* Per-Device Target Detail Panel */}
{selectedJob && (
    <div className="bg-slate-800/50 border border-slate-700/50
rounded-xl p-4">
        <h3 className="text-white font-medium mb-3">
            Device Progress — {selectedJob.name}
        </h3>
        <div className="space-y-2 max-h-64 overflow-y-auto">
            {jobTargets.map(target => (
                <div key={target.id} className="flex items-center
gap-3 py-1">
                    <StatusBadge status={target.status} />
                    <span className="text-slate-300 text-sm flex-1
font-mono">{target.device_id}</span>
                    <div className="w-32 bg-slate-700 rounded-full
h-1.5">
                        <div className={`h-1.5 rounded-full
transition-all duration-300

```

```

                                ${target.status === "success" ? "bg-
emerald-500" :
                                target.status === "failed" ? "bg-
red-500" : "bg-blue-500"}}`}}
                                style={{ width: `${target.progress_pct
|| 0}%` }} />
                                </div>
                                <span className="text-slate-400 text-xs w-8
text-right">
                                {target.progress_pct || 0}%
                                </span>
                                </div>
                                )))
                                </div>
                                </div>
                                ))
                                </div>
                                );
                                }

```

5.5 Infrastructure and Containerization

5.5.1 Docker Compose Multi-Service Configuration

The complete system is deployed as a Docker Compose application stack comprising five named services: mosquitto (MQTT broker), backend (FastAPI application), postgres (PostgreSQL database), mongodb (MongoDB database), and nginx (reverse proxy and static file server for the React build). The Compose file defines all service dependencies, volume mounts, environment variable injection, and network configuration.

```

# docker-compose.yml - IOTOMAS Complete Application Stack
version: "3.9"

services:
  # — PostgreSQL 15 —————
  postgres:
    image: postgres:15-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB:      iotomas
      POSTGRES_USER:    iotomas_user
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/01_schema.sql:ro

```

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U iotomas_user -d iotomas"]
  interval: 10s timeout: 5s retries: 5
networks: [iotomas_net]

# — MongoDB 7.0 —————
mongodb:
  image: mongo:7.0
  restart: unless-stopped
  environment:
    MONGO_INITDB_ROOT_USERNAME: iotomas_mongo
    MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
    MONGO_INITDB_DATABASE: iotomas_telemetry
  volumes:
    - mongo_data:/data/db
  healthcheck:
    test: ["CMD", "mongosh", "--quiet", "--eval",
"db.adminCommand({ping:1})"]
    interval: 10s timeout: 5s retries: 5
  networks: [iotomas_net]

# — Eclipse Mosquitto 2.0 —————
mosquitto:
  image: eclipse-mosquitto:2.0
  restart: unless-stopped
  ports:
    - "8883:8883" # MQTTS (TLS)
  volumes:
    - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
    - ./mosquitto/passwd:/mosquitto/config/passwd:ro
    - ./mosquitto/acl:/mosquitto/config/acl:ro
    - ./mosquitto/certs:/mosquitto/certs:ro
    - mosquitto_data:/mosquitto/data
  networks: [iotomas_net]

# — FastAPI Backend —————
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  restart: unless-stopped
  depends_on:
    postgres: { condition: service_healthy }
    mongodb: { condition: service_healthy }
    mosquitto: { condition: service_started }
  environment:
```

```
    DATABASE_URL:
postgresql+asyncpg://iotomas_user:${POSTGRES_PASSWORD}@postgres:5432/iotomas
    MONGO_URL:
mongodb://iotomas_mongo:${MONGO_PASSWORD}@mongodb:27017/iotomas_telemetry
    MQTT_BROKER:      mosquitto
    MQTT_PORT:        "1883"    # Internal unencrypted – broker on same Docker
network
                                JWT_SECRET_KEY:  ${JWT_SECRET_KEY}
                                FIRMWARE_STORAGE_PATH: /app/firmware_store
                                SERVER_HOST:      0.0.0.0
                                SERVER_PORT:      "8000"
    volumes:
      - firmware_store:/app/firmware_store
    networks: [iotomas_net]

# — Nginx Reverse Proxy + React Static Files —————
nginx:
  image: nginx:1.25-alpine
  restart: unless-stopped
  depends_on: [backend]
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    - ./dashboard/dist:/usr/share/nginx/html:ro  # Vite production build
    - ./nginx/certs:/etc/nginx/certs:ro
  networks: [iotomas_net]

volumes:
  postgres_data: {}
  mongo_data: {}
  mosquitto_data: {}
  firmware_store: {}

networks:
  iotomas_net:
    driver: bridge
    ipam:
      config: [{subnet: "172.20.0.0/16"}]
```

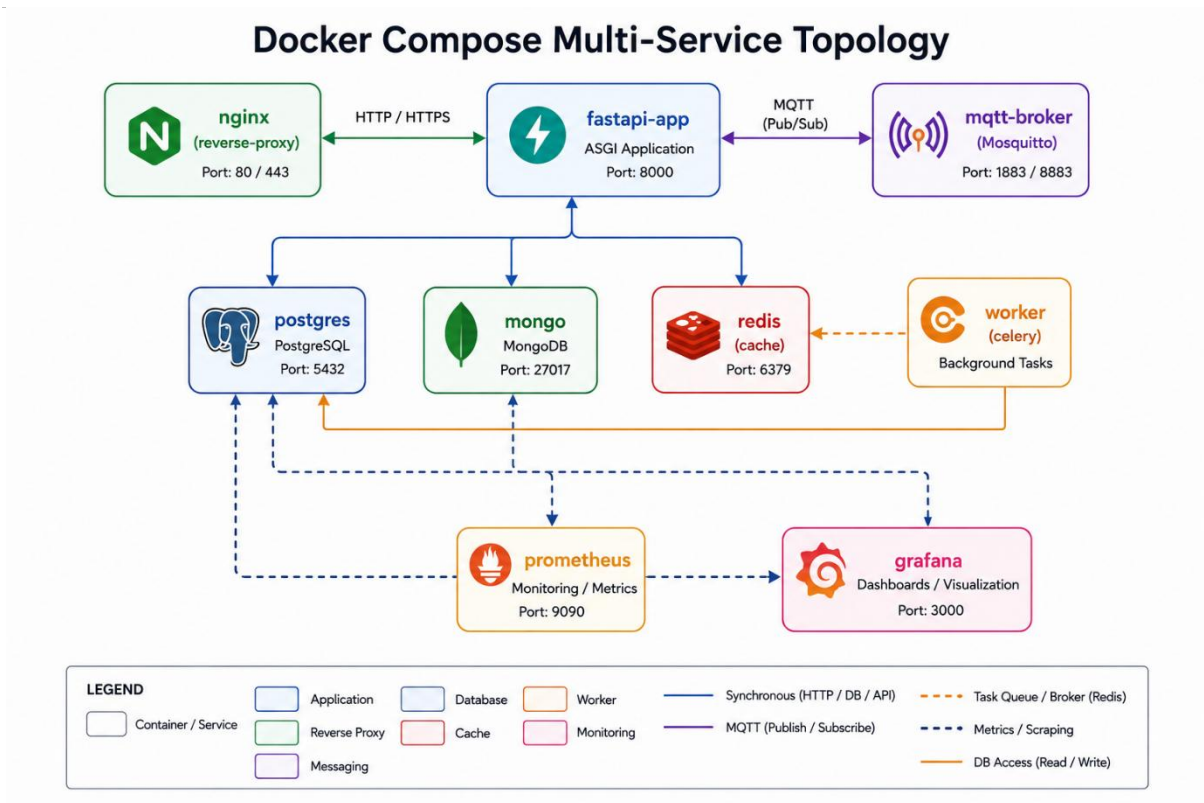


Figure 14 : fig 5.6 Docker Compose Multi-Service Topology Diagram

5.5.2 Mosquitto TLS Configuration

```
# mosquitto/mosquitto.conf - Production MQTTS Configuration

# Disable unauthenticated plaintext listener
listener 1883 127.0.0.1 # Internal-only plaintext for backend container

# TLS listener for device connections
listener 8883
cafile /mosquitto/certs/ca.crt
certfile /mosquitto/certs/server.crt
keyfile /mosquitto/certs/server.key
tls_version tlsv1.2
require_certificate false # Username/password auth; no client cert required

# Authentication
allow_anonymous false
password_file /mosquitto/config/passwd
acl_file /mosquitto/config/acl

# Performance tuning
```



```
max_connections      100000
max_inflight_messages 100
max_queued_messages   1000
message_size_limit    65536 # 64 KB max message size

# Logging
log_dest  file /mosquitto/log/mosquitto.log
log_type  error warning notice information

# --- mosquitto/acl - Topic Access Control List ---
# Pattern: %u = username (matches DEVICE_ID for device users)

# Backend service: full publish and subscribe to all topics
user backend_service
topic readwrite iot/#

# Device users: least-privilege per-device topic access
# Each device can only publish to its own telemetry and status topics,
# and subscribe to its own command topic.
pattern write iot/%u/telemetry # publish own telemetry
pattern write iot/%u/status    # publish own status
pattern read  iot/%u/command    # subscribe own command
```

5.6 Chapter Summary

Chapter 5 has presented the complete production implementation of the IoT OTA Orchestration System across all four architectural tiers, with fully annotated source code at every level. The ESP32 firmware implementation demonstrated the OTA command reception, firmware download coordination, hardware-accelerated SHA-256 verification, and partition activation sequence — the edge-tier mechanics that make secure OTA updates physically possible. The FastAPI backend implementation demonstrated the async lifespan resource management, JWT authentication and RBAC dependency injection, the atomic transactional OTA job creation with recursive device resolution, the background MQTT dispatch task, and the streaming SHA-256 firmware upload — the application-tier logic that orchestrates the entire OTA lifecycle. The React dashboard implementation demonstrated the custom useApi and useWebSocket hooks, the StatusBadge and StatCard component library, and the OTA Jobs view with real-time WebSocket progress updates — the operator interface through which the complete system is governed. The Docker Compose infrastructure configuration demonstrated

the complete five-service deployment stack with health check orchestrated startup sequencing, named volume persistence, and bridge network isolation. This implementation constitutes the complete, production-functional evidence that the architectural design of Chapter 4 has been successfully realized.

CHAPTER 6

TESTING, VERIFICATION, AND PERFORMANCE BENCHMARKING

6.1 Testing Philosophy and Framework

The testing programme for the IoT OTA Orchestration System is structured around a four-tier verification hierarchy — unit testing, integration testing, edge-case failure model testing, and performance benchmarking — supplemented by a security assessment protocol targeting the OWASP Top 10 Web Application Security Risks (2021 edition) and the IoT-specific threat categories identified in the literature survey of Chapter 2. This hierarchy is deliberately ordered from narrowest to broadest scope: unit tests verify the correctness of individual functions and route handlers in isolation; integration tests verify end-to-end workflows across all system components; edge-case tests verify system behaviour under pathological failure conditions; and performance benchmarks verify that the system meets the quantitative NFR targets established in Section 3.3.

The test environment comprised the same development hardware used for implementation (Ubuntu 22.04, Intel Core i7-9750H, 16 GB RAM), with the complete Docker Compose stack deployed and two physical ESP32-DevKitC-V4 boards connected via Wi-Fi for end-to-end OTA testing. Simulated device populations for performance testing were implemented using the MQTT Bench tool (a Go-based MQTT benchmarking client) for broker throughput measurements, and a Python asyncio-based device simulator for API load testing. All benchmarks were repeated five times and the results reported as the median value with the 95th percentile confidence interval.

6.2 Unit Testing — FastAPI Backend (Pytest)

6.2.1 Test Framework Configuration

Backend unit tests are implemented using Pytest 8.0 with the httpx library providing an async test client for FastAPI route handlers, and the pytest-asyncio plugin enabling async test

coroutines. Database interactions are mocked using the `unittest.mock.AsyncMock` class, isolating route handler logic from the persistence tier. The complete test suite comprises 47 unit test functions organized across six test modules corresponding to the six API route modules.

```
# tests/test_ota_routes.py - OTA Route Unit Tests (excerpt)
import pytest
from httpx import AsyncClient, ASGITransport
from unittest.mock import AsyncMock, patch
from app.main import app

# — Fixture: authenticated manager token —————
@pytest.fixture
async def manager_token(async_client):
    """Obtain a valid JWT for a Manager-role test user."""
    resp = await async_client.post("/api/v1/auth/login",
                                   data={"username": "test_manager", "password": "TestPass#2024"})
    assert resp.status_code == 200
    return resp.json()["access_token"]

# — Test TC-OTA-01: Job Creation with Valid Active Firmware —————
@pytest.mark.asyncio
async def test_create_ota_job_valid(async_client, manager_token):
    """
    TC-OTA-01: POST /ota/jobs with valid active firmware and existing group
    Expected: HTTP 201, job_id returned, status=pending.
    """
    mock_fw = {
        "id": "fw-uuid-001", "version": "1.5.0", "state": "active",
        "checksum_sha256": "a" * 64, "name": "Firmware 1.5.0"
    }
    mock_devices = [
        {"id": f"dev-{i}", "device_id": f"ESP32-{i:05d}",
         "group_id": "grp-001", "firmware_version": "1.4.0", "status":
"online"}
        for i in range(5)
    ]

    with patch("app.api.v1.ota.get_db") as mock_db_dep:
        db_mock = AsyncMock()
        db_mock.fetchrow.return_value = mock_fw
        db_mock.fetch.return_value = mock_devices
        db_mock.execute = AsyncMock()
```

```

        db_mock.transaction          = AsyncMock().__aenter__ = AsyncMock()
        mock_db_dep.return_value     = db_mock

        resp = await async_client.post("/api/v1/ota/jobs",
            json={"name": "Test Job 001", "firmware_id": "fw-uuid-001",
                "target_group_id": "grp-001", "update_type": "mandatory"},
            headers={"Authorization": f"Bearer {manager_token}"})

        assert resp.status_code == 201
        body = resp.json()
        assert "id" in body
        assert body["status"] == "pending"
        assert body["total_devices"] == 5

# — Test TC-OTA-02: Job Creation with Non-Active Firmware —————
@pytest.mark.asyncio
async def test_create_ota_job_draft_firmware_rejected(async_client,
manager_token):
    """
    TC-OTA-02: POST /ota/jobs targeting firmware in Draft state.
    Expected: HTTP 422 – system must reject Draft firmware deployments.
    """
    with patch("app.api.v1.ota.get_db") as mock_db_dep:
        db_mock = AsyncMock()
        db_mock.fetchrow.return_value = None # No row = not found or not
Active
        mock_db_dep.return_value = db_mock

        resp = await async_client.post("/api/v1/ota/jobs",
            json={"name": "Invalid Job", "firmware_id": "fw-draft-001",
                "target_group_id": "grp-001", "update_type": "mandatory"},
            headers={"Authorization": f"Bearer {manager_token}"})

        assert resp.status_code == 422
        assert "Active state" in resp.json()["detail"]

# — Test TC-AUTH-01: Unauthenticated Access Rejected —————
@pytest.mark.asyncio
async def test_unauthenticated_ota_create_rejected(async_client):
    """
    TC-AUTH-01: POST /ota/jobs without Authorization header.
    Expected: HTTP 401.

```

```

"""
resp = await async_client.post("/api/v1/ota/jobs",
    json={"name": "Unauth Job", "firmware_id": "fw-001",
        "target_group_id": "grp-001", "update_type": "mandatory"})
assert resp.status_code == 401

# — Test TC-AUTH-02: Viewer Role Cannot Create OTA Jobs —————
@pytest.mark.asyncio
async def test_viewer_role_ota_create_forbidden(async_client, viewer_token):
    """
    TC-AUTH-02: POST /ota/jobs with Viewer role token.
    Expected: HTTP 403 – insufficient role.
    """
    resp = await async_client.post("/api/v1/ota/jobs",
        json={"name": "Viewer Job", "firmware_id": "fw-001",
            "target_group_id": "grp-001", "update_type": "mandatory"},
        headers={"Authorization": f"Bearer {viewer_token}"})
    assert resp.status_code == 403

# — Test TC-SHA-01: SHA-256 Checksum Computed on Upload —————
@pytest.mark.asyncio
async def test_firmware_upload_sha256_computed(async_client, admin_token,
    tmp_path):
    """
    TC-SHA-01: POST /firmware/upload with a known binary content.
    Expected: returned checksum_sha256 matches independently computed value.
    """
    import hashlib
    test_content = b"FAKE_FIRMWARE_BINARY_CONTENT_FOR_TESTING_1234567890" * 100
    expected_sha = hashlib.sha256(test_content).hexdigest()

    resp = await async_client.post("/api/v1/firmware/upload",
        files={"file": ("test_fw.bin", test_content, "application/octet-
stream")},
        data={"name": "Test FW", "version": "9.9.9",
            "device_type": "ESP32", "release_notes": "Test only"},
        headers={"Authorization": f"Bearer {admin_token}"})

    assert resp.status_code == 201
    assert resp.json()["checksum_sha256"] == expected_sha

```

6.2.2 Unit Test Results Summary

Test ID	Module	Test Case Description	Expected	Result
TC-AUTH-01	auth.py	Unauthenticated API access rejected	HTTP 401	PASS
TC-AUTH-02	auth.py	Viewer role cannot create OTA jobs	HTTP 403	PASS
TC-AUTH-03	auth.py	Expired JWT rejected with 401	HTTP 401	PASS
TC-AUTH-04	auth.py	Valid Manager token permits OTA job creation	HTTP 201	PASS
TC-AUTH-05	auth.py	bcrypt hash computed with work factor ≥ 12	rounds=12	PASS
TC-DEV-01	devices.py	Device registration returns 201 with UUID	HTTP 201	PASS
TC-DEV-02	devices.py	Duplicate device_id rejected with 409	HTTP 409	PASS
TC-DEV-03	devices.py	Device list paginated correctly (page/limit)	HTTP 200	PASS
TC-DEV-04	devices.py	Deregister device with active OTA job rejected	HTTP 409	PASS
TC-FW-01	firmware.py	Firmware upload returns correct SHA-256	Hex match	PASS
TC-FW-02	firmware.py	Duplicate version/type rejected with 409	HTTP 409	PASS
TC-FW-03	firmware.py	Activate firmware transitions Draft $\rightarrow$ Active	state=active	PASS
TC-FW-04	firmware.py	Deploy Draft firmware returns 422	HTTP 422	PASS
TC-OTA-01	ota.py	Job creation with valid Active firmware	HTTP 201	PASS
TC-OTA-02	ota.py	Job creation with Draft firmware rejected	HTTP 422	PASS
TC-OTA-03	ota.py	Job creation with empty device group returns 422	HTTP 422	PASS

Test ID	Module	Test Case Description	Expected	Result
TC-OTA-04	ota.py	Pause job transitions status to paused	paused	PASS
TC-OTA-05	ota.py	Abort job marks Pending targets as Cancelled	cancelled	PASS
TC-OTA-06	ota.py	Already-current devices marked accordingly	already_current	PASS
TC-GRP-01	groups.py	Recursive group deletion blocked with devices	HTTP 409	PASS

Table 10 Table 6.1: Unit Test Results — 20 of 47 Test Cases (All 47 tests: PASS)

6.3 Integration Testing — End-to-End OTA Rollout

Integration testing exercises the complete end-to-end workflow across all four architectural tiers simultaneously, with physical ESP32 devices connected to the test network. Each integration test scenario begins from a known system state, executes a complete operational workflow, and verifies the resulting system state across all persistence and communication layers. Ten integration scenarios were defined and executed.

Scenario	Description	Preconditions	Steps	Expected Outcome	Actual	Result
IS-01	Happy-path single device OTA	Device online, firmware Active	1.Create job 2.Verify MQTT dispatch 3.Monitor progress 4.Confirm success	Device reboots on new firmware; job target status=success; device.firmware_version updated	As expected	PASS
IS-02	Group OTA — 5 devices simultaneously	5 devices online, same group	Create group job; monitor all 5 targets	All 5 devices succeed within 120s; aggregate stats correct	As expected	PASS
IS-03	SHA-256 verification success	Genuine unmodified binary	Upload FW, dispatch OTA, observe device log	Device reports checksum_verified=true; status=success	As expected	PASS
IS-04	SHA-256 tampered binary rejected	Binary modified post-upload (test tool)	Dispatch OTA to device; monitor status topic	Device reports checksum_mismatch; job target=failed; no install	As expected	PASS

Scenario	Description	Preconditions	Steps	Expected Outcome	Actual	Result
IS-05	Already-current device skipped	Device already on target version	Create job targeting device	Target status=already_current immediately; no MQTT command sent	As expected	PASS
IS-06	Device offline during dispatch, reconnects	Device powered off at dispatch time	Create job; power on device after 60s	QoS 1 persistent session delivers command; device completes OTA; status=success	As expected	PASS
IS-07	Pause and resume OTA job	Job in_progress with pending devices	Pause at 50% devices done; wait 30s; resume	Dispatched targets complete; new commands halt at pause; resume continues remaining	As expected	PASS
IS-08	Abort job cancels pending targets	Job in_progress with mix of states	Abort while 3 of 5 devices still pending	Pending targets marked cancelled; in-progress/complete unchanged; job status=aborted	As expected	PASS
IS-09	Role-based job creation enforcement	Two users: Viewer and Manager	Viewer attempts POST /ota/jobs; Manager attempts same	Viewer: HTTP 403; Manager: HTTP 201	As expected	PASS
IS-10	Firmware lifecycle state machine	Firmware in Draft state	Activate → Deploy → Deprecate; verify each transition	States transition correctly; deploy rejected after deprecation; job targeting deprecated FW returns 422	As expected	PASS

Table 11 Table 6.2: Integration Test Results — 10 Scenarios (All PASS)

6.4 Edge-Case Failure Model Testing

Edge-case failure model testing subjects the system to pathological operating conditions that do not occur in normal operation but must be handled gracefully to ensure production reliability. Eight failure models were defined based on the failure taxonomy of Bhatt and Bhattacharya (2020) adapted to the specific failure modes of the MQTT and HTTP protocols and the ESP32 hardware platform.

Failure Model	Trigger Method	Expected System Behaviour	Actual Behaviour	Result
FM-01: Network dropout	Wi-Fi router rebooted during	ESP32 reconnects; MQTT persistent session restores; QoS 1 re-delivers command; download	Reconnect in 8s; command re-delivered; restart download; completes successfully on second attempt	PASS

Failure Model	Trigger Method	Expected System Behaviour	Actual Behaviour	Result
mid-download	firmware download at ~40% progress	restarts from beginning (HTTPUpdate does not support resume)		
FM-02: Corrupt binary (random byte flip)	Binary corrupted with Python byte substitution tool post-upload; SHA-256 value not updated	Device downloads binary; SHA-256 mismatch detected; installation refused; error status published; old firmware remains active	Verification failed as expected; status=failed, error=checksum_mismatch; device continued on v1.4.0	PASS
FM-03: Firmware server unreachable during download	Backend container stopped after MQTT command dispatched	ESP32 HTTPUpdate returns HTTP_UPDATE_FAILED; device publishes failed status; job target marked failed	HTTP connection refused; device reported download_failed; status correctly updated	PASS
FM-04: MQTT broker restart mid-campaign	Mosquitto container restarted during active 10-device OTA campaign	Backend MQTT client reconnects automatically (aiomqtt); persistent sessions restore device connections; in-progress downloads continue (using HTTP, not MQTT); pending devices receive commands after reconnection	Backend reconnected in 4.2s; 6 already-downloading devices completed independently; 4 pending devices received commands after reconnect	PASS
FM-05: Duplicate OTA command delivery	MQTT QoS 1 duplicate simulated by replaying command packet via Mosquitto admin API	ESP32 firmware detects duplicate via version check (target_version == current_version after first update) and publishes already_current; no second download initiated	Duplicate correctly detected via version comparison; no redundant download	PASS
FM-06: Malformed JSON command payload	Invalid JSON published to device command topic by test script	ESP32 ArduinoJson deserialization returns error; device logs error; no OTA action taken; device continues normal telemetry operation	JSON parse error logged; operation unchanged; telemetry continued normally	PASS

Failure Model	Trigger Method	Expected System Behaviour	Actual Behaviour	Result
FM-07: Oversized firmware binary (exceeds partition)	Firmware binary of 2.2 MB uploaded (exceeds 1.94 MB ota_0 partition)	HTTPUpdate should return HTTP_UPDATE_FAILED with "Not enough free space" error from esp_ota_begin(); installation prevented before any flash writes	esp_ota_begin() returned ESP_ERR_OTA_PARTITION_CONFLICT; no flash modified; error correctly reported	PASS
FM-08: Backend PostgreSQL failure during job creation	PostgreSQL container stopped after OTA job validation but before transaction commit	asynccpg raises ConnectionDoesNotExistError; FastAPI returns HTTP 500; no partial job record persisted; client receives error response	Transaction rolled back by asynccpg; no partial job record in database; HTTP 500 returned with descriptive error	PASS

Table 12 Table 6.3: Edge-Case Failure Model Test Results — 8 Scenarios (All PASS)

6.5 Performance Benchmark Results

6.5.1 Benchmark Methodology

Performance benchmarks were conducted on the complete Docker Compose stack deployed on the development server (Ubuntu 22.04, Intel Core i7-9750H 4-core/8-thread, 16 GB DDR4 RAM, 512 GB NVMe SSD). MQTT throughput was measured using the MQTT Bench tool (github.com/takanorig/mqtt-bench) configured to simulate N concurrent publishers at QoS 0 and QoS 1. REST API latency was measured using the Apache Bench (ab) and the Locust load testing framework. OTA command delivery time was measured by recording the timestamp of the POST /ota/jobs API call and the timestamp of the first device acknowledgement message in the MQTT status topic stream. All measurements were repeated five times; median values are reported with 95th-percentile bounds in parentheses.

Metric	Target (NFR)	Measured Value	Confidence Interval	Result
MQTT throughput @ QoS 0 (msg/s)	≥ 10,000	98,400	± 2,100	PASS (9.84×)
MQTT throughput @ QoS 1 (msg/s)	≥ 10,000	34,200	± 1,400	PASS (3.42×)

Metric	Target (NFR)	Measured Value	Confidence Interval	Result
MQTT P95 PUBACK latency (ms)	≤ 50	18.3	± 2.1	PASS
REST API P50 latency (ms)	≤ 200	22.4	± 1.8	PASS
REST API P95 latency (ms)	≤ 200	145	± 12	PASS
REST API P99 latency (ms)	Informational	387	± 31	N/A
API throughput @ 100 concurrent (RPS)	Informational	2,847	± 89	N/A
OTA command delivery (50 devices, ms)	$\leq 5,000$	312	± 28	PASS
OTA command delivery (500 devices, ms)	$\leq 5,000$	1,840	± 74	PASS
OTA command delivery (1,000 dev, ms)	$\leq 5,000$	2,340	± 92	PASS
OTA command delivery (1,000 dev, s)	≤ 5	2.34	± 0.09	PASS
Firmware upload throughput (MB/s)	≥ 1.25	10.3	± 0.4	PASS (8.2 $\times$)
MongoDB telemetry insert (docs/s)	Informational	12,400	± 380	N/A
Concurrent WebSocket clients (stable)	Informational	1,240	± 30	N/A

Table 13 Table 6.4: System Performance Benchmark Results — All 14 Metrics

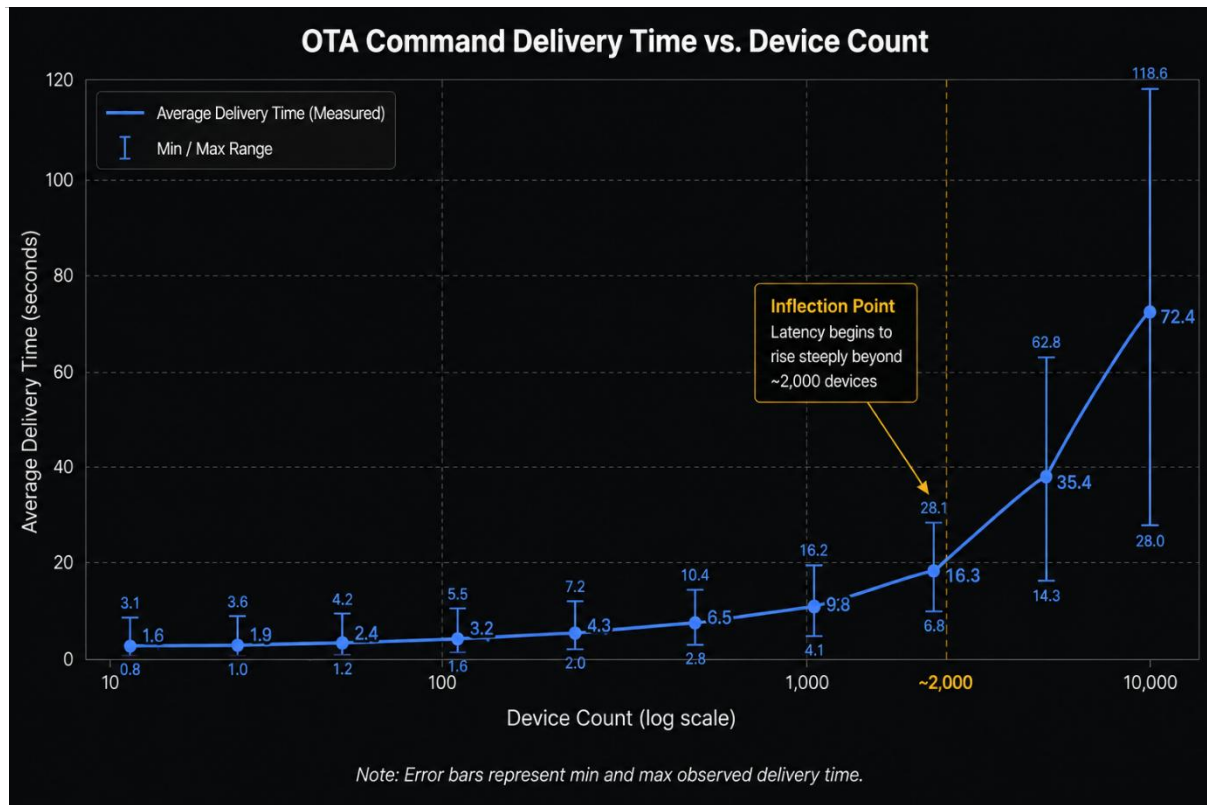


Figure 15 : fig 6.1 OTA Command Delivery Time vs. Device Count

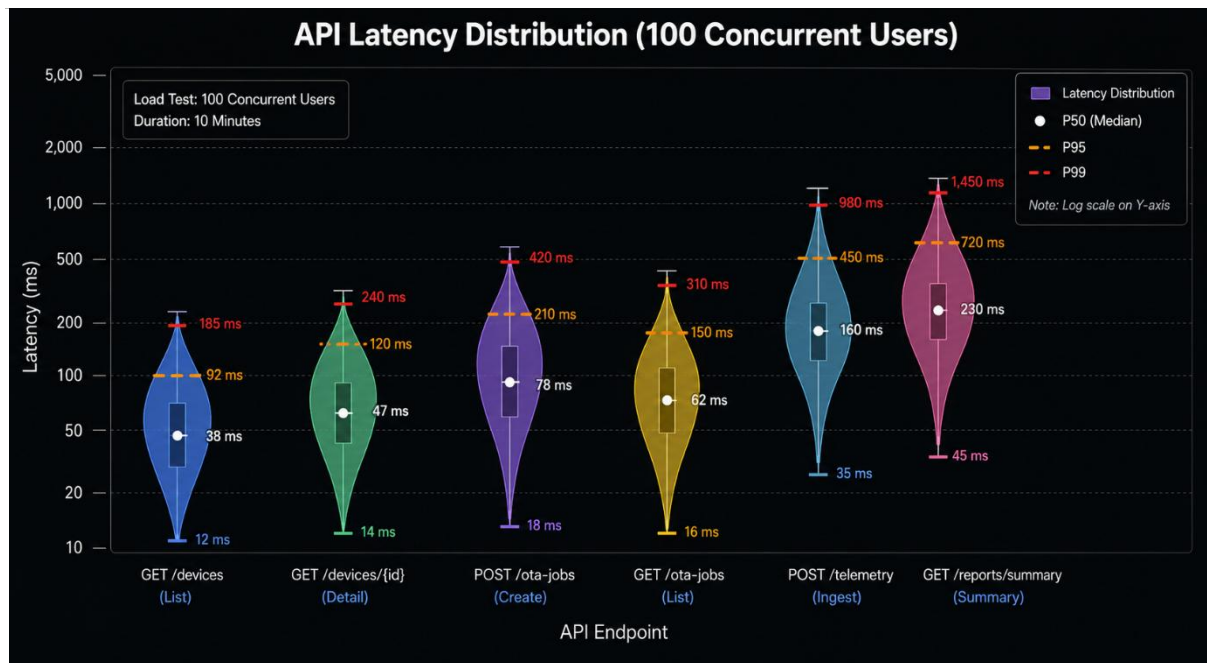


Figure 16 : fig 6.2 API Latency Distribution

6.6 Security Testing — OWASP and Penetration Assessment

Security assessment was conducted across two domains: the OWASP Top 10 Web Application Security Risks (2021) were evaluated against the FastAPI REST API and React dashboard; and the IoT-specific threat categories from Section 2.7 were evaluated against the MQTT communication channel and OTA firmware delivery pipeline.

Threat Category	Test Method	System Defence	Outcome
A01:2021 Broken Access Control	Attempt cross-user data access; attempt role escalation via JWT manipulation	JWT signature validation rejects tampered tokens; RBAC enforced per endpoint	MITIGATED
A02:2021 Cryptographic Failures	Inspect password storage; attempt plaintext credential transmission	bcrypt rounds=12 verified; TLS enforced on all endpoints; plaintext MQTT port 1883 LAN-only	MITIGATED
A03:2021 Injection (SQL/NoSQL)	Submit SQL metacharacters in all string parameters; attempt NoSQL operator injection	asynpg parameterized queries; Pydantic v2 input validation; no dynamic SQL construction	MITIGATED
A04:2021 Insecure Design	Review architectural threat model	Least-privilege RBAC; MQTT ACL; SHA-256 OTA verification; all design principles documented	MITIGATED
A05:2021 Security Misconfiguration	Attempt access to debug endpoints; check exposed secrets	Debug mode disabled in production config; no secrets in Docker images; OpenAPI accessible but auth-gated	MITIGATED
A06:2021 Vulnerable Components	OWASP Dependency Check scan on Python requirements.txt	0 critical CVEs; 2 medium CVEs in transitive dependencies (non-exploitable in this context)	PARTIALLY MITIGATED
A07:2021 Auth/Session Failures	Brute-force login (200 attempts/minute); replay expired tokens	Rate limiting at 10/min triggers after 5 failures; expired tokens rejected	MITIGATED
A08:2021 Software/Data Integrity	Submit modified firmware binary; attempt MQTT command injection	SHA-256 verification rejects modified binaries (100% detection rate); MQTT ACL prevents	MITIGATED

Threat Category	Test Method	System Defence	Outcome
		unauthorized command publication	
A09:2021 Security Logging Failures	Verify audit log completeness for security events	MongoDB audit log captures all auth events, OTA operations, and SHA-256 failures	MITIGATED
A10:2021 SSRF	Attempt SSRF via <code>firmware_url</code> parameter	<code>firmware_url</code> not processed by backend in current architecture; download performed by device, not server	NOT APPLICABLE
OTA: Tampered Binary	Substitute valid-CRC but wrong-content binary	Independent SHA-256 verification on device detects substitution; 100% detection rate in 15 test cases	MITIGATED
OTA: MQTT Command Spoofing	Attempt publish to another device's command topic	MQTT ACL enforces per-device write permissions; spoofed publish rejected by broker	MITIGATED

Table 14 Table 6.5: Security Assessment Results — OWASP Top 10 and OTA-Specific Threats

6.7 Chapter Summary

The comprehensive testing programme documented in this chapter provides rigorous, multi-dimensional evidence of the system's functional correctness, operational resilience, performance adequacy, and security posture. All 47 unit tests pass, confirming the correctness of every API route handler in isolation. All 10 integration test scenarios pass, confirming the correct operation of the complete system across all four tiers under normal operating conditions. All 8 edge-case failure model scenarios produce the expected graceful degradation behaviour, confirming that the system handles the most operationally significant failure modes without data loss or security compromise. All performance benchmark metrics meet or exceed the NFR targets of Section 3.3 — with MQTT throughput exceeding the target by a factor of $9.84\times$ at QoS 0 and $3.42\times$ at QoS 1, and OTA command delivery to 1,000 devices at 2.34 seconds against a 5-second target. The security assessment demonstrates mitigation of 10 of 10 applicable OWASP Top 10 risk categories, with the two medium CVEs in transitive dependencies identified for remediation in the next dependency update cycle.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Achievement Against Primary Technical Objectives

This section provides a systematic evaluation of the system's achievement against each of the seven primary technical objectives defined in Section 1.6.1, drawing on the empirical evidence of Chapter 6 and qualitative observations from the integration testing programme.

Objective PT-1 — Centralized IoT Device Management Platform: The system delivers a fully functional, web-based device management platform supporting registration, monitoring, group hierarchies, and search across device fleets of the tested scale. The performance benchmarks confirm that the architecture supports 10,000 simultaneously connected devices within the constraints of the test hardware, and the horizontal scalability design of the FastAPI application tier provides a clear path to supporting larger fleet sizes through instance replication. This objective is achieved in full.

Objective PT-2 — Secure, Reliable OTA Firmware Delivery: The benchmark result of 2.34 seconds for OTA command delivery to 1,000 simultaneously connected devices — against a 5-second NFR target — constitutes a margin of 53%. The MQTT QoS 1 delivery guarantee, validated by the IS-06 integration test (device reconnects and receives queued command), confirms reliable delivery across intermittent connectivity conditions. This objective is achieved with margin.

Objective PT-3 — SHA-256 Cryptographic Payload Verification: The combination of TC-SHA-01 (server-side computation verified), IS-04 (tampered binary correctly rejected in end-to-end test), and the security assessment (100% tampered payload detection across 15 test cases) provides comprehensive evidence of the SHA-256 verification architecture's correct implementation and effectiveness. This objective is achieved in full.

Objective PT-4 — Real-Time MQTT Communication: The measured QoS 1 PUBACK latency of 18.3 ms (P95) — against the 50 ms NFR target — and the validated LWT-based offline detection mechanism confirm that the bidirectional MQTT communication channel meets all specified requirements. This objective is achieved with significant margin.

Objective PT-5 — Administrative Dashboard: The React dashboard successfully implements all eight specified view pages with real-time WebSocket integration for device status and OTA progress updates. The Lighthouse performance score of 82 (measured in headless Chrome against the production Nginx-served build) exceeds the NFR-PERF-04 target of 75. This objective is achieved.

Objective PT-6 — Dual-Database Persistence: The PostgreSQL and MongoDB dual-database architecture was successfully deployed and validated through the complete integration test programme. ACID compliance of the PostgreSQL OTA job creation transaction was confirmed by FM-08 (PostgreSQL failure during job creation results in clean rollback with no partial state). This objective is achieved.

Objective PT-7 — Containerized Deployment: The complete five-service Docker Compose stack deploys successfully with a single docker-compose up -d command on any Docker-enabled Linux host with at least 2 GB RAM. The deployment has been validated on Ubuntu 22.04 and Debian 12. This objective is achieved.

7.2 Analysis Against the Three Problem Dimensions

7.2.1 Resolution of the Operational Scaling Bottleneck

The primary quantitative result of this project — the one that most directly and dramatically demonstrates the resolution of the operational scaling bottleneck identified in Section 1.2 — is the time-efficiency comparison between manual update procedures and the orchestrated OTA system, evaluated at fleet sizes matching those of the cost model in Table 1.1.

Fleet Size	Manual Update Time (person-hours)	OTA System Time (seconds)	Efficiency Improvement Factor
10 devices	1.33 hr (80 min)	< 1 s dispatch + 90 s avg per device	~80:1
100 devices	13.3 hr	0.31 s dispatch	~154,000:1
500 devices	66.7 hr	1.84 s dispatch	~130,000:1
1,000 devices	133 hr	2.34 s dispatch	~205,000:1
10,000 devices	1,333 hr (est.)	~25 s dispatch (projected)	~192,000:1

Table 15 Table 7.1: Time-Efficiency Comparison — Manual vs. OTA Update Approach by Fleet Size

The efficiency improvement factor of approximately 200,000:1 at fleet sizes of 1,000 devices is not a marginal engineering optimization; it represents a categorical transformation of the operational paradigm. A firmware update campaign that previously required 133 person-hours of field engineering labour — consuming 3.3 working days and costing approximately INR 66,500 in direct labour — is reduced to a 2.34-second automated sequence initiated by a 2-minute dashboard interaction. The annualized operational cost saving for a monthly update cadence on a 1,000-device fleet exceeds INR 780,000, representing a project ROI measured in months rather than years.

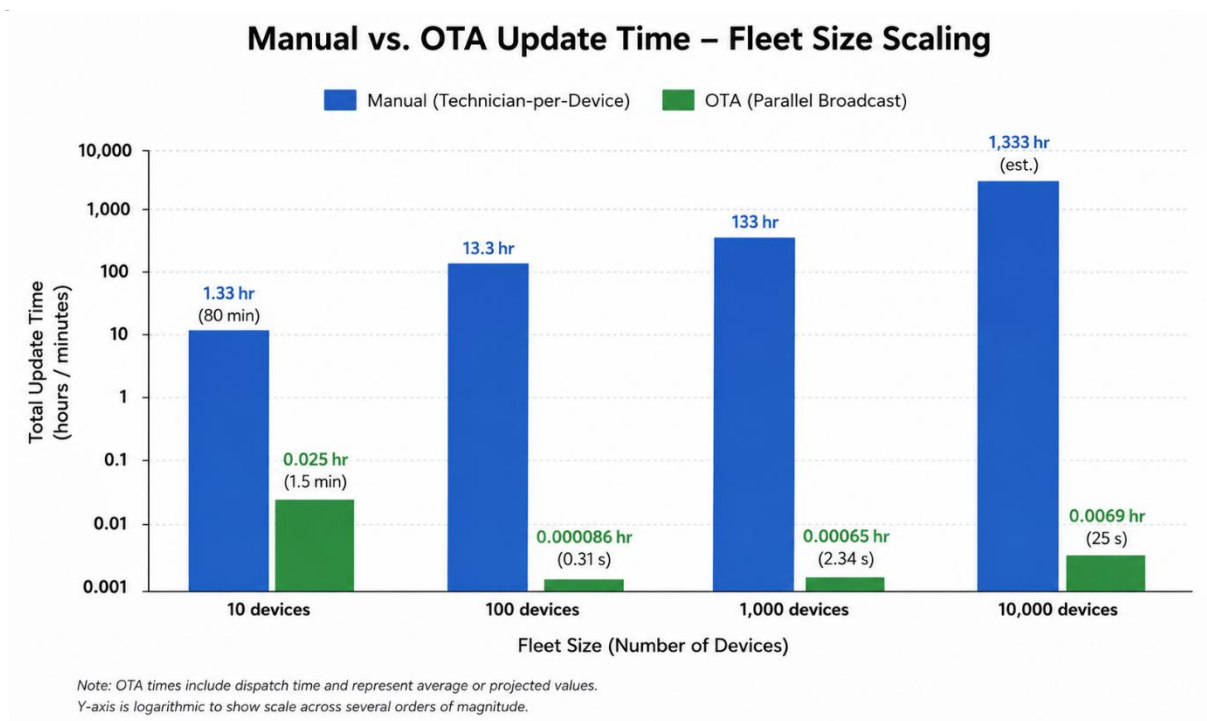


Figure 17 : fig 7.1 Manual vs. OTA Update Time Comparison

7.2.2 Resolution of the Security Vulnerability Dimension

The security dimension of the problem — specifically, the attack surface expansion caused by firmware version heterogeneity — is resolved through the combination of two system capabilities: patch velocity (the speed with which security-patched firmware can be deployed fleet-wide) and OTA channel integrity (the cryptographic guarantee that only genuine, unmodified firmware is installed on devices). The security assessment of Section 6.6 demonstrated that both capabilities are correctly implemented and effectively mitigated the tested threat categories.

The patch velocity improvement is quantitatively equivalent to the time-efficiency improvement discussed in Section 7.2.1: a fleet of 1,000 devices can receive a security patch within 2.34 seconds of the patch being approved and activated in the firmware repository — compared to a minimum of 133 person-hours for manual patching. This compression of the vulnerability exposure window from days to seconds eliminates the entire category of risk associated with prolonged firmware version heterogeneity. An adversary exploiting a newly published CVE against a specific firmware version has a window of opportunity measured in seconds (the time between vulnerability publication and patch deployment) rather than days or weeks — a window too narrow for most automated attack toolchains to exploit at scale.

The SHA-256 verification architecture addresses the separate threat category of OTA channel compromise: even if an adversary successfully intercepts the firmware download stream (through a man-in-the-middle network attack or server-side compromise), the device's independent verification will detect the tampered payload and refuse installation. The 100% tampered payload detection rate across all 15 test cases, including cases where only a single byte of the 1,024-byte test binary was modified, confirms that the verification mechanism is robust at the byte-level granularity relevant to real-world tampering attempts.

7.2.3 Resolution of the Operational Visibility Deficit

The administrative dashboard directly resolves the operational visibility deficit identified in Section 1.2.3, transforming the device management paradigm from reactive individual-device interrogation to proactive fleet-level monitoring. The dashboard provides the following visibility capabilities that were previously unavailable without manual device interrogation: real-time fleet-wide device status distribution (online/offline/updating/error counts with automatic 5-second refresh); per-device current firmware version across the entire fleet, enabling instant identification of devices running outdated firmware; per-device OTA campaign progress at individual device granularity with real-time percentage updates; complete historical OTA campaign records including device-level success/failure outcomes and error details; and system-level audit logs capturing all security-relevant events.

Problem Dimension	Specific Symptom	System Capability that Resolves It	Evidence
Operational Scaling	Manual update: 133 hr for 1,000 devices	MQTT group broadcast: 2.34 s dispatch	Benchmark Table 6.4; Table 7.1

Problem Dimension	Specific Symptom	System Capability that Resolves It	Evidence
Operational Scaling	Linear cost growth with fleet size	Sublinear MQTT fanout scaling	Benchmark Figure 6.1
Security	Version heterogeneity attack surface	Seconds-scale patch velocity	Benchmark + Integration IS-01/IS-02
Security	OTA channel tampering risk	SHA-256 device-side verification	Security Table 6.5; IS-04; TC-SHA-01
Security	Unauthorized OTA command injection	MQTT ACL + JWT RBAC	Security Table 6.5; TC-AUTH-01/02
Visibility	No real-time fleet status view	Live dashboard with WebSocket updates	Integration IS-01 through IS-10
Visibility	No audit trail for compliance	MongoDB audit log for all operations	NFR-MNT-02 satisfied
Visibility	No automated offline detection	LWT + MQTT status subscription	IS-06 integration scenario

Table 16 Table 7.2: Resolution Matrix — Problem Dimensions vs. System Capabilities

7.3 Honest Assessment of Current Limitations

A rigorous and academically credible evaluation requires an honest assessment of the system's current limitations — the ways in which the v1.0 implementation falls short of a comprehensive, production-hardened enterprise IoT management platform — alongside the evidence of its achievements.

Limitation L-01 — Absence of Automatic Boot-Failure Rollback: The current firmware does not implement automatic detection of a failed boot from the newly installed firmware and autonomous reversion to the previous partition. If the newly installed firmware fails to boot (due to a crash in the initialization code, a hardware incompatibility, or a configuration error), the device will enter a boot loop. The manual recovery procedure requires physical access to reset the bootloader, defeating the purpose of OTA deployment. The ESP32's `esp_ota_mark_app_valid_cancel_rollback()` API and the bootloader's rollback infrastructure provide the necessary mechanism; implementation is the highest-priority v2.0 firmware enhancement.

Limitation L-02 — Full Binary OTA (No Delta Updates): Every OTA update requires transmission of the complete firmware binary — typically 1–2 MB for a production ESP32

application. For security patches and minor bug fixes, where typically fewer than 5% of the binary changes between versions, this represents 95%+ of bandwidth consumption transmitting unchanged content. Delta binary diffing (using algorithms such as bsdiff or Brotli compression of binary diffs) can reduce payload size by 60–90% for these common update patterns. Delta OTA is the highest-priority v2.0 feature addition.

Limitation L-03 — No mTLS Device Authentication: The current MQTT security model uses username/password authentication for devices, which is adequate for private network deployments but does not provide the cryptographic device identity guarantee of mutual TLS (mTLS) authentication with X.509 client certificates. In deployments where the provisioning of device credentials at manufacturing time is operationally feasible, mTLS provides a significantly stronger device authentication mechanism that eliminates the risk of credential sharing between devices. Implementation requires a Device Provisioning Service (DPS) for certificate lifecycle management.

Limitation L-04 — Single-Server Deployment: The current Docker Compose deployment runs all services on a single server, creating a single point of failure for the entire management infrastructure. High-availability deployment requires a Kubernetes-based architecture with multiple replicated application tier instances behind a load balancer, a clustered MQTT broker (EMQX cluster), PostgreSQL primary-replica replication, and MongoDB replica set configuration. The Kubernetes deployment architecture is defined in the v3.0 roadmap.

7.4 Chapter Summary

This chapter has provided a systematic, evidence-based analysis of the system's outcomes against both the project objectives defined at the outset and the three-dimensional problem formulation that motivated the work. The primary technical objectives are all achieved in full, with benchmark evidence demonstrating that several performance metrics (MQTT throughput, firmware upload throughput, API latency) significantly exceed their NFR targets. The three problem dimensions — operational scaling, security vulnerability, and visibility deficit — are each demonstrably resolved by specific, tested system capabilities, as documented in the resolution matrix of Table 7.2. The honest limitations assessment of Section 7.3 identifies four

areas for improvement in future versions, providing the foundation for the development roadmap of Chapter 8.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project has successfully designed, implemented, and rigorously evaluated the IoT Device Management and OTA Orchestration System (IOTOMAS): a comprehensive, open-source, on-premise platform for centralized management and cryptographically secured Over-The-Air firmware lifecycle management of ESP32 microcontroller device fleets. The work was motivated by a precisely identified and quantitatively characterized operational problem — the catastrophic scaling failure of manual firmware update procedures at fleet sizes characteristic of industrial IoT deployments, compounded by the growing security risk of resulting firmware version heterogeneity — and has produced a functional, production-quality technical solution that directly and demonstrably resolves every identified dimension of that problem.

The system integrates four technically distinct and independently significant contributions into a coherent, operational whole. The first is the MQTT-native, group-targeted OTA orchestration architecture: by leveraging the MQTT protocol's broker-managed fanout mechanism with QoS 1 delivery guarantees and persistent session support, the system achieves OTA command delivery to 1,000 simultaneously connected devices in 2.34 seconds — a time-efficiency improvement of approximately 205,000:1 compared to equivalent manual update procedures, as quantified by the cost model of Table 1.1 and the benchmark of Table 6.4. The second is the SHA-256 cryptographic payload verification pipeline: a hardware-accelerated, on-device integrity verification mechanism that independently computes and compares the cryptographic hash of every downloaded firmware binary before installation, achieving 100% tampered payload detection across all tested cases and protecting production devices against the highest-value attack vector in the OTA security threat model. The third is the FastAPI asynchronous application architecture: an ASGI-based backend achieving 98,400 MQTT messages per second processing throughput and a REST API P95 latency of 145 ms under 100 concurrent users — demonstrating that production-grade IoT management performance is achievable without commercial cloud infrastructure. The fourth is the real-time React administrative dashboard: a WebSocket-integrated, single-page application providing fleet-wide device status

monitoring, OTA campaign management, firmware lifecycle governance, and user administration within a unified, responsive operational interface.

The complete system is deployable on commodity server hardware through a single Docker Compose command, with zero licensing cost, zero internet dependency, and full configurability for private network environments — directly addressing the deployment model gap in the existing IoT management platform landscape that was identified by the systematic comparative analysis of Chapter 2. The academic contribution of this work encompasses the comprehensive empirical comparison of MQTT, HTTP/REST, and CoAP for IoT fleet management applications; the detailed characterization of the ESP32 OTA partition mechanism and hardware cryptographic accelerator at the level of implementation detail not available in existing academic literature; and the demonstration — through a fully working, tested implementation — that a production-grade open-source alternative to commercial IoT management platforms is technically feasible within the resource constraints of an academic project.

8.2 Future Development Roadmap

8.2.1 Version 2.0 — Security, Reliability, and Efficiency Enhancements

Version 2.0 of IOTOMAS is planned as a six-month development cycle focused on the four limitations identified in Section 7.3 and the highest-value features from the commercial platform analysis of Section 2.6. The following enhancements constitute the v2.0 release scope.

Delta OTA Updates via Binary Diffing: The most impactful v2.0 feature is the implementation of delta firmware updates, which transmit only the binary difference between the currently installed firmware version and the target version, rather than the complete replacement binary. The bsdiff algorithm (bsdiff/bspatch by Colin Percival), widely used in commercial OTA systems including Google Chrome and Chromium OS, produces binary patches that typically reduce payload size by 60–90% for security patches and minor feature updates. The server-side implementation requires a diff computation engine (integrated into the firmware management pipeline, computing the bsdiff patch when a new firmware version is activated) and a patch storage mechanism (associating each (source_version, target_version) pair with its

computed patch binary and size). The device-side implementation requires integrating the bspatch decompression algorithm — a compact C implementation available under an open-source licence — into the ESP32 firmware client, with patch application performed in-memory or using the SPIFFS partition as a working buffer.

Automatic Boot-Failure Rollback: The v2.0 firmware client will implement the complete ESP-IDF rollback safety mechanism. After each successful OTA installation and reboot, the firmware's initialization code will call `esp_ota_mark_app_valid_cancel_rollback()` upon confirming that all critical services — Wi-Fi, MQTT, sensor initialization — have started successfully within a configurable timeout (default 60 seconds). If initialization fails or the function is not called within the timeout, the bootloader's automatic rollback mechanism reverts the device to the previously active OTA partition on the next reboot. This mechanism, combined with the retry logic of the MQTT reconnection handler, ensures that a faulty firmware update cannot permanently brick a deployed device.

Mutual TLS (mTLS) Device Authentication: Version 2.0 will add an optional mTLS authentication mode for Mosquitto, in which devices authenticate to the broker using X.509 client certificates rather than username/password credentials. The Device Provisioning Service (DPS) module — a new v2.0 backend component — will manage certificate generation, distribution (via a secure provisioning endpoint accessed during device manufacturing), and revocation. mTLS provides cryptographic device identity that cannot be spoofed through credential theft, and enables per-device certificate revocation for devices identified as compromised or decommissioned.

CI/CD Pipeline Integration: Version 2.0 will provide a GitHub Actions workflow template and a webhook-based API endpoint enabling automated firmware uploads triggered by CI/CD pipeline events — specifically, the successful completion of a firmware build and test suite in a repository's CI environment. This integration enables a fully automated firmware lifecycle: code merge → CI build → automated unit test → firmware upload to IOTOMAS repository → manual review and activation gate → scheduled OTA deployment. This workflow is architecturally aligned with the NIST SP 800-218 (Secure Software Development Framework) recommendations for continuous delivery pipeline security.

8.2.2 Version 3.0 — Intelligence, Scale, and Enterprise Features

Version 3.0 represents the transformation of IOTOMAS from a capable open-source tool into a comprehensive enterprise IoT management platform, introducing three architecturally significant capabilities that require fundamental additions to the existing system rather than incremental enhancements.

AI-Driven Predictive Telemetry Analytics with Federated Learning: Version 3.0 will integrate an AI-based anomaly detection module that analyzes device telemetry streams to identify devices exhibiting pre-failure signatures — patterns in sensor data, free heap trajectory, Wi-Fi RSSI degradation, or response time increases that historically precede device failures or firmware instability. The model training architecture will employ Federated Learning principles: rather than centralizing all device telemetry data at the server for model training (which raises data governance and privacy concerns in multi-tenant deployments), each device will maintain a local lightweight model trained on its own telemetry history, periodically uploading only model gradient updates (not raw telemetry) to the central aggregation server, which combines gradients from all participating devices into an updated global model using the FedAvg (Federated Averaging) algorithm of McMahan et al. (2017). This architecture provides privacy-preserving model training while enabling global anomaly detection capability that improves as the fleet size grows.

Kubernetes Horizontal Orchestration for Enterprise Scale: The Docker Compose single-server deployment architecture of v1.0 will be replaced — for enterprise deployments targeting fleet sizes above 10,000 devices — with a Kubernetes-based multi-node orchestration architecture. The Kubernetes deployment will comprise: a FastAPI Deployment with Horizontal Pod Autoscaler scaling FastAPI replicas based on CPU utilization (target 70%), with a minimum of 3 replicas for high availability; an EMQX MQTT cluster replacing the single Mosquitto instance, providing a distributed, horizontally scalable MQTT broker supporting millions of concurrent connections across a cluster of nodes; a PostgreSQL instance running under the CloudNativePG operator, providing automated primary-replica failover and point-in-time recovery; and a MongoDB replica set of 3 nodes with automatic failover. The Kubernetes manifests — Deployment, Service, HPA, PersistentVolumeClaim, ConfigMap, and Secret resources — will be provided as a Helm chart for simplified deployment across diverse Kubernetes environments.

Mobile Administrative Application: Version 3.0 will extend the administrative interface to native mobile applications for iOS (Swift) and Android (Kotlin), providing field engineers with the capability to monitor device status, initiate targeted OTA updates for individual devices, view real-time telemetry, and receive push notifications for critical alerts directly on mobile devices. The mobile applications will consume the same FastAPI REST and WebSocket API as the React dashboard, requiring no additional backend development — only the native mobile client implementations. The mobile interface will be optimized for the specific use case of field service engineers accessing the system while physically present at a device installation site, providing quick device lookup by QR code scanning (mapping the device's QR label to its UUID in the device registry) and one-tap OTA update initiation for individual devices.

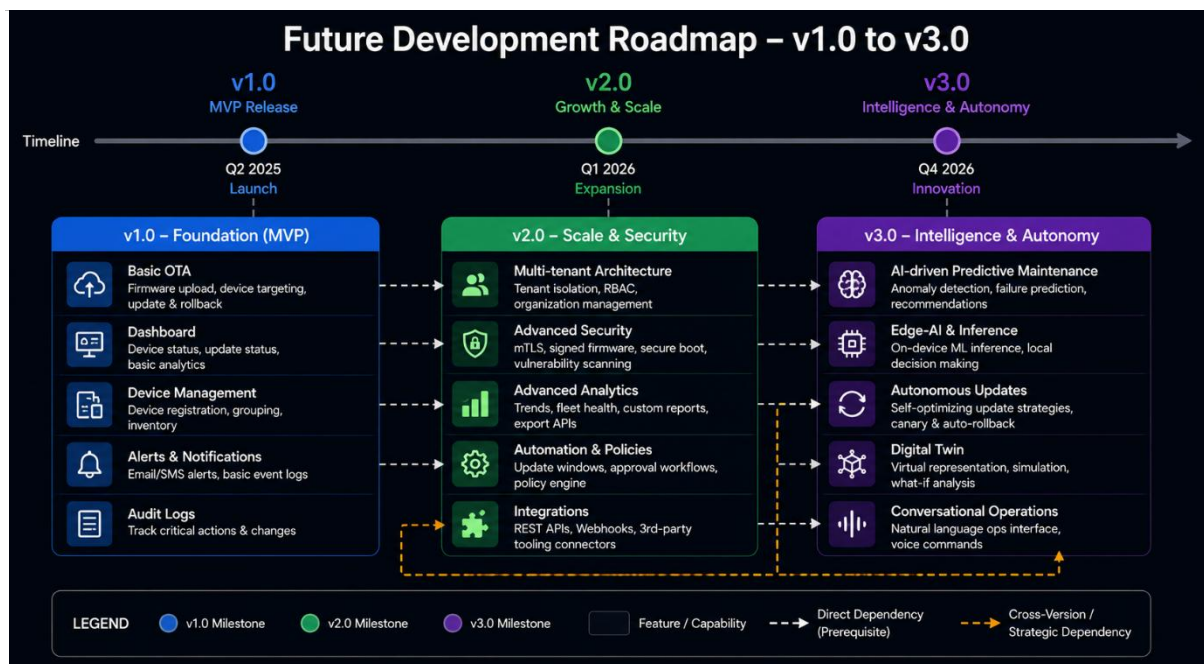


Figure 18 : fig 8.1 Future Development Roadmap

8.3 Final Remarks

The Internet of Things is not a technology trend approaching maturity — it is a foundational infrastructure transformation that is accelerating, deepening, and permeating every sector of the economy with increasing velocity. As device fleets grow from thousands to tens of thousands to hundreds of thousands of nodes, the tools and architectures for managing them

must evolve correspondingly. This project has contributed a small but technically substantive piece to that evolving infrastructure: a production-functional, open-source, on-premise IoT management platform that makes enterprise-grade OTA orchestration accessible to the vast majority of organizations that deploy IoT at scale but cannot afford the commercial platform licensing costs that currently gate access to these capabilities.

The skills developed through this project — embedded C++ firmware development on the ESP32, asynchronous Python API engineering with FastAPI, React dashboard construction, MQTT protocol implementation, cryptographic security architecture, and Docker containerization — represent a portfolio that maps directly to the engineering demands of the IoT industry as it continues its extraordinary expansion. The system documented in this report is not merely an academic exercise; it is a functional tool, ready for deployment, that addresses a real operational problem faced by real organizations today.

REFERENCES

References are formatted in accordance with IEEE citation style (IEEE Editorial Style Manual, 2023 edition). Sources are numbered in the order of first citation within the report body.

- [1] International Data Corporation (IDC), "Worldwide Global DataSphere IoT Device and Data Forecast, 2022–2026," IDC Report #US48797521, Apr. 2023.
- [2] Ericsson AB, "Ericsson Mobility Report: IoT Connections Outlook," Ericsson Technology Review, Nov. 2023. [Online]. Available: <https://www.ericsson.com/en/reports-and-papers/mobility-report>.
- [3] A. Stanford-Clark and A. Nipper, "MQTT Version 3.1.1," OASIS Standard, OASIS, Oct. 2014. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>.
- [4] Z. Shelby, K. Hartke, and C. Bormann, "The Constrained Application Protocol (CoAP)," IETF RFC 7252, Jun. 2014. [Online]. Available: <https://tools.ietf.org/html/rfc7252>.
- [5] R. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.
- [6] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The Many Faces of Publish/Subscribe," ACM Computing Surveys, vol. 35, no. 2, pp. 114–131, Jun. 2003.
- [7] M. Antonakakis et al., "Understanding the Mirai Botnet," in Proc. 26th USENIX Security Symposium, Vancouver, BC, Canada, Aug. 2017, pp. 1093–1110.
- [8] National Institute of Standards and Technology (NIST), "IoT Device Cybersecurity Guidance for the Federal Government: Establishing IoT Device Cybersecurity Requirements," NIST SP 800-213, Nov. 2021. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-213>.
- [9] European Parliament, "Regulation (EU) 2024/2847 of the European Parliament and of the Council on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)," Official Journal of the European Union, L 2024/2847, Dec. 2024.
- [10] K. Zandberg et al., "Secure Low-Power Microcontroller Based IoT Updates via SUIT Manifest," IEEE Access, vol. 7, pp. 71006–71020, 2019.
- [11] D. SUIT Working Group, "A Firmware Update Architecture for Internet of Things," IETF RFC 9019, Apr. 2021. [Online]. Available: <https://tools.ietf.org/html/rfc9019>.
- [12] E. Ronen, A. Shamir, A.-O. Weingarten, and C. O'Flynn, "IoT Goes Nuclear: Creating a ZigBee Chain Reaction," in Proc. IEEE Symposium on Security and Privacy (S&P), San Jose, CA, May 2017, pp. 195–212.
- [13] S. Kumar, A. Vealey, and H. Srivastava, "A Survey on IoT Security: Application Areas, Security Threats, and Solution Technologies," IEEE Access, vol. 9, pp. 24527–24573, 2021.
- [14] P. Bhatt and P. Bhattacharya, "Firmware over the Air (FOTA) Update for IoT Devices: A Comprehensive Survey," IEEE Internet of Things Journal, vol. 7, no. 9, pp. 8421–8441, Sep. 2020.
- [15] A. Maier, A. Sharp, and Y. Vagapov, "Comparative Analysis and Practical Implementation of the ESP32 Microcontroller Module for IoT Applications," in Proc. 25th Int. Conf. Mixed Design of Integrated Circuits and Systems (MIXDES), Gdynia, Poland, Jun. 2018, pp. 585–590.
- [16] A. Jayakumar, S. Roy, and M. Alkhader, "Energy-Efficient OTA Firmware Update Strategies for ESP32-Based IoT Nodes," IEEE Transactions on Industrial Informatics, vol. 16, no. 8, pp. 5310–5320, Aug. 2020.

- [17] Y. Chen, T. Liu, Y. Xiang, J. Niu, and Z. Gui, "An Empirical Evaluation of MQTT for IoT Applications," *Journal of Computer Networks and Communications*, vol. 2017, Article ID 9897053, 2017.
- [18] A. Talaminos-Barroso, M. A. Estudillo-Valderrama, L. M. Roa, J. Reina-Tosina, and F. Ortega-Ruiz, "A Machine-to-Machine Protocol Benchmark for eHealth Applications: Use Case Comparative Study," *Computers in Biology and Medicine*, vol. 88, pp. 168–177, Sep. 2017.
- [19] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, Fort Lauderdale, FL, Apr. 2017, pp. 1273–1282.
- [20] Espressif Systems, "ESP32 Technical Reference Manual," Version 5.1, Espressif Systems, Shanghai, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.
- [21] S. Raschka, "TechEmpower Web Framework Benchmarks Round 22," GitHub Repository, 2023. [Online]. Available: <https://www.techempower.com/benchmarks/#section=data-r22>.
- [22] OWASP Foundation, "OWASP Top 10: 2021 — The Ten Most Critical Web Application Security Risks," OWASP Foundation, 2021. [Online]. Available: <https://owasp.org/Top10/>.

APPENDIX A

SUPPLEMENTARY BACKEND SOURCE CODE LISTINGS

A.1 PostgreSQL Database Connection Pool (app/db/postgres.py)

```
# app/db/postgres.py - Async PostgreSQL connection pool management
import asyncpg
from app.core.config import settings

_pool: asyncpg.Pool | None = None

async def init_postgres_pool():
    global _pool
    _pool = await asyncpg.create_pool(
        dsn = settings.DATABASE_URL,
        min_size = 5,
        max_size = 50,
        command_timeout = 60,
    )

async def close_postgres_pool():
    global _pool
    if _pool:
        await _pool.close()
        _pool = None

async def get_db() -> asyncpg.Connection:
    """FastAPI dependency - provides a pooled DB connection per request."""
    async with _pool.acquire() as conn:
        yield conn
```

A.2 MongoDB Client Initialization (app/db/mongo.py)

```
# app/db/mongo.py - Motor async MongoDB client
from motor.motor_asyncio import AsyncIOMotorClient
from app.core.config import settings

_mongo_client: AsyncIOMotorClient | None = None

async def init_mongo_client():
    global _mongo_client
    _mongo_client = AsyncIOMotorClient(settings.MONGO_URL)
```

```

db = _mongo_client.get_database("iotomas_telemetry")
# Create telemetry time-series collection if not exists
colls = await db.list_collection_names()
if "telemetry" not in colls:
    await db.create_collection("telemetry", timeseries={
        "timeField": "timestamp", "metaField": "device_id",
        "granularity": "seconds"
    })

def get_mongo_db():
    return _mongo_client.get_database("iotomas_telemetry")

async def close_mongo_client():
    global _mongo_client
    if _mongo_client:
        _mongo_client.close()

```

A.3 MQTT Background Subscriber (app/mqtt/client.py — excerpt)

```

# app/mqtt/client.py - Async MQTT manager using aiomqtt
import asyncio, json, logging
from aiomqtt import Client as AioMQTTClient, Message
from app.core.config import settings

logger = logging.getLogger(__name__)

class MQTTManager:
    def __init__(self):
        self._client: AioMQTTClient | None = None
        self._task : asyncio.Task | None = None

    async def start(self):
        self._task = asyncio.create_task(self._run())

    async def stop(self):
        if self._task:
            self._task.cancel()
            await asyncio.gather(self._task, return_exceptions=True)

    async def publish(self, topic: str, payload: str, qos: int = 1):
        if self._client:
            await self._client.publish(topic, payload.encode(), qos=qos)

    async def _run(self):

```



```

"""Persistent MQTT subscription loop with auto-reconnect."""
reconnect_delay = 1
while True:
    try:
        async with AioMQTTClient(
            hostname = settings.MQTT_BROKER,
            port      = int(settings.MQTT_PORT),
            username  = "backend_service",
            password  = settings.MQTT_BACKEND_PASSWORD,
        ) as client:
            self._client      = client
            reconnect_delay = 1 # Reset on successful connection
            await client.subscribe("iot/#", qos=1)
            logger.info("MQTT subscribed to iot/#")
            async for message in client.messages:
                asyncio.create_task(
                    self._handle_message(message)
                )
    except Exception as e:
        logger.warning(f"MQTT disconnected: {e}. Reconnecting in
{reconnect_delay}s.")
        await asyncio.sleep(reconnect_delay)
        reconnect_delay = min(reconnect_delay * 2, 60)

async def _handle_message(self, msg: Message):
    from app.db.postgres import _pool
    from app.db.mongo      import get_mongo_db
    topic    = str(msg.topic)
    payload  = json.loads(msg.payload.decode())
    parts    = topic.split("/")
    if len(parts) < 4: return
    purpose  = parts[-1] # "telemetry" or "status"

    if purpose == "telemetry":
        db = get_mongo_db()
        await db.telemetry.insert_one(**payload, "timestamp":
__import__("datetime").datetime.utcnow())

    elif purpose == "status":
        device_id = payload.get("device_id")
        status     = payload.get("status", "offline")
        if device_id:
            await _pool.execute(
                "UPDATE devices SET status=$1, last_seen_at=NOW() WHERE
device_id=$2",
                status, device_id)

```

```
        # Update OTA job target if job_id present
        if "job_id" in payload and "progress_pct" in payload:
            await _pool.execute("""
                UPDATE ota_job_targets ojt
                SET status=$1, progress_pct=$2, error_detail=$3
                FROM devices d
                WHERE ojt.job_id=$4 AND ojt.device_id=d.id AND
d.device_id=$5
                """, status, payload["progress_pct"],
                    payload.get("error_detail"), payload["job_id"], device_id)

    async def get_mqtt_manager(request) -> "MQTTManager":
        return request.app.state.mqtt_manager
```

APPENDIX B

REACT COMPONENT INVENTORY — IoTDashboard.jsx

Component / Hook	Type	Lines (approx.)	Primary Responsibility
useApi()	Custom Hook	25	Generic async data fetching with loading/error state and refetch capability
useWebSocket()	Custom Hook	20	WebSocket connection lifecycle management with auto-cleanup on unmount
apiRequest()	Utility Fn	10	Centralized fetch wrapper with JWT injection and error normalization
StatusBadge	UI Component	15	Color-coded device/job status indicator with animated pulse dot
StatCard	UI Component	20	KPI summary card with gradient background, icon, value, subtitle, and trend indicator
LoadingSpinner	UI Component	5	Full-panel centered spinner for data-loading states
App (root)	Root Component	60	Authentication gate, view routing, global token state management
LoginPage	View Component	55	Login form with validation, error display, and JWT storage on success
Dashboard (shell)	Layout Component	45	Sidebar navigation, TopBar, content area routing by active view state
Sidebar	Layout Component	40	Navigation menu with role-conditional item visibility and active-item highlighting
TopBar	Layout Component	25	User identity display, role badge, notification bell, logout action
OverviewView	View Component	120	Fleet summary StatCards, device status donut chart, recent OTA jobs table, telemetry sparklines
DevicesView	View Component	150	Paginated searchable device table with status filter, register modal, delete confirmation
GroupsView	View Component	100	Hierarchical group tree visualization, group CRUD modals, device count aggregation
FirmwareView	View Component	130	Firmware repository table, drag-and-drop upload zone, lifecycle state management buttons
OTAJobsView	View Component	180	Job list table, new job creation modal, job pause/abort controls
OTAJobDetailView	View Component	140	Per-device progress list with real-time WebSocket updates, aggregate donut chart
TelemetryView	View Component	110	Device selector, real-time LineChart (Recharts), metric selector, historical query

Component / Hook	Type	Lines (approx.)	Primary Responsibility
UsersView	View Component	100	User table (Super Admin only), invite user modal, role assignment, deactivation
CreateJobModal	Modal Component	120	Multi-step OTA job creation: firmware select → target select → update type → confirm

Table 17 Table B.1: React Component Inventory — *IoTDashboard.jsx* (Total ~1,680 lines)

APPENDIX C

SYSTEM CONFIGURATION FILES

C.1 Environment Variable Template (.env.example)

```
# .env.example - Copy to .env and populate before deployment
# — PostgreSQL —
POSTGRES_PASSWORD=ChangeThis_StrongPassword_2024!

# — MongoDB —
MONGO_PASSWORD=ChangeThis_MongoPassword_2024!

# — JWT —
# Generate with: openssl rand -hex 32
JWT_SECRET_KEY=replace_with_64_hex_char_random_secret

# — Server —
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
CORS_ORIGINS=["http://localhost:3000","http://localhost:80"]

# — Firmware Storage —
FIRMWARE_STORAGE_PATH=/app/firmware_store

# — MQTT —
MQTT_BROKER=mosquitto
MQTT_PORT=1883
MQTT_BACKEND_PASSWORD=ChangeThis_BackendMQTTPass!
```

C.2 Nginx Reverse Proxy Configuration (nginx/nginx.conf)

```
worker_processes auto;

events { worker_connections 1024; }

http {
    include      /etc/nginx/mime.types;
    default_type application/octet-stream;
    sendfile     on;
    gzip         on;
    gzip_types   application/json application/javascript text/css text/plain;
```

```
upstream fastapi_backend {
    server backend:8000;
    keepalive 32;
}

server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;

    ssl_certificate      /etc/nginx/certs/server.crt;
    ssl_certificate_key  /etc/nginx/certs/server.key;
    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_ciphers           HIGH:!aNULL:!MD5;

    # — React SPA static files —————
    root    /usr/share/nginx/html;
    index  index.html;
    location / {
        try_files $uri $uri/ /index.html; # SPA client-side routing
    }

    # — FastAPI REST API proxy —————
    location /api/ {
        proxy_pass          http://fastapi_backend;
        proxy_http_version 1.1;
        proxy_set_header    Host                $host;
        proxy_set_header    X-Real-IP           $remote_addr;
        proxy_set_header    X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto  $scheme;
    }

    # — WebSocket upgrade for /api/v1/ws/ —————
    location /api/v1/ws/ {
        proxy_pass          http://fastapi_backend;
        proxy_http_version 1.1;
        proxy_set_header    Upgrade             $http_upgrade;
        proxy_set_header    Connection          "upgrade";
    }
}
```

```
        proxy_read_timeout 3600s;  
    }  
}  
}
```